Artificial Intelligence Review (2024) 57:312
https://doi.org/10.1007/s10462-024-10947-4

Cost optimization in edge computing: a survey

Liming Cao[1] · Tao Huo[2] · Shaobo Li[1] · Xingxing Zhang[1] · Yanchi Chen[3] · Guangzheng Lin[2] · Fengbin Wu[1] · Yihong Ling[2] · Yaxin Zhou[4] · Qun Xie[2]

Accepted: 10 September 2024 / Published online: 1 October 2024
© The Author(s) 2024

Abstract
The edge computing paradigm is becoming increasingly commercialized due to the widespread adoption of wireless communication technologies and the growing demand for compute-intensive mobile applications. Edge computing complements the cloud computing model by deploying computation, storage, and network resources to the edge locations of wireless access networks, empowering end devices to run resource-intensive applications. In order to promote the commercialization of edge computing, it is important to explore effective ways to reduce the cost of edge computing networks. This paper provides a comprehensive review of the research findings in recent years, offering a clear perspective on the research dynamics. This paper first recalls the architectural framework of edge computing. Then, the main optimization objectives and optimization methods are comprehensively described. Mainstream mathematical models for cost reduction are then shown in depth. The paper also discusses the methods used to evaluate the effectiveness. Then, typical examples of typical application scenarios for edge computing networks are examined in depth. Finally, the paper identifies some unresolved issues. We expect future research to make more attempts in these directions.

Keywords Artificial intelligence · Computation offloading · Edge computing · Internet of things · Resource scheduling

✉ Shaobo Li
lishaobo@gzu.edu.cn

[1] State Key Laboratory of Public Big Data, Guizhou University, Guiyang 550025, China

[2] Guizhou Baishan Cloud Technology Co., Ltd, Guiyang 550025, China

[3] School of Computer Science and Technology, China University of Mining and Technology, Xuzhou 221116, China

[4] Zhenjiang Water Works Co., Ltd, Zhenjiang 212000, China

1 Introduction

Computationally intensive mobile applications are springing up, including speech recognition, image processing, augmented reality (AR), virtual reality (VR), and large language models. However, implementing these applications on terminal devices has encountered significant challenges due to the limitation of terminal device resources. Although high-speed and highly reliable communication protocols enable end devices to access configurable resources on-demand with the help of cloud computing services, the communication delays between the user and the Cloud Service Center (CSC) are large due to the excessive physical distance. Although Mobile Cloud Computing (MCC) provides a resource-rich environment for mobile devices (MDs), users are not able to fully utilize its potential due to the distance, limited bandwidth, and seamless connectivity challenges between remote clouds (RCs) and MDs in the MCC model.

As an emerging solution, Edge Computing aims to deploy compute, storage, and network resources to the edge of the network and provide intelligent services directly at the edge to meet the challenges of specific applications regarding critical requirements such as latency, security, and privacy protection. Notably, "edge" refers to the pathway between the data source and the cloud data center. By introducing edge computing on this path, data transmission time can be significantly reduced, thus providing users with a faster, more secure, and personalized service experience. Edge servers can offer better quality of service (QoS) and lower latency even if they are not as good as cloud servers in computing power due to their location advantage of being close to end devices.

The development of edge computing can be divided into three phases: before 2015, as a technology paving period; from 2015 to 2019, a period of rapid growth; and in 2020 and beyond, a period of steady development. In the robust development period, edge computing is expected to achieve deep integration between the academic and industrial sectors, and it becomes critical to explore strategies to reduce network costs during its commercialization.

According to the OpenFog Consortium, there are significant differences between fog computing and edge computing: fog computing is designed to collaborate with the cloud, while edge computing is more independent of the cloud; fog computing adopts a multi-layered, hierarchical architectural design, compared to edge computing, which is typically confined to the third or fourth layer of the network; and in addition to computational power, fog computing also includes networking, storage, control, and acceleration capabilities. Although there exists a certain view that there is a clear distinction between edge computing and fog computing, more research suggests that the two concepts overlap in the research area and that the terms are interchangeable. The difference is mainly in the focus: edge computing is more concerned with the role and functionality of end devices. In contrast, fog computing focuses more on the role of the network infrastructure. Specifically, edge computing is primarily deployed at the edge of the internet close to the user's end, while fog computing is laid out at the edge of the Internet of Things (IoT) close to the physical device's end. Since the concept of Mobile Edge Computing (MEC) was first introduced by the European Telecommunications Standards Institute (ETSI) in 2014 (ETSI subsequently renamed MEC to Multi-Access Edge Computing), the mobile computing paradigm has undergone a significant shift from centralized Mobile Cloud Computing (MCC) to MEC. This change means that compute, storage and network resources can be centrally deployed at the edge of the mobile radio access network, thus providing an essential complement to

the cloud computing model, enabling mobile devices (MDs) to execute real-time, compute-intensive applications directly at the edge of the network, and significantly broadening the application scenarios and potential of mobile computing (Shi et al. 2016; Yu 2018; Shi et al. 2019; Escamilla-Ambrosio et al. 2018; Luo et al. 2021; Till et al. 2014; Alkaabi et al. 2024; Nayyer et al. 2019).

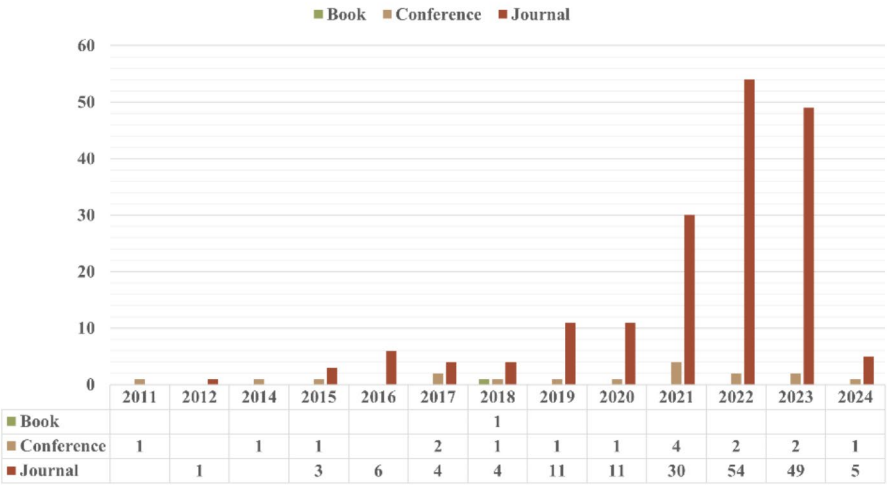
Edge computing is a transformative approach that brings computation and data storage closer to the source of data generation, resulting in several key strengths. It significantly reduces latency by processing data locally, which is crucial for real-time applications like autonomous vehicles and smart manufacturing. It also improves bandwidth efficiency by minimizing the amount of data that needs to be transmitted to centralized data centers, and enhances privacy and security by keeping sensitive information close to its source. Additionally, edge computing is highly scalable, supports a wide range of IoT devices, and offers resource optimization, energy efficiency, and flexibility in deployment across various environments.

Despite these benefits, edge computing faces several challenges. Managing a distributed network of edge devices can be complex, and while it enhances security in some ways, it also introduces new vulnerabilities across multiple endpoints. The limited processing power and storage of edge devices compared to centralized data centers can restrict their functionality. Interoperability issues, inconsistent data processing, dependency on network connectivity, and the high initial investment required for setting up edge infrastructures are additional concerns. Furthermore, the need for regular maintenance, the lack of industry standardization, and potential data duplication complicate the adoption and efficiency of edge computing.

To improve edge computing, several strategic changes are recommended. Enhancing security protocols and developing industry standards can address security and interoperability challenges. Creating user-friendly management tools and providing specialized training for IT professionals can simplify the management of distributed systems. Increasing the capabilities of edge devices and improving data analytics at the edge can enhance functionality and decision-making. Collaboration between vendors, the development of remote management solutions, and exploring cost reduction strategies can further support the widespread adoption of edge computing. Finally, focusing on energy-efficient technologies for edge devices can reduce environmental impact and operational costs, ensuring that edge computing continues to evolve as a powerful complement to traditional cloud computing (Luo et al. 2021; Hu 2021; Li 2022; Diao 2019).

This paper presents a comprehensive survey of state-of-the-art research focusing on reducing the cost of edge computing networks. Figure 1 shows the distribution of papers surveyed by year and source. Specifically, the focus of this article is five-fold.

- Edge Computing Paradigm (Section 2): First, a three-tier edge computing architecture including the Thing Layer, the Edge Layer, and the Cloud Layer is introduced, and then four different collaborations based on the three-tier architecture are presented, i.e., Thing-Edge Collaboration, Edge-Cloud Collaboration, Edge-Edge Collaboration, and Thing-Edge-Cloud Collaboration.
- Optimization Goals (Section 3): The main optimization goals are described in detail: Energy, Server Deployment, Storage, Communication, Task Execution Time, etc., and the methods used for each optimization goal are summarized.



	2011	2012	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024
Book							1						
Conference	1		1	1		2	1	1	1	4	2	2	1
Journal		1		3	6	4	4	11	11	30	54	49	5

Fig. 1 Distribution of papers surveyed by year and source

- Problem Modeling (Section 4): The results of recent years in modeling the problem of reducing the cost of edge computing networks as a mathematical model are presented, and the problems of Markov Decision Process (MDP), Game Theory (GT), and Integer Nonlinear Programming (INLP), etc., are described in detail, and the mathematical principles are elaborated.
- Evaluation methods (Section 5): The Data Set and Experimental Environment in which the experiments were conducted are presented, and Evaluation Metrics from existing work, such as Response Time, Resource Utilization, Task Offloading Rate, Security, etc., are summarized.
- Case Study (Section 6): Typical cases of several typical application scenarios are studied such as UAV, CAV, Industrial IoT etc. In-depth study is made on how they build mathematical models after determining the optimization objectives, and how to evaluate the effect.
- Open Issues (Section 7): Several open research challenges that have not been well explored, such as Data Security, Real Data, Equipment Radiation, Sustainable Development, etc., are presented.

To help the readers have a comprehensive picture of the structure of this survey, Fig. 2 outlines the organization of the survey, and Table 1 lists the acronyms that will be frequently used in the survey.

2 Edge computing paradigm

This section introduces the edge computing paradigm, and we outline the architecture of edge computing, as shown in Fig. 3. This architecture is divided into three layers: the thing layer is located at the bottom and interacts directly with the user or device; the edge layer resides in the middle, between the user and the cloud layer, and is responsible for handling

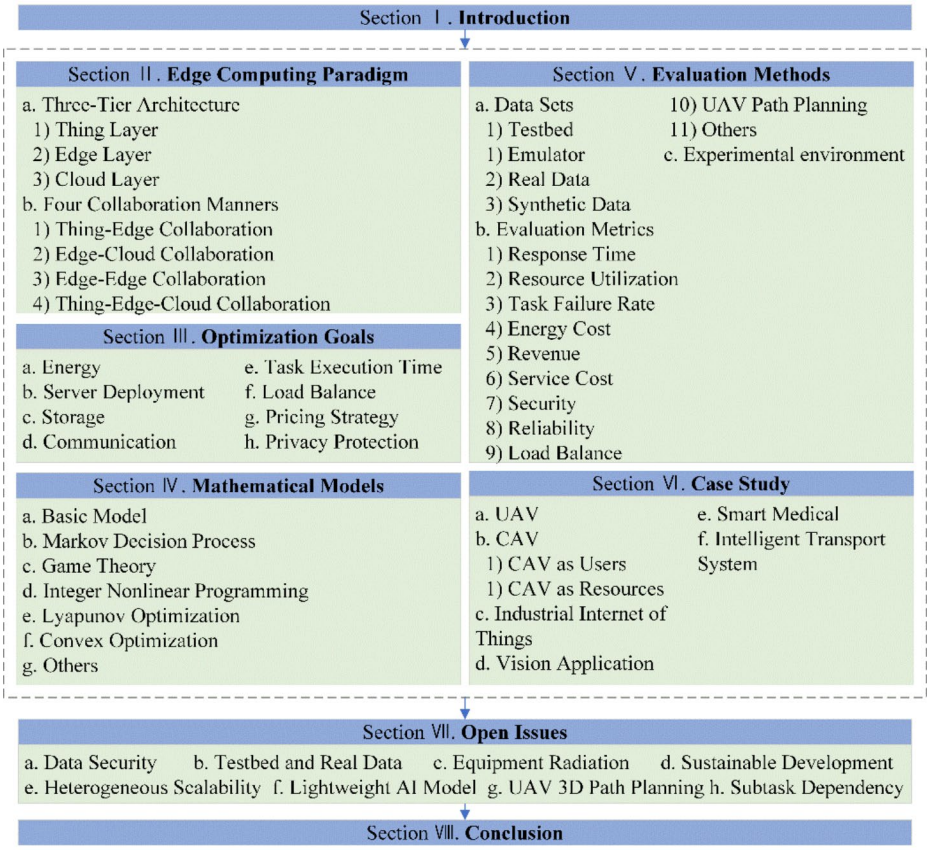


Fig. 2 Road map of the survey

computation and storage tasks close to the user's end, to reduce latency; and the cloud layer is located at the top, which possesses powerful computational processing and large-scale storage capabilities. Based on this architecture, this paper will also detail four collaboration manners: thing-edge collaboration, edge-cloud collaboration, edge-edge collaboration, and thing-edge-cloud collaboration. Edge computing can adapt to various requirements and network conditions through these collaboration manners to effectively perform diverse tasks. Using these four collaboration methods can reduce operating costs and increase revenue.

2.1 Three-tier architecture

2.1.1 Thing layer

The thing layer, also known as the user layer, forms the base layer of the edge computing architecture. This layer covers numerous types of end devices, such as smartphones, unmanned aerial vehicles (UAVs), connected and autonomous vehicles (CAVs), sensors, and various IoT devices. These devices, also known as MDs or mobile users (MUs), are not only consumers of data but also producers of data. These devices usually have certain

Table 1 Summary of acronyms frequently used in the paper

Acronym	Definition	Acronym	Definition
AP	Access Point	LSTM	Long Short-Term Memory
AoI	Age of Information	MD	mobile device
ADMM	alternating direction multiplier method	MU	mobile user
CSC	Cloud Service Center	MDP	Markov Decision Processes
CAV	connected and autonomous vehicle	MINLP	Mixed Integer Nonlinear Programming
CMDP	Constrained Markov Decision Process	NS-2	Network Simulator 2
DNN	Deep Neural Network	NS-3	Network Simulator 3
DQN	Deep Q-Network	NSGA	Non-dominated Sorting Genetic Algorithm
DRL	Deep Reinforcement Learning	PSO	particle swarm optimization
DDPG	Deep Deterministic Policy Gradients	POSG	Partially Observable Stochastic Game
D3RQN	Duel Dual Deep Recurrent Q-Network	POTAM	parallel optimal task allocation mechanism
FL	Federated Learning	PAMDP	parameterized action Markov decision process
GT	Game Theory	QoS	quality of service
IoT	Internet of Things	QoE	quality of experience
INLP	Integer Nonlinear Programming	UAV	unmanned aerial vehicle
IndRNN	Independent Recurrent Neural Network	VM	Virtual Machine

computing, storage, and communication capabilities. They can decide whether to offload data processing tasks to the upper layer architecture based on quality of experience (QoE) and QoS requirements. With the explosive growth in the number of end devices, the cost management of edge computing networks is facing unprecedented challenges. To this end, researchers have conducted numerous studies aimed at balancing the relationship between QoS and operational cost to achieve efficient resource utilization and optimize the overall performance of edge computing networks (Luo et al. 2021; Xu 2019; Sun et al. 2018; Li 2020; Liao 2020).

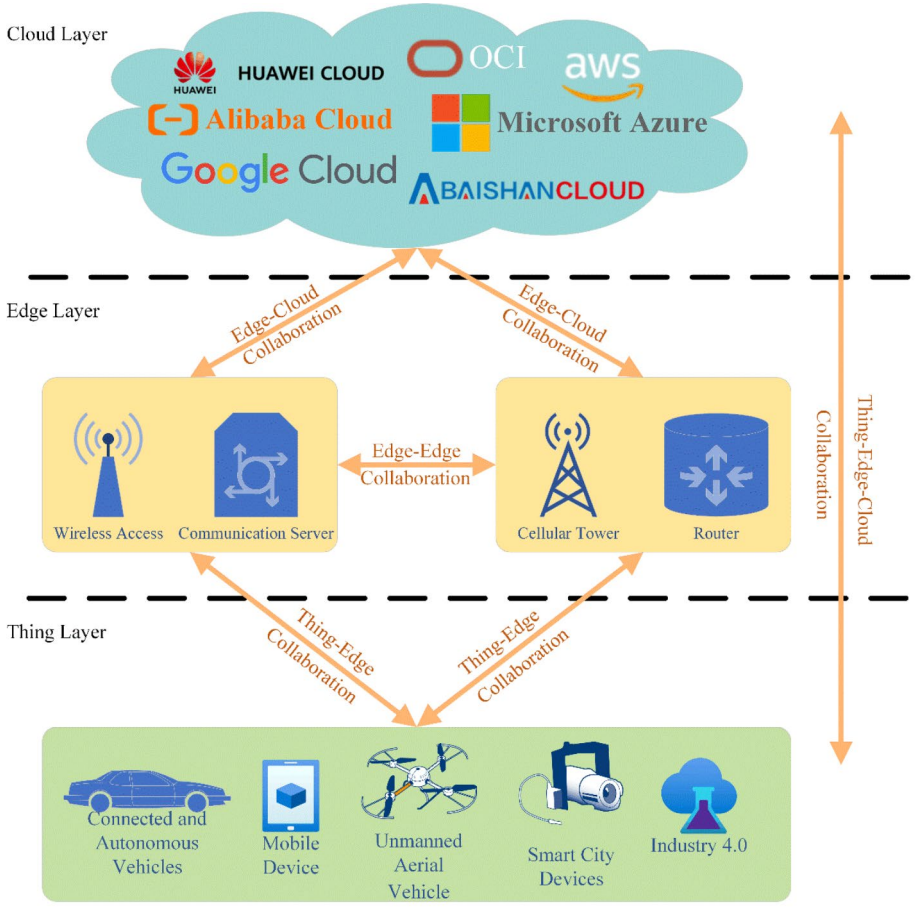


Fig. 3 Three-tier architecture and four collaboration manners in edge computing

2.1.2 Edge layer

The edge layer, located at the center of the three-tier architecture, connects the thing layer and the cloud layer. It comprises various network devices, including routers, switches, gateways, cell towers, and roadside units (RSUs). This layer provides computing and storage capabilities beyond the thing layer and establishes connectivity with end devices through wireless access technologies. The edge layer can efficiently execute data processing tasks from the thing layer through resource scheduling techniques. Thanks to its low latency, some cloud layer services can be placed in the edge layer. At the same time, the edge layer can further offload compute-intensive tasks to the cloud layer. This can balance the load and optimize the QoE. The fact that the operational costs of edge computing networks are mainly concentrated at the edge layer has motivated researchers to conduct in-depth studies and research from multiple dimensions, including resource scheduling techniques, task offloading strategies, service provisioning schemes, etc. Through these researches, we aim to explore the potential of cost optimization and improve the overall efficiency and economy

of edge computing networks, and these efforts are crucial to driving edge computing networks towards a more mature and sustainable direction (Luo et al. 2021; Kaur and Batth 2021; Kong 2023; Huda 2022).

2.1.3 Cloud layer

The cloud layer constitutes the top level of the edge computing architecture, formed based on the existing cloud computing infrastructure. It is connected to the edge layer through the core network (CN, also known as the backbone network). This layer provides vast computing, storage, and network resources for edge computing. It demonstrates superior processing power and makes it an ideal place to take on complex applications and handle large-scale data tasks. Thanks to its top-level positioning and powerful computing, the cloud layer can perform advanced functions such as dynamic deployment of resources, multi-network collaboration, and decision support, and it applies a holistic view to the edge computing network, aiming to minimize overall costs. The edge computing architecture, through its multi-tiered and collaborative working mechanism, effectively integrates the robust processing capabilities of cloud computing with the geographical advantages of being closer to users inherent in edge computing. This integration aims to enhance the overall performance and cost-efficiency of the edge computing network. These capabilities and resources in the cloud layer provide a robust support platform for the edge computing network, ensuring that even the most complex and data-intensive application scenarios can be processed effectively and efficiently (Xue 2023; Iftikhar 2023; Maray and Shuja 2022; Cruz 2023).

2.2 Four collaboration manners

2.2.1 Thing-edge collaboration

Thing-Edge collaboration refers to the direct interaction between edge nodes and end devices that work together to accomplish computation and data processing tasks. This collaboration mode utilizes the powerful computing and storage capabilities of edge nodes, enabling end devices to undertake more complex and large tasks. In addition, Thing-Edge collaboration also covers the management and optimization of end devices resources, and ensures that end devices participation in edge computing tasks is more efficient and effective through intelligent scheduling and resource allocation. This type of collaboration not only improves the working efficiency of end devices but also extends the service life of end devices by sharing the computational load and improving the operational efficiency of the entire edge computing network (Iftikhar 2023; Maray and Shuja 2022; Mao et al. 2017; Feng 2022; Kong et al. 2022).

2.2.2 Edge-cloud collaboration

Edge-Cloud collaboration refers to the collaborative work between edge nodes and cloud servers, where the edge nodes collect and pre-process data and then transfer some of the data to cloud servers for further computation and storage. Edge-Cloud collaboration can fully utilize the computing and storage resources of the edge nodes and reduce the dependence on the cloud layer while enjoying the powerful computing and storage capabilities of the cloud

layer. In addition, the significant response latency of cloud computing cannot meet the QoE of users, while the cloud layer can deploy part of the services or tasks of the cloud center to the edge, reducing data transmission and latency and improving response speed and QoE (Kong 2023; Cruz 2023; Moura and Hutchison 2019).

2.2.3 Edge-edge collaboration

Edge-Edge collaboration describes the direct communication and cooperation between multiple edge nodes within the edge layer to collaborate on data processing tasks. Edge-Edge collaboration facilitates load balancing and task allocation among edge nodes and significantly improves the overall performance and efficiency of edge computing networks. In general, the Edge-Edge collaboration approach is not used alone; it is usually combined with Thing-Edge collaboration or thing-cloud-edge collaboration to realize more flexible and efficient edge computing solutions. This cross-node collaboration strengthens the collaborative processing capability of the edge computing network and ensures a more balanced and efficient resource utilization in the face of complex computing tasks (Xue 2023; Abreha 2022; Fazeldehkordi and Grønli 2022).

2.2.4 Thing-edge-cloud collaboration

Thing-Edge-Cloud collaboration refers to the collaboration between edge nodes, cloud servers, and end devices, aiming to complete complex computing tasks and data processing work jointly. This collaboration mode can flexibly and dynamically allocate tasks and resources according to the current demand and resource conditions, thus realizing the collaborative enhancement of the overall performance of the edge computing network. Thing-Edge-Cloud collaboration not only maximizes the computing and storage potential of the edge and cloud layers but also ensures the effective participation and contribution of end devices. With the increasing growth of smart devices and their resource-intensive applications, it is difficult to satisfy all service demands by relying solely on resources at the edge layer, making thing-edge-cloud collaboration an increasingly critical solution to meet this challenge (Iftikhar 2023; Cruz 2023; Wang 2023; Ning 2024).

3 Optimization goals

The cost associated with edge computing encompasses a broad spectrum, and in recent years, researchers have devised many strategies to optimize these costs for varying objectives. Broadly speaking, these objectives can be categorized into hardware cost and operational cost. Hardware costs, which are typically one-off or periodic, offer limited scope for optimization once initial investments are made. In contrast, operational cost presents a broader array of opportunities for ongoing optimization, enabling continuous cost reduction. While hardware cost constitutes a significant portion of the initial investment, operational cost profoundly impacts the total cost over the long term. This is because operational cost accumulates over time, making their optimization an essential strategy for significant cost reductions in the long run. Operational costs generally include maintenance and management expenses, energy, communication, etc. In the following sections, we delve into the

main optimization directions for energy cost, server deployment cost, storage cost, communication cost, and task execution times, providing a comprehensive summary of each optimization objective. The outline of optimization goals is shown in Fig. 4 and is described in detail below.

3.1 Energy

In the actual operation and maintenance of an edge computing network, the cost of energy consumption often accounts for the majority of the total cost. For this reason, a considerable amount of research is dedicated to reducing this cost. A comparison of papers focusing on Energy Cost is presented in Table 2.

Several studies have optimized the energy consumption of edge servers. Hosseinzadeh (2021) introduced three Dynamic Voltage and Frequency Scaling (DVFS) in a mobile edge computing network, which can be dynamically adjusted according to the workload and reduce energy consumption. Zhou et al. (2022) proposed a computation offloading method based on GT in an edge computing network using cloud-edge collaboration. This algorithm enables cloud servers with a large number of computing tasks to offload more computing tasks to edge servers with idle computing resources to reduce their own pressure and energy consumption.

The flexibility and low cost of UAVs have made them popular among researchers. However, the limited battery capacity of UAVs is the biggest nuisance that restricts the application of UAVs in edge computing networks. Miao (2023) used UAVs as edge servers in mobile edge computing and proposed an optimization algorithm with double-loop iteration. This algorithm can minimize the flight path length of UAVs and reduce energy consumption. Zhao (2021) not only considered offloading part of the computing tasks of industrial vehicles to an industrial mobile edge computing network, but also considered using UAVs to assist in offloading computing tasks. They proposed an optimal task allocation algorithm called MITA, which can reduce the overall energy consumption of the system.

Smart mobile devices have exploded in recent years, but limited by battery capacity and computing power, many user needs cannot be met. Edge computing networks break this dilemma and reduce the energy burden on user devices. Wang et al. (2023) proposed a time series prediction algorithm based on IndRNN and LSTM to predict link quality accurately. Combined with a distributed edge computing server task scheduling model based on GT, it can reduce system and user energy consumption. In the field of collaborative intrusion

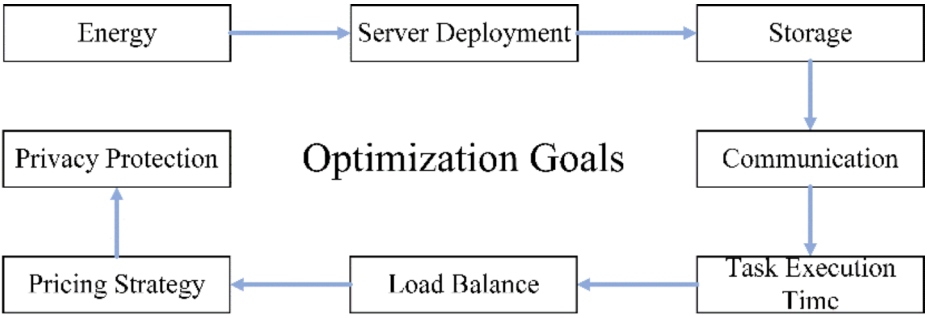


Fig. 4 Outline of the optimization goals

Table 2 Comparison of Papers focusing on Energy cost

Paper	Device	Collaboration Manner	Method	Paradigm
(Zhao 2022)	MD	Thing-Edge-Cloud	DQN	Mobile Edge Computing
(He et al. 2022)	Edge Server (ES)	Thing-Edge-Cloud	Lagrange	Mobile Edge Computing
(Li 2022)	ES	Thing-Edge-Cloud	DNN	Mobile Edge Computing
(Wang et al. 2023)	ES	Thing-Edge	GT; Time Series Neural Network	Mobile Edge Computing
(Tang 2022)	ES	Thing-Edge-Cloud	D3RQN	Serverless Edge Computing
(Zhou et al. 2022)	ES	Edge-Cloud	Stackelberg Game	Edge Computing
(Zhou et al. 2023)	ES、UAV	Thing-Edge-Cloud	GDISA	Mobile Edge Computing
(Chu et al. 2023)	MD	Thing-Edge-Cloud	Lagrange Dual Decomposition Theory	Multi-Access Edge Computing
(Deng et al. 2021)	User Equipment (UE)	Thing-Edge-Cloud	Branch and Boundary Algorithm	Edge Computing
(Feng et al. 2021)	MD	Thing-Edge-Cloud	CDCCO	Mobile Edge Computing
(Zhong et al. 2022)	ES	Thing-Edge	ADMM	Mobile Edge Computing
(Liao 2023)	ES、MD	Thing-Edge-Cloud	DRLCO; DDPG	Mobile Edge Computing
(Liu 2022)	ES	Thing-Edge-Cloud	Convex Optimization; Lagrange	Multi-Access Edge Computing
(Lan 2022)	ES	Thing-Edge-Cloud	DAG	Multi-Access Edge Computing
(Wang et al. 2022)	ES	Thing-Edge-Cloud	Lagrange	Mobile Edge Computing
(Hosseinzadeh 2021)	ES	Thing-Edge-Cloud	Butterfly Optimization Algorithm (BOA)	Mobile Edge Computing
(Kong 2022)	Base Station (BS)	Thing-Edge-Cloud	DDPG	Mobile Edge Computing
(Teng et al. 2022)	ES	Thing-Edge	Non-Cooperative Game; Greedy Strategy	Mobile Edge Computing
(Zhao et al. 2021)	ES	Thing-Edge-Cloud	Energy Harvesting Equipment	Mobile Edge Computing
(Hao 2021)	ES	Thing-Edge-Cloud	Green Energy Harvesting Equipment	Edge Computing
(Chen 2022)	ES	Thing-Edge-Cloud	Energy Harvesting Equipment	Mobile Edge Computing
(Sun et al. 2022)	CAV	Thing-Edge-Cloud	FL	Mobile Edge Computing
(Guo 2022)	ES	Thing-Edge-Cloud	FL; DQN	Mobile Edge Computing
(Miao 2023)	UAV	Thing-Edge	Double Loop Iterative Optimization Algorithm	Mobile Edge Computing
(Zhang 2023)	ES; UAV	Thing-Edge-Cloud	DRL	Multi-Access Edge Computing
(Lin 2022)	MD; UAV	Thing-Edge-Cloud	Lyapunov Optimization	Mobile Edge Computing
(Zhao 2021)	UAV; Industrial Vehicle	Thing-Edge-Cloud	Minimum Incremental Task Allocation Algorithm	Mobile Edge Computing

Table 2 (continued)

Paper	Device	Collaboration Manner	Method	Paradigm
(Jayanetti et al. 2022)	ES	Edge-Cloud	DRL	Edge Computing
(Mahenge et al. 2022)	ES	Thing-Edge-Cloud	PSO; GWO	Mobile Edge Computing
(Huang et al. 2021)	ES; MD	Thing-Edge	Contract Theory; Prospect Theory	Mobile Edge Computing
(Gao et al. 2023)	MD	Thing-Edge-Cloud	Aggregate Game Theory	Mobile Edge Computing
(Zhu and Zhou 2021)	MD	Thing-Edge-Cloud	Whale Optimization Algorithm	Mobile Edge Computing

detection systems. Zhao (2022), studied offloading intrusion detection tasks to edge servers with better performance, thereby reducing users' energy consumption.

Several studies have been devoted to optimizing energy consumption from a holistic perspective. Liu (2022). proposed an algorithm based on mobility prediction, MCPO, in a multi-access edge computing network, which can effectively reduce energy consumption and also shows the advantage of energy saving in the long-term operation of the system. Wang et al. (2022) proposed the P-MCO algorithm in a distributed heterogeneous multi-layer mobile edge computing (HetMEC). By adjusting the offloading strategy based on price, this algorithm can reduce energy consumption by about 30%.

Some studies have combined multiple optimization objectives to reduce energy consumption while optimizing other metrics. Deng et al. (2021) comprehensively considers the three factors of time delay, energy consumption, and price, and can effectively balance the computing load and optimize resource allocation through an improved branch and bound algorithm based on linear relaxation and an improved PSO based on 0–1 weight. This article minimizes the weighted costs of the three factors under the constraints of meeting the user's personalized needs. Mahenge et al. (2022). proposed an energy-efficient task offloading strategy (ETOS), which makes full use of the advantages of PSO and gray wolf optimization (GWO). This strategy reduces energy consumption when the task execution speed differs from the optimal strategy. In addition, as the number of tasks increases, the proposed ETOS reduces energy consumption and lowers the overall task execution time compared to the baseline algorithm, proving its superiority in large-scale task scenarios.

In addition to reducing the energy consumption of each device in the edge computing network, there are some studies that are thinking about saving energy through energy-harvesting devices. Zhao et al. (2021). introduced energy harvesting (EH) devices in a distributed mobile edge computing network to reduce energy waste. This paper achieves the goal of minimizing energy consumption in MEC while meeting the QoS requirements of IoT devices.

Compared to fossil energy, green energy is environmentally friendly and has a price advantage. Green energy can also be deployed close to the edge nodes, which enhances the energy autonomy of the edge nodes. Hao (2021) introduced devices that can harvest green energy (such as wind or solar energy) in an edge computing network. This paper also proposes a scheduling heuristic method based on the clustering method for transmitting green energy, which can reduce energy transmission losses by 20–80%. Chen (2022) studied a hybrid energy-driven mobile edge computing network using digital twins, introduced green

energy collection technology, and deployed energy collection devices in each edge server. The first is to minimize the linear combination of delay and energy consumption, the second is to minimize delay under energy consumption constraints, and the third is to minimize energy consumption under delay constraints.

Summary The principal strategies for reducing energy costs in an edge computing network focus on lowering device energy consumption, energy harvesting, and using green energy sources. Current research primarily targets optimizing intermediate processes in edge computing, such as task offloading, resource allocation, task scheduling, and data caching. These efforts aim to enhance energy efficiency from a software perspective, thereby reducing device energy consumption. Additionally, some researchers explore hardware-based approaches, including adjusting device power settings and optimizing UAV flight paths to minimize energy usage. Another group of researchers seeks to reduce energy consumption by lowering certain performance requirements to maintain an acceptable QoE, thus decreasing the energy demands on devices. Furthermore, some researchers have begun to think about energy and are introducing energy recovery devices and green energy to reduce energy costs. From the content distribution perspective, researchers mainly focus on devices such as ESs, MDs, and UAVs. The edge computing paradigms of primary concern are mobile edge computing and multi-access edge computing. The solutions primarily include intelligent optimization algorithms, reinforcement learning, neural networks, etc.

3.2 Server deployment

The deployment of servers significantly impacts the overall cost of the edge computing network. Expenses related to the transportation, installation, and maintenance of servers, as well as the costs associated with network fault tolerance, communication, and reliability, are all influenced by how servers are deployed. Consequently, researchers have explored various strategies to reduce deployment costs. A comparison of papers focusing on Server Deployment is presented in Table 3.

The location of the servers directly affects the service coverage of the edge computing network. It also affects the cost of transportation, installation, and maintenance. In addition, it affects the number of servers to be installed, the quality of service, etc. Wang (2022) proposed an edge server deployment optimization model and a fault-tolerant server deployment scheme based on the binary gray wolf genetic policy algorithm. In smart manufacturing factories, the model is able to achieve optimal deployment with reduced deployment cost and load balancing, as well as server deployment under fault-tolerant conditions. Cong et al. (2022) propose a mobile resource-sharing framework called EdgeGO. This framework can realize the effect of the traditional edge computing paradigm by deploying fewer servers, and it can reduce the cost of deploying edge computing network servers in a 6G environment by nearly 30%.

In addition to traditional edge servers, UAVs and CAVs are not only users in the edge computing network, but can also be used as servers to share computing resources. Zhang (2023) noted that it would be costly to provide edge computing service in remote areas or complex terrain but that the cost of deploying servers in these environments could be effectively reduced by introducing UAVs as aerial base stations. Ma et al. (2021) utilizes

Table 3 Comparison of Papers focusing on server Deployment

Paper	Content	Collaboration Manner	Method	Paradigm
(Huang et al. 2022)	Location of Deployment	Thing-Edge-Cloud	Gibbs Sampling	Mobile Edge Computing
(Zhang 2023)	Scope of Service	Thing-Edge	DRL	Multi-Access Edge Computing
(Wang 2022)	Location of Deployment	Thing-Edge	Binary Grey Wolf Genetic Strategy Algorithm	Edge Computing
(Cong et al. 2022)	Number of servers	Thing-Edge-Cloud	EdgeGo Framework	Mobile Edge Computing
(Ma et al. 2021)	Rent Parked Vehicle	Thing-Edge; Edge-Edge	Local Task Scheduling Strategy	Vehicle Edge Computing
(Deng 2021)	Location of Deployment	Thing-Edge	IDA-4ReE	Mobile Edge Computing

parked vehicles as virtual edge servers, effectively solving the physical edge server resource bottleneck problem. Without additional investment, the method also significantly extends the service scope of the edge computing network.

Summary Key strategies to reduce the cost of deploying servers for edge computing include reducing the number of servers, expanding the scope of service, and leveraging MDs. Some researchers aim to reduce the number of servers while ensuring a satisfactory user experience. Others focus on optimizing the deployment locations of servers to serve more users, thereby reducing the need for additional servers. To address challenges posed by geographic factors, such as deployment difficulties and maintenance issues, researchers have proposed using communication relay devices to extend the service range of servers. This approach allows server deployment in geographically optimal locations, reducing deployment costs.

3.3 Storage

Storage resources are cost-intensive and optimizing their utilization can reduce the need for expensive storage hardware, thereby lowering overall operational costs. Additionally, efficient use of storage resources can enhance the efficiency of data processing and access, enabling faster response to user requests.

Avoiding duplicate storage is an effective way to improve storage resource utilization. Wang (2021) uses blockchain technology to manage data decentralized, reducing the need for secondary repetitive storage of centralized data. The powerful anonymity mechanism

of blockchain also avoids the need to store user identity information, and users can interact with different edge servers by registering only once at the Key Generation Center (KGC), which reduces the problem of storing information caused by repeated registration. In addition, different servers do not have to store keys repeatedly, and all these advantages reduce the storage resource consumption of edge computing networks.

An excellent storage strategy can fully utilize the storage resources in the edge computing network and avoid wasted and unbalanced storage resources. Huang et al. (2022) describes an approach to reduce storage costs in a tiered mobile edge computing environment. The proposed iterative service deployment algorithm using Gibbs sampling considers the diversity of edge nodes and the different requirements of services to decide which services should be cached in each edge node and how many resources will be used. The approach ensures efficient utilization of storage resources while meeting service-specific requirements. Similarly, Yan et al. (2021) minimizes the storage cost of edge servers with limited cache storage capacity by selecting appropriate service caching policies.

Summary The primary strategies for optimizing storage resource consumption include avoiding redundant storage and optimizing storage strategy. Some researchers focus on leveraging the advantages of blockchain technology to develop methods for one-time registration information storage. This approach allows users to register just once and interact with various edge servers, simplifying the user experience and preventing the duplication of registration information, thereby reducing storage resource consumption. Additionally, there is a subset of researchers who focus on storage strategies by choosing where data and services are cached to reach as many users as possible. This effectively minimizes storage resource usage and lowers storage costs.

3.4 Communication

In edge computing, communication resources are often scarce, particularly in networks with a high density of devices. Congested network channels can significantly degrade the performance of edge computing networks. Consequently, reducing communication resource consumption has become a focal point for many researchers. A comparison of papers focusing on Communication Costs is presented in Table 4.

Reducing the overhead of communication resources through optimization methods is a straightforward and effective approach. Wang (2021) proposed a blockchain-assisted switching authentication and key agreement scheme for Intelligent Telemedicine Systems (ITS) in a multi-server edge computing environment. The scheme achieves low overhead interaction with servers anytime, anywhere, and the scheme has minimal overhead in switching servers. In addition, the scheme remains effective in reducing communication overhead as the number of edge servers increases. Furthermore, Hosseinzadeh (2021) investigated data placement strategies in order to reduce the communication consumption associated with replacing data blocks. This strategy avoids frequent data replacement and thus reduces the overhead of communication resources.

In a congested edge computing network, too many communication requests can lead to a strain on communication resources. We can try to mitigate this situation by reducing the communication requirements between devices. Liu (2023) proposed an experience-driven

Table 4 Comparison of Papers focusing on communication costs

Paper	Content	Collaboration Manner	Method	Paradigm
(Zhou et al. 2023)	Communication Distance	Thing-Edge-Cloud	GDISA	Mobile Edge Computing
(Hosseinzadeh 2021)	Communication Distance	Thing-Edge-Cloud	BOA	Mobile Edge Computing
(Ma et al. 2021)	Communication Distance; Data Transfer Volume	Thing-Edge-Cloud	Driving Trajectory Prediction Model	Mobile Edge Computing
(Wang 2021)	Data Transfer Volume	Thing-Edge-Cloud	Blockchain	Edge Computing
(Hu 2021)	Communication Distance; Data Transfer Volume	Thing-Edge-Cloud	Blockchain	Edge Computing
(Guo 2022)	Network Topology	Thing-Edge-Cloud	NSGA-III	Mobile Edge Computing
(Zhu et al. 2022)	Communication Loss	Thing-Edge-Cloud	Greedy Algorithm	Multi-Access Edge Computing
(Liu 2023)	Communication Need	Edge-Cloud	Asynchronous Federated Learning (AFL)	Edge Computing
(Liu et al. 2022)	Communication Need	Thing-Edge-Cloud	FL	Mobile Edge Computing
(Naouri et al. 2021)	Task Offloading	Thing-Edge-Cloud	Greedy Task Graph Partition Offloading Algorithm (GTGP)	Mobile Edge Computing
(Chai et al. 2023)	Communication Distance	Thing-Edge-Cloud	A-PPO	Mobile Edge Computing
(Miao et al. 2022)	Communication Distance	Edge-Cloud	Pond Scheduler	Edge Computing

based solution. The solution can intelligently select the working nodes participating in each round of FL based on resource constraints, thus reducing communication consumption. In addition, in terms of a global update to edge node communication consumption, the paper introduces a binary variable to indicate whether or not to involve locally updated models in the global update, which further optimizes communication consumption. Naouri et al.

(2021) considered the current situation of communication congestion in the cloud layer and proposed a three-layer task offloading framework called DCC, which categorizes tasks into those with high computational demands and those with low computational and high communication costs. Tasks with high computational demands are offloaded to the cloud layer for processing. In contrast, low-computation, high-communication-cost tasks are executed at the edge layer, thus avoiding the transmission of large amounts of data to the cloud. Also, a greedy task graph partitioning offloading algorithm is introduced. This algorithm uses a greedy optimization method to reduce the communication cost of tasks. When combined with the above framework, it effectively reduces the communication consumption of the edge computing network.

Wireless communications are susceptible to inefficiencies due to interference from the external environment. Excessive distance or interference can seriously reduce the availability of communication resources. Zhu et al. (2022) considered the problem of signal attenuation caused by building occlusion in the 5G network and proposed the use of Intelligent Reflective Surfaces (IRSs) to mitigate this problem, which effectively improves the communication performance of urban city 5G (B5G) vehicular networking. Considering the vast communication consumption caused by massive communication rounds during traditional FL, Miao et al. (2022) investigated how to reduce communication costs in large-scale geographically distributed simulations.

Summary The main directions for optimizing communication resource consumption include streamlining communication content, shortening communication distances, and reducing communication demands. Some researchers have managed to decrease the size of data packets transmitted, thereby lowering communication resource usage by optimizing the content of communications. Many focus on optimizing the deployment locations of resources and the placement of data to prevent long-distance, large-scale data transfers that occupy network links and lead to widespread communication congestion. Additionally, some researchers have refined task-offloading strategies by balancing computation execution time against task-offloading time. In scenarios where channel congestion leads to prolonged offloading times, they prioritize processing tasks locally. This approach is particularly effective in FL, where an intelligent selection of users for aggregate updates can significantly reduce the immense communication resource consumption caused by numerous communication rounds. From a content perspective, researchers primarily target mobile edge computing networks and multi-access edge computing paradigms, focusing on reducing communication distances and data transmission volumes through intelligent optimization algorithms and reinforcement learning techniques.

3.5 Task execution time

Reducing task execution time has long been a concern for researchers, as computational tasks that occupy resources for extended periods can significantly degrade the performance and resource utilization of edge computing networks, increasing latency, diminishing user experience, and driving up overall costs. For mobile edge computing, in particular, longer task execution times entail more frequent server handovers, thereby consuming additional resources. A comparison of papers focusing on Task Execution Time is presented in Table 5.

The arithmetic power of an edge node is fixed, and allocating appropriate computational resources to a computational task is an important way to reduce the execution time of the task. Da and Costa (2023) designed an efficient VC task scheduler called MARINA based on approximation heuristics and resource prediction. The idea is to find the Pareto set of task execution time and task deadline to minimize the task execution time. Liang (2021) proposed a joint reordering and frequency scaling (JRFS) algorithm. The algorithm considers the uneven arrival and priority constraints of computing tasks for MDs. It minimizes the computation task execution time by adjusting the edge server computation frequency and task-server mapping relationship.

Parallel computing allows multiple edge nodes to serve a computational task at the same time, reducing the execution time of the task. Jayanetti et al. (2022) designed a workflow scheduling framework using DRL. The framework can overcome the cross-resource constraints of coordinating workflows in distributed edge computing networks; in addition, the applied DRL can effectively reduce task execution time. Chen (2023) proposed an edge coverage scheduling algorithm (ECSA) based on a DAG. The algorithm employs the estimation of distribution algorithm (EDA) and graph random wandering algorithm and combines the vertex greedy policy and edge greedy policy in the task scheduling strategy. The algorithm can improve the system parallelism and reduce the task execution time.

Summary The key strategy for reducing task execution time is optimal resource matching. This involves offloading computing tasks to the most suitable devices via scheduling algorithms and allocating the most appropriate resources through resource scheduling strategies, ensuring the right tasks run on the right resources. This approach takes full advantage of the unique characteristics of each task and the strengths of different resources. Therefore,

Table 5 Comparison of Papers focusing on Task execution time

Paper	Collaboration Manner	Method	Paradigm
(Jayanetti et al. 2022)	Thing-Edge-Cloud	DRL	Edge Computing
(Zhou et al. 2023)	Thing-Edge-Cloud	GDISA	Mobile Edge Computing
(Da and Costa 2023)	Thing-Edge-Cloud	MARINA	Vehicle Edge Computing
(Chen 2023)	Thing-Edge	ECSA	Edge Computing
(Liang 2021)	Thing-Edge-Cloud	JRFS	Mobile Edge Computing
(Xiao et al. 2022)	Thing-Edge; Edge-Edge	Adaptive Harmony Algorithm	Vehicle Edge Computing
(Liu 2022)	Thing-Edge-Cloud	Bilinear Pairing; BLS-HLA	Edge Computing
(Zhao 2021)	Thing-Edge-Cloud	Minimum Incremental Task Allocation Algorithm	Mobile Edge Computing
(Wang 2021)	Thing-Edge-Cloud	Blockchain	Multi-Server Edge Computing

sophisticated scheduling algorithms, resource allocation strategies, and task partitioning techniques are indispensable.

3.6 Load balance

Implementing load balancing in edge computing networks can mitigate the risk of overloading individual servers, ensure balanced resource utilization, and reduce service latency. This, in turn, enhances the overall network's resource efficiency and elevates the system's reliability and stability.

When the load on the edge nodes is too high, we can migrate the tasks to other nodes by edge-to-edge collaboration. When performing task offloading, the node with a lower load should be selected. In Nayyer (2022), a task is bundled as a VM and is placed on a physical server (cloudlet). All the workloads are considered in the form of VMs. The collaborative approach keeps track of resource utilization based on the preferences given by the cloudlet owners for all member cloudlets, and the one with adequate resources with minimum latency is selected. When selecting a cloudlet for a new task, the optimal cloudlet has adequate resources with minimum latency. In the case of load balancing, a VM is selected for migration based on the required resources to be released. In case a cloudlet is in a critical state, the selection process of optimal VM starts with the filtration of eligible VMs by comparing the utilized resource matrix (URM) with the demand resource matrix (DRM). The VM that consumes the least amount of resources and can be a cloudlet with a normalized resource profile after migration is selected because it can complete the migration in a short period of time.

In cases where there is no edge-to-edge collaboration model or edge nodes can only be connected nearby, we need to balance the load by optimizing the location allocation of resources. Wang (2022) proposed an optimization method for edge server deployment in smart manufacturing environments to reduce cost and achieve load balancing. The method employs a binary-based gray wolf genetic strategy (BGWGS) algorithm to find a near-optimal edge server deployment scheme, which saves 10.97% of the total load and an average of 16.15% of the time during peak hours. Tang et al. (2023) proposed an adaptive dynamic deployment optimization method, Adapt-SD, based on the Adam optimization algorithm and weighted polling scheduling algorithm for the microservice deployment optimization problem. The algorithm considers the heterogeneity and resource constraints of edge servers and can achieve a more balanced workload.

Summary The primary approach to balancing load is matching computational tasks with low-load computing resources. Researchers are dedicated to distributing computing tasks to servers with lower loads to alleviate various issues caused by load imbalance. From the perspective of users, some researchers seek suitable edge servers to avoid the risks associated with mindlessly offloading computational tasks to high-load servers, such as prolonged task completion times and the potential for task offloading failures. From the perspective of edge servers, others employ load detection and task migration techniques to offload computational tasks from high-load servers to those with lower loads. This strategy reduces the operational cost of the edge computing network and enhances the user experience.

3.7 Pricing strategy

A well-designed pricing strategy can promote the sustainable development of edge computing networks. Meeting user demands while reducing operational costs enhances the overall operational efficiency, making the edge computing network more competitive.

A suitable pricing strategy can increase the revenue of edge computing networks. Mitsis et al. (2022) constructed a multi-leader multi-follower Stackelberg game that uses GT and reinforcement learning methods. The model is used to determine the optimal pricing strategy for MEC servers, which is determined by considering users' risk-aware decisions and the probability of overexploitation of the server's computational resources. The model allows us to maximize both the users' perceived service satisfaction and the MEC servers' profit. Yan et al. (2021) proposed a pricing-based service caching and task offloading mechanism. Specifically, the base station first decides on the service caching and announces the prices of different service programs, and then the user decides whether to offload the task based on the price. With this pricing strategy, the base station can effectively control the allocation of computational resources and regulate the users' task offloading behavior and service caching decisions. This strategy makes it possible to reduce operation costs while guaranteeing service quality and maximizing the base station's profit.

Pricing strategy can also reduce the operational costs of edge computing networks. Zhou et al. (2023) proposed a Reverse Auction-based Computational Offloading and Resource Allocation Mechanism (RACORAM), which constructs the computational offloading and resource allocation problem as a MINLP. The primary objective of the paper is to decrease the cost of the CSC. The CSC acts as an auctioneer, recruiting edge server owners to participate in the computation offloading process and incentivizing their participation through a reverse auction. This mechanism helps to effectively reduce operational costs while meeting the QoS and user satisfaction constraints. Gao et al. (2023) proposed a dynamic pricing strategy to optimize task segmentation and offloading under task processing latency and energy consumption constraints. The pricing of this strategy is based on the offloading of tasks at different time slots, which can be realized to reduce the operation cost while meeting the QoS requirements.

Summary The primary approaches to optimizing pricing strategies include applying GT and incorporating more constraints. As a mathematical model for analyzing and predicting decision-making in competitive contexts, GT offers a theoretical foundation for analyzing interactions between users and edge servers. By applying GT, it is possible to identify Nash equilibria in these interactions, leading many researchers to prioritize GT in optimizing pricing strategies to enhance network efficiency and reduce operational costs. Additionally, some researchers attempt to integrate more constraints into pricing strategies, making these strategies more comprehensively reflective of the diversity in network environments and user demands. Such approaches can more accurately match supply and demand, cater to different users' payment capabilities and needs, increase the likelihood of transactions, and lower long-term operational costs.

3.8 Privacy protection

Privacy protection measures are an indispensable part of edge computing networks, crucial for ensuring network security, enhancing user trust, and ensuring compliance. Privacy issues in MEC specifically include data privacy, location privacy, identity privacy, and computational offloading. However, edge computing devices often have limited resources, making it more challenging to implement cost-effective privacy protection measures in such environments. Additionally, maintaining network performance and response speed while ensuring privacy protection is a difficult balance to achieve. Therefore, exploring ways to implement low-cost privacy protection measures within edge computing networks is necessary (Ranaweera et al. 2021). A comparison of papers focusing on Privacy Protection is presented in Table 6.

For data privacy issues. Wang (2021) considers the risk of large-scale private data leakage in the Internet of Health Things (IoHT) and designs an instant messaging private data protection model. Terminal devices under the same LAN connect to the edge server via sockets without the need for cloud server forwarding, which reduces communication pressure, lowers the risk of private communication data exposure, and improves the security of private data. At the same time, the model also reduces the encryption and decryption computation of data, saving computational resources. Wang (2022) proposed Double Local Disturbance Local Difference Privacy (DLD-LDP) to protect data privacy and location privacy. Task participants use the DLD-LDP algorithm to locally perturb the eigenvalues of the sensed task data so that even if an attacker is able to obtain the data submitted by a task participant, he or she is unable to obtain the real data of the task participant. The use of blockchain technology can effectively reduce the possibility of Sybil attacks and malicious adversary attacks. In addition, the credibility calculation method proposed in this paper attaches a credibility value to each task participant, which can help the system select high-quality task participants and task data, thus effectively reducing the possibility of Byzantine attack, label flipping attack, and data poisoning attack. From the experimental results, it can be seen that the data loss of the DLD-LDP algorithm is always lower than other algorithms regardless of the increase or decrease of the privacy budget, and the amount of the privacy budget has little effect on the effect of this algorithm.

For the location privacy issue. Zhang et al. (2021) proposed a scalable system built on top of a public cloud service that hides a mobile user's network location and traffic from communication peers. The system creates a dynamic distributed agent network for each mobile user, which protects location privacy while minimizing performance overhead and operational costs. Cui et al. (2021) proposes a novel architecture called LBS@E for building off-domainized location services in 5G mobile edge computing environments that do not require user location. Mobile users can retrieve local information from LBSs deployed on nearby edge servers according to LBS@E and minimize the risk of localization. The proposed ELP-P and ELP-U algorithms aim to maximize the privacy region and utility region, respectively, and take only slightly more time than the random and greedy algorithms.

For identity privacy issues. Irshad et al. (2022) proposes a novel two-factor biometric authentication protocol for MEC that implements a Physical Unclonable Function (PUF). The protocol proposed in this paper has the most complete security features when compared with eight algorithms under the Real or Random (ROR) form of security analysis model. Jia et al. (2022) redesigns the authentication protocol using passwords and biometrics and

Table 6 Comparison of Papers focusing on privacy protection

Papers	Collaboration Manner	Issue	Method	Paradigm
(Wang 2021)	Thing-Edge-Cloud	data privacy	Instant messaging private data protection model	Mobile Edge Computing
(Qu 2020)	Thing-Edge-Cloud	data privacy	Homomorphic encryption	Intelligent Edge Computing
(Xiao et al. 2020)	Thing-Edge-Cloud	data privacy	Layered edge computing architecture	Multi-Access Edge Computing
(Wang 2022)	Thing-Edge-Cloud	data privacy	DLD-LDP	Mobile Edge Computing
(Nguyen 2021)	Thing-Edge-Cloud	data privacy	Blockchain technology	Mobile Edge Computing
(Zhang et al. 2021)	Thing-Edge-Cloud	location privacy	Scalable systems built on top of public cloud services	Multi-Access Edge Computing
(Cui et al. 2021)	Thing-Edge-Cloud	location privacy	A new architecture called LBS@E	Mobile Edge Computing
(Irshad et al. 2022)	Thing-Edge-Cloud	identity privacy	New two-factor biometric authentication protocol	Mobile Edge Computing
(Jia et al. 2022)	Thing-Edge-Cloud	identity privacy	Authentication protocols using passwords and biometrics	Mobile Edge Computing
(Li et al. 2021)	Thing-Edge-Cloud	identity privacy	Lightweight device solution based on anonymous identity	Mobile Edge Computing
(Xu 2019)	Thing-Edge	computing offloading privacy	Calculations with IoV privacy protection features	Mobile Edge Computing
(Pang et al. 2022)	Thing-Edge-Cloud	computing offloading privacy	Online Privacy ProtectionComputing Offloading Mechanism OffloadingGuard	Mobile Edge Computing

introduces a new authenticator as MU, with the option of achieving pseudo-anonymization effects as well. Outperforms four other competing schemes in terms of computational and communication costs.

For computing offloading privacy issue. Xu (2019) proposes a computational approach with IoV privacy protection for the privacy conflicts challenge of offloading computations

to edge computing devices (ECDs). The computational method first uses an algorithm based on the V2V transmission to obtain the task offloading path and then uses NSGA-II to find the globally optimal offloading policy. The method achieves avoiding privacy conflicts while reducing task execution time and energy consumption. Pang et al. (2022) proposed OffloadingGuard, an online privacy-preserving computational offloading mechanism. Specifically, it employs a restricted-range Laplace distribution to perturb the original offloading rate of each user for differential privacy and restricts the perturbed offloading rate to a reasonable range to strictly protect the user's privacy from being stolen by untrustworthy edge computing servers.

Summary The main methodologies to reduce the cost of privacy protection include designing new protocols and schemes, introducing noisy technologies, and blockchain technologies. Traditional privacy-preserving methods require a large amount of resources, which is usually unacceptable in resource-constrained edge computing networks. Therefore, researchers are exploring new approaches aimed at reducing the resources required to achieve privacy protection. From the survey, the main idea is to reduce the cost of privacy preservation indirectly by reducing the computational and communication costs required to realize the privacy-preserving features. The main idea behind designing new protocols and schemes is to reduce or eliminate the cost of encryption, decryption, and authentication. The main idea of introducing noise and blockchain technology is to achieve a balance between privacy protection and computation cost by lowering the computation cost of traditional encryption and decryption schemes. It is undeniable that realizing the privacy protection function will increase resource consumption with a high probability, but we can make that increased cost become tiny through our efforts. Realizing the privacy protection function is also an inevitable requirement for developing edge computing networks.

4 Mathematical models

In this section, we provide an overview of scholars' progress in building mathematical models to reduce the cost of edge computing networks in recent years. Researchers have commonly employed Markov decision processes, game theory, and integer nonlinear programming to model the problem formally and have thoroughly explained the theoretical foundations of these mathematical models. These models not only provide profound insights but also provide solid theoretical support for developing effective solution strategies, enabling researchers to identify and solve edge computing network cost problems in practice with greater precision. By applying these mathematical models, researchers could theoretically explore and optimize the cost-effectiveness of edge computing networks, laying a solid foundation for the development of edge computing. The outline of problem modeling is shown in Fig. 5 and is described in detail below.

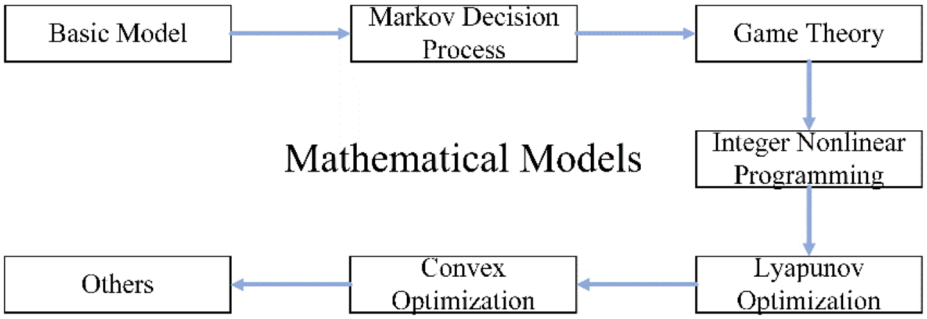


Fig. 5 Outline of mathematical models

4.1 Basic model

In a typical edge computing scenario, user devices, edge devices, and cloud devices all have computing capabilities and user devices and edge nodes usually use wireless connections. The user devices generate various tasks.

The set of user devices is $N = \{N_1, N_2, N_3, \ldots, N_n\}$. The set of edge servers is $M = \{M_1, M_2, M_3, \ldots, M_m\}$. The set of tasks is $T = \{T_{11}, T_{12}, \ldots, T_{1m}, \ldots, T_{nm}\}$. Task j generated by vehicle i is presented by a three-tuple $T_{ij} = (s_{ij}, c_{ij}, t_{ij}^{max})$, where s_{ij} denotes the size of the input data of the task, c_{ij} represents the amount of resources needed to complete the task, t_{ij}^{max} is the maximum tolerated delay of this task. The CPU power of user device is p_i. The CPU power of edge server is p_j.The distance between user device and edge server is d_{ij}. The path loss index is β. The wireless channel gain is h_{ij}. The Gaussian noise power inside the channel is ω_0. The transmission power of the user device is $P_{tra,i}$. The transmission power of the edge server is $P_{tra,j}$.The assigned wireless bandwidth is B_{ij}. The size of the output data of the task is o_{ij}.

When the offload policy determines that the task is executed locally. The computational power of the user device (the number of CPU cycles per second) is f_i. The time delay in the processing of the local computing can be expressed as $t_i^{loc} = c_{ij}/f_i$. The energy consumption can be expressed as $E_i^{loc} = p_i t_i^{loc} = c_{ij} p_i / f_i$.

When the offloading policy decides that the tasks are all offloaded to the edge nodes for execution. The data transmission rate from the user device to the edge server is r_{ij}. According to Shannon's theorem, r_{ij} can be expressed as:

$$r_{ij} = B_{ij} \log_2 \left(1 + \frac{P_{tra,i} {h_{ij}}^2}{\omega_0 + {d_{ij}}^{-\beta}}\right). \tag{1}$$

The data transmission rate from the edge server to the user device is:

$$r_{ji} = B_{ij} \log_2 \left(1 + \frac{P_{tra,j} {h_{ij}}^2}{\omega_0 + {d_{ij}}^{-\beta}}\right). \tag{2}$$

The computational resource provided by the edge server is f_j. Thus, the upload delay is $t_{ij}^{up} = s_{ij}/r_{ij}$. The compute delay is $t_{ij}^{com} = c_{ij}/f_j$. The download delay is $t_{ij}^{down} = o_{ij}/r_{ji}$.

The energy consumption of the user device for transmitting the offloaded task is expressed as $E_{ij}^{up} = P_{trans} t_{ij}^{up} = s_{ij} P_{tra,i} / r_{ij}$. Similarly, $E_{ij}^{down} = o_{ij} P_{tra,j} / r_{ji}$, $E_{ij}^{com} = c_{ij} p_j / f_j$.

When the offloading policy decides to offload some of the tasks to the edge nodes for execution. The task offload ratio is $\rho_j \in (0,1)$. Similar to the binary uninstallation, the upload delay is $t_{ij}^{p-up} = \rho_j s_{ij} / r_{ij}$, the download delay is $t_{ij}^{p-down} = \rho_j s_{ij} / r_{ji}$, The compute delay is $t_{ij}^{p-com} = \rho_j c_{ij} / f_j$. The energy consumption of upload, download, and compute are expressed as $E_{ij}^{p-up} = \rho_j s_{ij} P_{tra,i} / r_{ij}$, $E_{ij}^{p-down} = \rho_j o_{ij} P_{tra,j} / r_{ji}$, and $E_{ij}^{p-com} = \rho_j c_{ij} p_j / f_j$.

Based on the analysis above, the total delay of processing task is formulated as $t = min\left\{t_i^{loc}, t_{ij}^{up} + t_{ij}^{com} + t_{ij}^{down}\right\}$. The energy consumption cost can be formulated as $C_{ij}^{E} = \varrho\left(E^{l} + E^{tr} + E^{e} + E^{down}\right)$, where ϱ denotes the weight coefficient that indicates the energy consumption cost of one unit energy during task computing and transmitting. Let Pr_1 denote the cost of using per unit of bandwidth per unit of time, the bandwidth cost can be formulated as $C_{ij}^{t} = Pr_1 B_{ij}(t_{ij}^{up} + t_{ij}^{down})$.

4.2 Markov decision process

MDP is a mathematical framework for modeling sequential decision problems based on an extension of the Markov chain that combines the concepts of decision-making and uncertainty. MDP is widely used in reinforcement learning for solving decision problems with delayed rewards, where the goal is to find an optimal policy that maximizes the long-term accumulated reward. MDP typically uses a value function to measure the long-term value of each state and uses algorithms such as policy evaluation and policy improvement to optimize the policy progressively. A comparison of papers modeling as Markov Decision Process is presented in Table 7.

MDP includes state space, action space, decision rules, rewards, and transfer probabilities (Qu et al. 2020). The state space is the set of all states, the state is a description of the environment in which the system is located during the decision-making process. The state will change after the agent takes action, and the evolution has Markov properties.

The state space is generally denoted using S, where $s \in S$ represents a specific state. The action space, also called the decision space, is the set of all possible actions, and the intelligent body will choose an action at each decision point based on the current system state, which is the result of the intelligent body's decision. The action space is generally denoted by A, where $a \in A$ represents a specific decision. A decision rule is a set of action choices that determines which action the intelligence chooses to give as an outcome in different states.

The probability of taking action a in state s is generally denoted by $\pi(a|s)$. The reward is an immediate reward received by an intelligent body after giving a decision action, and the reward represents how good or bad the action is. The immediate reward of a system state transfer to s' after taking action a in state s is generally denoted by $R(s,a,s')$. The transfer probability represents the likelihood that the system will transfer from one state to another, with the state transfer occurring after the intelligence gives a decision action. Generally, $P(s' \mid s,a)$ is used to denote the probability of the system state transferring to s' after taking action a in state s.

In addition, in order to facilitate learning, researchers generally also introduce a state-value function $V(s)$, a discount factor $\gamma\,(0 \leq \gamma \leq 1)$, and an action-value function

Table 7 Comparison of papers modeling as Markov decision process

Paper	Problem	Model	Optimization Goals	Method
(Zhao 2022)	Task Offloading	MDP	1) Execution Time Cost; 2) Energy Cost; 3) Packet Loss;	1) Deep Q-Network (DQN);
(Zhao et al. 2022)	Task Offloading	MDP	1) Total System Cost;	1) Collaborative Multi-Agent Deep Reinforcement Learning;
(Gao et al. 2023)	Task Offloading	MDP	1) Privacy Protection; 2) Energy Cost; 3) Execution Time Cost;	1) PPO2;
(Xu 2022)	Resource Allocation	PAMDP	1) Standby Cost; 2) Computing Cost;	1) Pixel-Based Deep Q-Network(P-DQN)
(He et al. 2022)	Minimize AoI	CMDP	1) Energy Cost; 2) AoI;	1) Lagrange Transformation Framework; 2) Q-Learning; 3) Heuristic Algorithm;
(Li 2022)	Reduce Service Migration Cost	MDP	1) Service Migration Cost; 2) Cache Policy; 3) System Power Consumption; 4) Resource Utilization;	1) Deep Reinforcement Learning (DRL); 2) Deep Neural Network (DNN); 3) DQN;

$Q(s,a)$, where $V(s)$ denotes the expected cumulative reward for taking action according to the decision rule π in state s, $Q(s,a)$ denotes the expected cumulative reward for taking action a in state s according to the decision rule π, and $\gamma\,(0 \leq \gamma \leq 1)$ denotes the future discount factor for balancing immediate and future rewards. Based on the above notation, the state value function can be expressed as

$$V^{\pi}(s) = E\left[R(s,\ a, s')\right] + \gamma \times \sum P(s'|s,\ a) \times V(s'). \tag{3}$$

The action value function can be expressed as

$$Q(s,\ a) = E\left[R(s,\ a, s')\right] + \gamma \times \sum P(s'|s,\ a) \times \sum \pi\ (a'|s') \times Q(s',\ a'). \tag{4}$$

The strategy evaluation equation can be expressed as

$$V_{k+1}(s) = \sum \pi\ (a|s) \times \sum P(s'|s,\ a) \times [R(s,\ a,\ s') + \gamma\ \times\ V_k(s')]. \tag{5}$$

The strategy improvement equation can be expressed as

$$\pi'(a|s) = argmax \sum P(s'|s,\ a) \times [R(s,\ a,\ s') + \gamma\ \times\ V(s')]. \tag{6}$$

The Bellman equation embodies the maximum expected payoff that can be obtained by following the optimal policy from the state s. From (3) and (5), the Bellman equation can be expressed as

$$V^*(s) = max_a \sum_{s',R} P(s'|s,\ a) \times [R(s,\ a,\ s') + \gamma\ \times\ V_k(s')]. \tag{7}$$

In order to construct a CMDP suitable for minimizing the AoI (He et al. 2022), introduces the amount of remaining data d_i^r, the value of the current AoI a_i, the number of executing tasks q_i, the channel state h_i, and $g_i(t) \in (0,1)$ to indicate whether or not a new task is generated at the moment of t. Under the constraints of the energy and the communication resources, the state vector of the moment t is represented as

$$s(t) = (d_i^r(t), a_i(t), q_i(t), h_i(t))_{i \in N} \in S. \tag{8}$$

For the state transfer probability, the paper gives the transfer probability function for each of the three different cases. 1) when the execution queue and the waiting task queue are empty, it can be expressed as

$$P(s'|s,a) = \begin{cases} P(g_i(t) = 0), d_i^r(t+1) = 0 \\ P(g_i(t) = 1), d_i^r(t+1) > 0 \end{cases} \tag{9}$$

2) When there are execution tasks, and the waiting task queue is not empty, it can be expressed as

$$P(s'|s,a) = P(d_i^r(t+1) - d_i^r(t) = a_i(t) + 1 - a_i(t+1)). \tag{10}$$

3) When there are execution tasks and the waiting queue is empty, it can be expressed as

$$P(s'|s,a) = \prod_{i \in N} P\left(d_i^{r\prime}, a', q_i'|s,a\right) P\left(h_i'|h_i\right). \tag{11}$$

In the PAMDP used in Xu (2022), the action space A_θ is the preferred set of parameterized actions, and the parameterized actions are denoted as $a(\theta)$, at which point the state transfer probabilities, immediate rewards, and action probabilities are denoted as $P(s'|s, a(\theta))$, $R(s, a(\theta))$, and $\pi\ (a(\theta)|s)$, respectively.

4.3 Game theory

Game theory is a mathematical model that studies the choices made by decision-makers in an interacting environment. It involves decision-making and strategy choices between multiple players and examines their outcomes and implications. The solution concepts of

GT include Nash equilibrium, dominant strategy of the game, optimal strategy of the game, and so on. Nash equilibrium refers to the strategy combination selected by each player in the game. It ensures no player can obtain higher returns by unilaterally changing their strategy. A dominant strategy of a game is one strategy that is better than the others in all situations. An optimal game strategy is one in which a player chooses the strategy that maximizes his or her payoff given information. A comparison of papers modeling as Game Theory is presented in Table 8.

Table 8 Comparison of papers modeling as game theory

Paper	Problem	Model	Optimization Goals	Method
(Wang et al. 2023)	Task Scheduling	GT	1) Resource Scheduling Cost; 2) Total Energy Consumption; 3) Communication Energy Consumption;	1) Independent Recurrent Neural Network (IndRNN); 2) Long Short Term Memory (LSTM);
(Tang 2022)	Task Scheduling	POSG	1) Average Utility; 2) Average Delay; 3) Average Energy Consumption;	1) Duel Dual Deep Recurrent Q-Network (D3RQN);
(Tütüncüo 2024)	Task Offloading	Single-Leader Multi-Follower Stackelberg Game	1) Storage Cost; 2) Computing Cost;	1) Bayesian Gaussian Process Bandit Algorithm; 2) Greedy Heuristics for Gaussian Process Approximation;
(Mitsis et al. 2022)	Task Offloading	Multi-Leader Multi-Follower Stackelberg Game	1) Operating Cost; 2) Utility; 3) Revenue;	1) Completely Autonomous Reinforcement Learning; 2) Semi-Autonomous Game Theory;
(Zhang and Zeng 2023)	Task Scheduling; Task Offloading	Stackelberg game	1) Task Offloading Cost;	1) Search Algorithm Based on Genetic Algorithm;
(Zhou et al. 2022)	Task Scheduling	Stackelberg game	1) Task Offloading Cost; 2) Energy Cost;	1) Gradient-Based Iterative Search Algorithm (GISA);
(Zhou et al. 2023)	Task Offloading	Stackelberg game	1) Communication Cost; 2) Execution Time Cost; 3) Energy Consumption; 4) Computing Cost;	1) Gradient-Based Dynamic Iterative Search Algorithm (GDISA);

In GT, a game consists of players, strategies, and payoff functions. A player is the decision maker of the game, which can be a person, a group, a concept, etc. A strategy is a collection of actions that a player in each state can take, and the payoff function determines the payoffs to each player after different players have taken decisions. In GT, strategies are categorized into two main types: pure strategies and mixed strategies. Pure strategies define the actions that a player must take in any state, while mixed strategies allow a player to choose a pure strategy randomly, and it is clear that a pure strategy can be a particular case of a mixed strategy (Sohrabi 2020).

In GT, there are two fundamental concepts: Nash equilibrium and Pareto Optimality. Nash equilibrium is a strategy combination in which each player cannot obtain higher returns by changing his strategy given the strategy choices of other players. It can be used to predict the results of the game, analyze strategic interactions, and guide the game strategy design, etc. The Pareto optimal solution describes a combination of strategies in which no player can change his strategy without reducing the gains of other players. In short, Nash equilibrium focuses on the best response strategy of individual players, and Pareto optimal solution focuses on maximizing overall benefits. It should be noted that Nash equilibrium is not necessarily the globally optimal result nor the only result. In some cases, there may be multiple Nash equilibria. GT and optimization are conceptually similar. In many studies, researchers regard GT as a kind of stochastic optimization and use GT and optimization together. The Nash equilibrium in GT is expressed as

$$u_i\left(s_i, s_{-i}\right) \geq u_i\left(s_i{}', s_{-i}\right), \forall\, s_i \neq s_i{}', \forall\, i \in \mathrm{N}, \tag{12}$$

i is the player, s_i is the strategy combination of player i, s_{-i} is the strategy combination of other players except player i, u_i is the payment function of player i, and $\mathrm{N}$is the set of players. Tang (2022) also represents POSG in the same way. The Pareto optimal solution is generally expressed as

$$u_i\left(s\right) \geq u_i\left(s'\right), \forall\, s \neq s', \forall\, i \in \mathrm{N}. \tag{13}$$

Specific to different application scenarios, researchers will modify the formula according to actual needs.

Wang et al. (2023) expressed the mobile edge computing task scheduling game as

$$\Gamma = \left(\mathcal{N}, \mathcal{P}, \mathcal{S}, \mathcal{T}, \left(f_{(s,t)}\right)_{(s,t)\in \mathcal{B}\times\mathcal{T}}, \left(l_{(s,n)}\right)_{(s,n)\in \mathcal{S}\times\mathcal{N}}, \mathcal{K}\right), \tag{14}$$

$\mathcal{N}$ is the set of users, $\mathcal{P}$ is the set of user priorities, $\mathcal{S}$ is the set of servers, $\mathcal{T}$ is the set of time slots, $f_{(s,t)}$ represents the non-increasing computing resource function of server s at time t, $l_{(s,n)}$ represents the link quality function between user n and server s, and $\mathcal{K}$ is the consumption of switching servers. This article expresses the user's utility function as

$$U_n\left(X\right) = \sum_{\nu\in V(X_n)} f_{\nu}(\psi\, X\left(\nu\right)) - \sum_{e\in E(X_n)} cost\left(e\right), \tag{15}$$

$cost\,(e)$ is the user's path cost, X_n is the user's path choice, $\psi\, X\,(\nu) = |n' \in \mathcal{N} : \nu \in V\,(X)_{n'}|$ is the user passes through the path of vertex ν. At this time, the Nash equilibrium is expressed as

$$U_n\left(X^*\right) \geq U_n\left(X_n{}', X_{-n}\right), \forall\, X \neq X_n{}', \forall\, n \in \mathcal{N}. \tag{16}$$

In the multi-leader multi-follower Stackelberg game of edge server pricing and user task offloading selection. Mitsis et al. (2022), uses$b_{n,s}^{MEC}$ to represent the amount of data for task offloading, $\boldsymbol{b}_s^{MEC}$ to represent the user's benefit, $P_{n,s}(b_{n,s}^{MEC}, \boldsymbol{b}_{-n,s}^{MEC})$ represents the user's expected benefit function. In this paper, the Nash equilibrium is expressed as

$$E\left(P_{n,s}\left(b_{n,s}^{MEC*}, \boldsymbol{b}_{-n,s}^{MEC*}\right)\right) \geq E\left(P_{n,s}\left(b_{n,s}^{MEC}, \boldsymbol{b}_{-n,s}^{MEC*}\right)\right), \tag{17}$$

$*$means that the parameter is a vector. Similarly Zhang and Zeng (2023), also used the multi-leader multi-follower Stackelberg game, which considered the optimal strategy of task scheduling and the pricing strategy of edge servers at the same time, and constructed the following Nash equilibrium formula:

$$\begin{array}{l} U_i\left(S_{1,j}^*, S_{2,j}^*, \omega\,_{1,i}^*, \omega\,_{2,i}^*\right) \geq U_i\left(S_{1,j}^*, S_{2,j}^*, \omega\,_{1,i}, \omega\,_{2,i}\right) \\ U_{j,c}\left(S_{1,j}^*, S_{2,j}^*, \omega\,_{1,i}^*, \omega\,_{2,i}^*\right) \geq U_{j,c}\left(S_{1,j}, S_{2,j}, \omega\,_{1,i}^*, \omega\,_{2,i}^*\right) \end{array}, \tag{18}$$

ω^* represents the task scheduling strategy, and S^*represents the pricing strategy (Zhou et al. 2022; Zhou et al. 2023; Tütüncüo 2024). used the Stackelberg game with a single leader and multiple followers, and proved through mathematical reasoning that there is one and only one Nash equilibrium.

4.4 Integer nonlinear programming

Integer Nonlinear Programming is a mathematical optimization problem aiming to find an integer solution that minimizes or maximizes the objective function under given constraints. In the edge computing paradigm, we generally use two special cases of INLP: Mixed Integer Nonlinear Programming (MINLP) and Nonlinear 0–1 Integer Programming to construct optimization problems. A comparison of papers modeling as Integer Nonlinear Programming is presented in Table 9.

MINLP combines the characteristics of integer programming and nonlinear programming, but some decision variables need to take integer values, and the calculation complexity is higher. The problem form can be expressed as

$$\min\left\{f^0\,(x, y) : f^j\,(x, y) \leq 0\,(j = 1, \ldots, m), x \in \mathbb{z}_+^{n1}, y \in \mathbb{R}_+^{n_2}\right\}, \tag{19}$$

$n1$ is the number of integer constraint variables, n_2 is the number of continuous variables, m is the number of constraints, and $f^j\,(x, y)$ is the realistic mapping of $\mathbb{z}_+^{n1} \times \mathbb{R}_+^{n_2}$(Burer 2012). Nonlinear 0–1 integer programming is a special case of MINLP. It combines the characteristics of nonlinear programming and 0–1 integer programming, restricts the deci-

Table 9 Comparison of papers modeling as Integer Nonlinear Programming

Paper	Problem	Model	Optimization Goals	Method
(Chu et al. 2023)	Maximize QoE	MINLP	1) Service Migration Cost; 2) Energy Consumption;	1) Heuristic Algorithm; 2) Lagrange Dual Decomposition Theory;
(Zhou et al. 2023)	Minimize Task Cost	MINLP	1) Total Task Cost;	1) Collaborative Computing Offloading and Resource Allocation Method (CCORAM)
(Huang et al. 2022)	Minimize Total Cost	MINLP	1) Service Deployment Cost; 2) Storage Cost; 3) Resource Utilization;	1) Gibbs Sampling;
(Deng et al. 2021)	Minimize Weighted Sum	MINLP	1) Delay; 2) Energy Consumption; 3) Price;	1) Improved Branch and Boundary Algorithm; 2) Particle Swarm Optimization Algorithm (PSO);
(Feng et al. 2021)	Minimize Network Cost	MINLP	1) Total Cost;	1) Dynamic Programming Algorithm (DP); 2) Distributed Collaborative Data Caching and Computation Offloading Iterative Algorithm (CDCCO)
(Zhong et al. 2022)	Task Scheduling	Nonlinear 0–1 Integer Programming	1) Delay; 2) Energy Cost;	1) Parallel Multi-Block Alternating Direction Multiplier Method; 2) Logarithmic Smoothing and Cyclic Block Coordinate Gradient Projection Method (CBGP); 3) Conjugate Gradient and Newton Linear Search Technique;

sion variables to only binary 0 or 1, and is an NP-Hard problem with a further increase in problem complexity. The form of the problem can be expressed as

$$\max\left\{f_0(x)\,|f_k(x) \leq\ b_k, k \in\ K, x\ binary\right\}, \tag{20}$$

where the multilinear function $f_k(x)$ can be expressed as

$$f(x) = \sum_{j\in N} a_j \left(\prod_{i\in O_j} x_i\right), x_i = 0\ or\ 1, i \in \bigcup_{j\in N} Q_j, \tag{21}$$

Q_j denotes the subscript of the variable corresponding to the non-zero coefficient in the function (Balas and Mazzola 1984).

Chu et al. (2023) jointly optimize offline joint service selection, Virtual Machine (VM) resource allocation and task offloading decisions with the objective of maximizing QoE, modeling the optimization problem as a MINLP with the constraints represented as

$$\text{Max}: \sum_{i=1}^{N}\sum_{j=1}^{N_i} \lambda_{ij} \times\ G_{ij}(F_i) \times\ x_{ij} \tag{22a}$$

$$\text{s.t.}: \sum_{i=1}^{N} F_i \leq F, \tag{22b}$$

$$\sum_{i=1}^{N} y_i \leq M, \tag{22c}$$

$$x_{ij} \leq y_i, \forall\, i, j, \tag{22d}$$

$$0 \leq F_i \leq F^{\text{MAX}}, \forall\, i, \tag{22e}$$

$$y_i \in \{0,1\}, x_{ij} \in \{0,1\}, \forall\, i, j. \tag{22f}$$

Where y_i, F_i and x_{ij} are variables for service selection, resource allocation and offloading decisions, respectively. (22b) and (22c) denote the resource constraints of the edge servers, and (22d) states that task offloading can be performed only if the service is hosted. In contrast Zhou et al. (2023), aims at minimizing the task cost, and accordingly, the constraints are denoted as

$$\min_{\xi, \mathcal{F}} \phi\ (\xi, \mathcal{F}) = \sum_{i \in \mathcal{I}} \phi^{\,i} \tag{23a}$$

$$s.t.: \sum_{i \in \mathcal{I}} f_m^i \leq F_m, \forall\, m \in \mathcal{M}, \tag{23b}$$

$$0 \leq f_m^i \leq F_m, \forall\, i \in \mathcal{I}, m \in \mathcal{M}, \tag{23c}$$

$$f_{loc}^i \geq 0, \forall\, i \in \mathcal{I}, \tag{23d}$$

$$T_j^i \leq T_{j,max}^i, \forall\, i \in \mathcal{I}, \tag{23e}$$

$$\sum_{j \in \mathcal{J}} U_j Y_j^m \leq D_m, \forall\, m \in \mathcal{M}, \tag{23f}$$

$$X_{j,m}^i \in \{0,1\}, \forall\, i \in \mathcal{I}, m \in \mathcal{M}, \tag{23g}$$

$$Y_j^m \in \{0,1\}, \forall\, m \in \mathcal{M}, j \in \mathcal{J}, \tag{23h}$$

$$\zeta_{\,m}^{\,i} \in \{0,1,2,3\}, \forall\, i \in \mathcal{I}, m \in \mathcal{M}. \tag{23i}$$

Where ξ is the set of task computation policies, $\mathcal{F}$ is the set of resource allocation policies, (23b), (23c), and (23d) represent that each edge node has to ensure that the resources are not overrun without reducing the resource allocations to the users, and (23e) and (23f) represent that the processing latency and the maximum cache capacity have to be guaranteed. In addi-

tion Deng et al. (2021); Huang et al. (2022); Zhou et al. (2023), and Feng et al. (2021) also model part of the problem as a MINLP with different constraints for different scenarios, and their specific representations are similar to the two examples above, although the notation used may be different.

Zhong et al. (2022) modeled task offloading as a nonlinear 0–1 integer programming such that the task offloading decision is denoted as $y_{1j} \in \{0,1\}$, where a value of 1 indicates that task offloading is performed, and a value of 0 indicates that no offloading is performed, in which case the total task offloading delay is denoted as

$$T^{(b_1,h_j)} = T_{up}^{(b_1,h_j)} + T_{execute}^{(b_1,h_j)} = y_{1j} \frac{c_j}{B_{MBS}\log_2\left(1+\frac{p^{(b_1,h_j)}G^{(b_1,h_j)}}{\sigma^2}\right)} + y_{1j}\frac{c_j u_j}{f^{(b_1)}}, \tag{24}$$

$T_{up}^{(b_1,h_j)}$ is the upload time, $T_{execute}^{(b_1,h_j)}$ is the task execution time, $p^{(b_1,h_j)}$ and $G^{(b_1,h_j)}$ are the transmit power of the base station and the interaction power between the base station and the device, respectively, σ^2 is the white noise power of the channel, and $f^{(b_1)}$ is the maximum number of tasks executable. And the parallel optimal task allocation mechanism (POTAM) framework proposed in this paper jointly optimizes the offloading decision and computational resource allocation, such that $|H|$ represents the number of tasks and $|BS|$ represents the number of nodes. At this point, the constraints are represented as

$$\begin{aligned} \min \alpha & \left(\sum_{j=1}^{|H|} z_j T_{local}^{(h_j)} + \sum_{i=2}^{|BS|}\sum_{j=1}^{|H|} x_{ij} T^{(b_i,h_j)} + \sum_{j=1}^{|H|} y_{1j} T^{(b_1,h_j)} \right) \\ + (1-\alpha) & \left(\sum_{j=1}^{|H|} z_j E^{(h_j)} + \sum_{i=2}^{|BS|}\sum_{j=1}^{|H|} x_{ij} E^{(b_i,h_j)} + \sum_{j=1}^{|H|} y_{1j} E^{(b_1,h_j)} \right), \end{aligned} \tag{25a}$$

$$s.t.: \ z_j T_{local}^{(h_j)} + \sum_{i=2}^{|BS|} x_{ij} T^{(b_i,h_j)} + y_{1j} T^{(b_1,h_j)} \le t_j^{max}, \tag{25b}$$

$$x_{ij} \in \{0,1\}, y_{1j} \in \{0,1\}, z_j \in \{0,1\}, \tag{25c}$$

$$\sum_{j=1}^{|BS|} h^{(b_i,h_j)} x_{ij} \le 1, 0 < h^{(b_i,h_j)} \le 1, \tag{25d}$$

$$\sum_{j=1}^{|BS|} h^{(b_i,h_j)} x_{ij} \le 1, 0 < h^{(b_i,h_j)} \le 1, \tag{25e}$$

$$\sum_{h_j \in H} \pi\left(h_i, l\right) \leq 1, l \text{ is a single core device,} \tag{25f}$$

$$\sum_{h_j \in H} \pi\left(h_i, l\right) \leq D_l, l \text{ is a multi-core device,} \tag{25g}$$

$$i \in \{2,3 \ldots,|BS|\}, j \in \{1,2,3 \ldots,|H|\}, \pi\left(h_j, l\right) \in \{0,1\}, \tag{25h}$$

$$\begin{aligned} &0 \leqslant \alpha \leqslant 1, \\ &0 \leqslant \theta_l \leqslant 1, \\ &c_{1j} + c_{ij} + c_{0j} = c_j, \\ &0 \leqslant c_{0j} \leqslant c_j, \\ &0 \leqslant c_{ij} \leqslant c_j, \\ &0 \leqslant c_{1j} \leqslant c_j. \end{aligned} \tag{25i}$$

(25b) implies that up-to-time requirements are to be met, (25d) implies that resources cannot be overrun, and (25e) implies that there is no segmentation of tasks, and that tasks are executed only at the edge server or locally.

4.5 Lyapunov optimization

Lyapunov optimization refers to the use of a Lyapunov function to optimally control a dynamical system. It is widely used in areas such as network control, queuing systems and resource allocation. Lyapunov optimization is based on the Lyapunov function, which is used to design a control strategy to ensure the stability and performance of the system by analyzing the change of the system state over time. When using Lyapunov optimization in edge computing, the key is to define the drift and the penalty. It is worth noting that solving the Lyapunov optimization has much lower computation complexity than convex optimization because the drift-plus-penalty expression is usually a deterministic problem. Moreover, in contrast to MDP, it does not require knowledge of the probability distribution of the random event process (Lin 2020; Sadatdiynov et al. 2023).

The Lyapunov function $V(x)$ is an energy function that measures the stability of the state x of a system. For any x, $V(x) \geq 0$ always holds and the system equilibrium point is reached when $V(x) = 0$.For the system state $Q(t)$, the Lyapunov function is expressed as $V(Q(t))$. The Lyapunov drift is the amount of change in the Lyapunov function over a time step and is expressed as $\Delta V(Q(t)) = V(Q(t+1)) - V(Q(t))$. The penalty function is generally denoted by $\boldsymbol{P}(t)$. In this case, the drift penalty function is expressed as $\Delta V(Q(t)) + V \cdot P(t)$, where V is a non-negative parameter.

Consider a discrete-time system where the change in the system state $Q(t)$ at time t can be expressed as:

$$Q(t+1) = [Q(t) + A(t) - B(t)]^{+}. \tag{26}$$

Where $A(t)$ is the arrival process, $B(t)$ is the service process, $[x]^+ = \max(x, 0)$. If the Lyapunov function is $V(Q(t)) = \frac{1}{2}Q(t)^2$, then the Lyapunov drift can be expressed as

$$\Delta V(Q(t)) = V(Q(t+1)) - V(Q(t)) = \frac{1}{2}(Q(t+1))^2 - \frac{1}{2}Q(t)^2. \tag{27}$$

After substituting (26), (27) can be expressed as

$$\Delta V(Q(t)) = \frac{1}{2}([Q(t) + A(t) - B(t)]^+)^2 - \frac{1}{2}Q(t)^2. \tag{28}$$

With the introduction of the penalty function $P(t)$, the optimization objective can be expressed as minimizing this function:

$$min\ \Delta V(Q(t)) + V \cdot P(t). \tag{29}$$

By designing a suitable control strategy $B(t)$, we can minimize the above objectives and thus optimize the stability and performance of the system.

Jia et al. (2022)investigates a multi-vehicle networking system assisted by a computationally resource limited MEC, in which vehicles with complex applications can offload their subtasks to the MEC server. To reduce the cooperation overhead, the paper proposes a multi-intelligence proximal policy optimization algorithm (Ly-MAPPO), which requires only local observation to solve the subproblems implemented by Lyapunov optimization techniques in real-time. In addition, to improve the performance of the Ly-MAPPO algorithm, the paper introduces Graph Convolutional Neural Networks (GCNs) to extract the interdependencies between subtasks (Lyu 2017). uses perturbed Lyapunov functions aiming to stochastically maximize network utility, balancing throughput and fairness. The backpack problem is equivalently formulated as a Lyapunov optimization problem.

Chen et al. (2019)proposed a dynamic computational offloading algorithm (DCOA). The algorithm decomposes the computational offloading optimization problem into a series of subproblems. The queuing model is constructed as a Lyapunov model to reduce the queue backlog and keep the MEC in a low congestion state. Similarly, modeling the queuing problem in the optimization problem as a Lyapunov model is done in Zhang et al. (2018) and Pu et al. (2016). Zhang et al. (2018) proposed an online dynamic task allocation scheduling approach to study the trade-off between energy consumption and execution latency for EH-capable MEC systems. It is formulated as an average weighted sum problem for minimizing energy consumption and execution delay of mobile devices with buffer queue stability and battery power as constraints. Construct the queuing model as a Lyapunov model. Pu et al. (2016). constructed the queuing model as a Lyapunov model that utilizes the stability of the virtual queue to ensure that the time-averaging constraints are satisfied. The design principle of the online task offloading algorithm is to minimize the upper bound of the drift plus penalty.

4.6 Convex optimization

Convex optimization is a powerful tool for optimization problems because it is solvable. The objective function of convex optimization is a convex function, and the constraints consist of a convex function and a linear function. If the formulated optimization model is

convex, classical methods such as the Lagrange duality method and Karush-Kuhn-Tucker (KKT) conditions can be applied to solve the model and achieve the global optimization objective. KKT conditions are necessary and sufficient optimality conditions for convex optimization problems. The convex optimization model consists of one or more objectives and some constraints. If the offloading model is a non-convex optimization problem, a usual way is to transform it to a convex optimization (Lin 2020; Sadatdiynov et al. 2023).

If a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex, then it is satisfied if and only if for all $x, y \in \mathbb{R}^n$ and $\theta \in [0,1]$:

$$f(\theta x + (1-\theta) y) \leq \theta f(x) + (1-\theta) f(y). \tag{30}$$

A standard convex optimization problem can be expressed as

$$\min f_0(x) \tag{31a}$$

$$s.t. \ f_i(x) \leq 0, i = 1, \ldots, m, \tag{31b}$$

$$h_j(x) = 0, j = 1, \ldots, p. \tag{31c}$$

Where (31a) is the objective function, a convex function. (31b) is the inequality constraint function and is a convex function. (31c) is the equational constraint function and is linear.

Cheng et al. (2024)investigated the application of online convex optimization for resource allocation in edge computing networks with time-varying cost and time-varying constraint functions. The paper uses an improved online saddle-point algorithm that defines dynamic regret and cumulative constraint violations to measure the performance of the algorithm. To reduce the computational complexity of projection in the improved online saddle point algorithm, the projection is reformulated as quadratic programming, which can be solved efficiently by convex optimization. Kiani (2018). proposes an edge computing-aware non-orthogonal multiple access (NOMA) technique that takes advantage of uplink NOMA to reduce the uplink energy consumption of MEC users. The power control convex optimization problem solved independently for each NOMA cluster is formulated. Ren (2019). proposed a joint communication and computation resource allocation problem to minimize the weighting and delay of all mobile devices. Based on this, the joint communication and computational resource allocation problem is further transformed into an equivalent convex optimization problem, and a closed-form computational resource allocation strategy is obtained using convex optimization theory.

4.7 Others

In addition to the mathematical models introduced above, Xiao et al. (2022) modeled the optimization problem as a max-min optimization problem in order to minimize the energy and delay cost of the CAV. Zhang et al. (2021) formulated the optimization problem as a non-convex and NP-hard quadratically constrained quadratic programming problem (QCQP) by considering the system cost as a weighted sum of computational and delay cost (including transmission delay and processing delay) for all users. With the goal of maximizing system utility, Ning (2021) proposes a long-term optimization problem by comprehensively consid-

ering the constraints of server storage capacity and service execution delay. Then, the long-term optimization problem is decomposed into a series of immediate optimization problems using the Lyapunov optimization method. Then, a stochastic algorithm based on sample average approximation is proposed to approximate the expected future system utility.

Specifically, Xiao et al. (2022) considers constraints such as communication energy consumption, time delay, vehicle mobility and image acquisition quality, considering a scenario with$n$ moving vehicles and many distributed edge servers, and the vehicle speed v_n changes very little. Within the service range $\mathcal{L}$ of each edge server, the distance between the moving vehicle and the service area of the edge server is l_n, so the vehicle's residence time τ_n can be expressed as

$$\tau_n = \frac{\mathcal{L}-l_n}{v_n}. \tag{32}$$

Let T represent the exposure time of the camera, f represents the focal length of the camera, Q is the horizontal pixel size of the camera, d is the vertical distance from the object to the camera, v_n' is the instantaneous relative speed of the moving vehicle and the subject. Considering only the parallel movement of the subject and the moving vehicle camera, the image quality collected can be expressed as

$$L_n = \frac{Tf}{dQ} v_n', \tag{33}$$

Tf/dQ is determined by the fixed parameters of the camera, so the quality of the collected image is directly proportional to the relative speed. Therefore, we need to put forward certain parameter requirements for the acquisition camera to meet the requirements for high-quality images. In federated learning (FL), the processing time of local computation is expressed as

$$T_n^{com} = \log\left(\frac{1}{\theta}\right) \frac{|D_n| q_n}{f_n}, \tag{34}$$

q_n is the average CPU cycle of the processing task, so when the sample data volume is D_n, a total of $|D_n|\, q_n$ CPU cycles are required for processing. Then f_n is used to represent the local computing power, which is the number of CPU cycles per second. The time spent in one training iteration can be expressed as $|D_n|\, q_n / f_n$, and the corresponding processing time when local convergence and accuracy is θ is expressed as (34). Let p_n^{com} represent the computing power of the CPU, k_n is the number of effective switched capacitors, and the computing power can be expressed as

$$p_n^{com} = k_n f_n^3. \tag{35}$$

Combining (34) and (35), the local computing energy consumption is expressed as

$$E_n^{com} = p_n^{com} T_n^{com} = \log\left(\frac{1}{\theta}\right) k_n\, |D_n|\, q_n f_n^2. \tag{36}$$

Let the constant K_0 represent the error recovery system performance of forward error correction technology (FEC), RRT represent the average round-trip time of the wireless

connection, and Z_n be the mobility function of V_n in the TCP/IP protocol, the vehicle connection state α_n can be expressed as

$$\alpha_n \triangleq \frac{K_0\sqrt{Z_n}}{RRT}. \tag{37}$$

At this time, the wireless transmission rate r_n is expressed as

$$r_n = \alpha_n\sqrt{p_n}, \tag{38}$$

p_n is the wireless transmission power. Let the local model parameter size during FL be expressed as C_n, and it can be deduced that the transmission time T_n^{tran} for transmitting local parameters to the edge server is

$$T_n^{tran} = \frac{C_n}{r_n} = \frac{C_n}{\alpha_n\sqrt{p_n}}. \tag{39}$$

From (38), we can get $p_n = (r_n/\alpha_n)^2$, and we can get the local parameter transmission energy consumption as

$$E_n^{tran} = p_n T_n^{tran} = \frac{C_n\sqrt{p_n}}{\alpha_n}. \tag{40}$$

Therefore, when we minimize the weighted sum $\mathcal{S}_n$ of the vehicle's energy consumption and delay cost during FL, we can express the optimization problem as

$$\min_{a_n,\theta,f,p} \max_n \mathcal{S}_n, \tag{41a}$$

$$s.t. a_n \in \{0,1\}, \forall n \in \mathcal{N}, \tag{41b}$$

$$a_n\left(T_n^{com} + T_n^{tran}\right) \leq \tau_n, \forall n \in \mathcal{N}, \tag{41c}$$

$$E_n^{tran} \leq E_{max}^{tran}, \forall n \in \mathcal{N}, \tag{41d}$$

$$a_n L_n \leq L_{max}, \forall n \in \mathcal{N}, \tag{41e}$$

$$0 \leq f_n \leq f_n^{max}, \forall n \in \mathcal{N}, \tag{41f}$$

$$0 \leq \theta \leq 1. \tag{41g}$$

Where E_{max}^{tran} represents the maximum transmission energy consumption, L_{max} represents the maximum acceptable image blur level, and f_n^{max} represents the highest CPU frequency. It is worth noting that (41c) limits the dwell time of the vehicle to ensure that when receiving service, at least one local training completes and updates the parameters

Zhang et al. (2021) Use$i \in \mathcal{N} \triangleq \{1,\ldots,N\}$ and $j \in K \triangleq \{1,\ldots,K\}$ to represent the set of users and the set of service types, respectively, and uses S_{ij}^U, S_{ij}^D and S_{ij}^C to represent the amount of task data, the processing results and the processing cycle time. Specifically, the storage capacity constraint is expressed as

$$\sum_{i=1}^{K} a_j r_j \leq R, \tag{42}$$

The service storage variable $a_j \in \{0,1\}$. When a_j is 1, it means caching the service and 0 means not caching. r_j is the storage space occupied by the service. The bandwidth allocation of user i is represented by b_i^u and b_i^d . The former represents the uplink, and the latter represents the downlink. At the same time, B^U and B^D represent the total bandwidth capacity of the uplink and downlink, respectively. The bandwidth constraint is then expressed as

$$\sum_{i=1}^{N} b_i^u \leq B^U, \sum_{i=1}^{N} b_i^d \leq B^D. \tag{43}$$

The offloading decision of the task is represented by $o_{ij} \in \{0,1\}$. When o_{ij} is 1, it means offloading to the edge node for processing, and 0 means it is processed locally. At the same time, the task offloading decision is also affected by the service cache decision, so the task offloading constraint is expressed as

$$o_{ij} \leq a_j, i \in N, j \in K. \tag{44}$$

When user i decides to offload tasks to edge nodes for processing, the processing time is expressed as $T_{ij}^a = S_{ij}^C / f_i^a$, where f_i^a is the calculation rate assigned to the user by the edge node. At the same time, the computation rate of all users is affected by the computational power of the edge nodes f^A , so the computing constraint is expressed as

$$\sum_{i=1}^{N} f_i^a \leq f^A. \tag{45}$$

At this time, the transmission and computing resource allocation decision of user i can be expressed as $\mathbf{b}_i = [b_i^a, b_i^d, f_i^\alpha]^T$, and C_i represents the computing cost of user i , T_i^A and T_i^C represent the total transmission delay and total calculation delay of the task respectively. The specific formula is as follows

$$C_i \triangleq \sum_{j=1}^{K} \left(C_{ij}^A o_{ij} + C_{ij}^C \left(1 - o_{ij}\right) \right), \tag{46a}$$

$$T_i^A = \sum_{j=1}^{K} T_{ij}^A o_{ij}, i \in \mathcal{N}, \tag{46b}$$

$$T_i^C = \sum_{j=1}^{K} T_{ij}^C \left(1 - o_{ij}\right), i \in \mathcal{N}. \tag{46c}$$

When constructing the optimization problem QCQP, the auxiliary variable t_i is used instead of $\max\{T_i^A, T_i^C\}$ to simplify the derivation process, 5 auxiliary variables $t_i^u, t_i^d, t_i^a, t_i^{\bar{u}}, t_i^{\bar{d}}$

are introduced to convert T_i^A and T_i^C into quadratic form. In addition, rewrite all variables of user i as decision vector $\mathbf{w}_i$:

$$\mathbf{w}_i \triangleq [o_{i1}, \ldots, o_{iK}, t_i^u, b_i^u, \bar{t_i^u}, t_i^d, b_i^d, \bar{t_i^d}, f_i^a, t_i^a, t_i]^T, i \in N, \tag{47}$$

and rewrite all service cache decisions of edge nodes into decision vectors:

$$\mathbf{w}_0 \triangleq [a_1, \ldots, a_K, 0_{1\times 9}]^T. \tag{48}$$

Then introduce a new variable $\mathbf{x}_i = [\mathbf{w}_i^T, 1]^T, i \in \mathcal{N} \cup \{0\}$, and then the storage capacity constraint (42) is rewritten as:

$$\mathbf{x}_0^T \mathbf{A}_0^R \mathbf{x}_0 \leq R, \tag{49}$$

Similarly, the bandwidth resource allocation constraint (43) and the computation allocation constraint (45) can be rewritten as

$$\sum_{i=1}^{N} \mathbf{x}_i^T \mathbf{A}_i^U \mathbf{x}_i \leq B^U, \sum_{i=1}^{N} \mathbf{x}_i^T \mathbf{A}_i^D \mathbf{x}_i \leq B^D, \sum_{i=1}^{N} \mathbf{x}_i^T \mathbf{A}_i^A \mathbf{x}_i \leq f^A. \tag{50}$$

The offload policy constraint (44) can be rewritten as

$$\mathrm{diag}(\mathbf{e}_j)\mathbf{w}_i - \mathbf{w}_0 \leq 0, i \in N, j \in K. \tag{51}$$

Now, we can express QCQP as follows:

$$\min_{\{\mathbf{x}_i\}} \sum_{i=1}^{N} \mathbf{x}_i^T \mathbf{A}_i^P \mathbf{x}_i, \tag{52a}$$

$$\begin{aligned} &s.t.\,(49), (50), (51), \\ &\mathbf{x}_i \geqslant 0, i \in N \cup \{0\}, \end{aligned} \tag{52b}$$

$$\mathbf{x}_i^T \mathbf{A}_i^I \mathbf{x}_i = 0, i \in \mathcal{N} \cup \{0\}, j \in \mathcal{K}, \tag{52c}$$

$$\mathbf{x}_i^T \mathbf{A}_i^\pi \mathbf{x}_i \leq 0, \pi \in \{u, d, a, t\}, i \in \mathcal{N}, \tag{52d}$$

$$\mathbf{x}_i^T \mathbf{A}_i^{\bar{t}} \mathbf{x}_i \leq -\sum_{j=1}^{\kappa} M_{ij}, i \in \mathcal{N}. \tag{52e}$$

Although researchers have conducted many studies on establishing mathematical models, we still expect new ideas to emerge to help optimize edge computing costs.

By integrating service implementation utility and migration costs (Ning 2021), derives the utility of MUi for the whole service execution procedure:

$$U_i = \beta \left(U_i^{\tau\, u \to p} + U_i^{\tau\, p \to b} + U_i^{\tau\, b \to c} \right) + \gamma \sum_{t=\tau_i^p+1}^{\tau_i^b} E_i(t). \tag{53}$$

Where $E_i(t)$ is the service migration cost, $U_i^{\tau\, u \to p}$ is the uploading cost of MU, $U_i^{\tau\, p \to b}$ is the processing cost of PEC server, $U_i^{\tau\, b \to c}$ is the backhaul cost of server. With the objective of maximizing the system utility, the stochastic process optimization problem can be formulated by:

$$\max_{\xi \in \xi} \alpha \sum_{k \in \mathcal{K}} h_k(\xi, t) - \sum_{i \in \mathcal{N}} U_i \tag{54}$$

$$s.t. \sum_{i=1}^{N} \xi_{i,k}(t)\, d_i \le D_k, \forall\, k \in K, \tag{54a}$$

$$\left| \tau_i^e - \tau_i^u \right| \le T_i^{max}, \forall\, i \in N, \tag{54b}$$

$$\xi_{i,k} \in \{0,1\}, \forall\, i \in N, k \in K. \tag{54c}$$

Where (54a) requires that the loaded data size of PEC server k cannot exceed its maximum storage capacity. (54b) guarantees that the service request can be accomplished within the tolerable latency. (54c) illustrates that the service request of each MU can be merely allocated to one PEC server in each time slot. However, it is unrealistic to derive a long-term solution to the above constraints, since it requires the global future information of all MUs. Therefore, we take maximizing the system utility in each time slot as the goal and reformulate the optimization problem as follows:

$$\max_{\xi \in \xi} \alpha \sum_{k \in \mathcal{K}} h_k(\xi, t) - \sum_{i \in \mathcal{N}} \mathcal{U}_i(t), \forall\, t \in T \tag{55}$$

$$\sum_{i=1}^{N} \xi_{i,k}(t)\, d_i \le D_k, \forall\, k \in K, \tag{55a}$$

$$|\tau_i^e - t| \le |T_i^{max} - t + \tau_i^u|, \forall\, i \in N, \tag{55b}$$

$$\xi_{i,k} \in \{0,1\}, \forall\, i \in N, k \in K. \tag{55c}$$

There are two main differences between problem (54) and (55). First, the utility function in (55) represents the utility of each time slot. Second, the service execution latency starts at time slot t, and the latency threshold needs to be minus the already consumed time.

5 Evaluation methods

This section outlines the sources of the experimental data as well as the experimental environment, summarizes the commonly used evaluation metrics in existing work, and gives several examples of corresponding applications. Reasonable evaluation methods can objectively assess the cost optimization effect and clearly reflect the strengths and weaknesses of the algorithms or models, which are crucial for promoting the development of edge computing networks. The outline of evaluation methods is shown in Fig. 6 and is described in detail below.

5.1 Data sets

In the process of model training, the quality and diversity of datasets directly affect the performance, generalization ability and practical application effect of the model. Testbeds can provide a real network environment and device behaviour, as well as generate real training and test data with high authenticity and reliability. Simulators can quickly generate a large amount of data but cannot fully reflect the complexity and uncertainty of the real world. Real data reflects the complexity and diversity of real-world application scenarios, and the models obtained from training usually have high practical application value. However, the acquisition cost of real datasets is expensive, and privacy and security issues are faced. Synthetic datasets enrich the diversity of datasets but may still lack certain realistic details and randomness. Synthetic datasets require feasible synthesis methods, and it is also a challenge to produce suitable synthetic datasets. Therefore, in the process of model training, the appropriate type of dataset should be chosen based on the specific application scenario, the difficulty of data acquisition, and the model requirements.

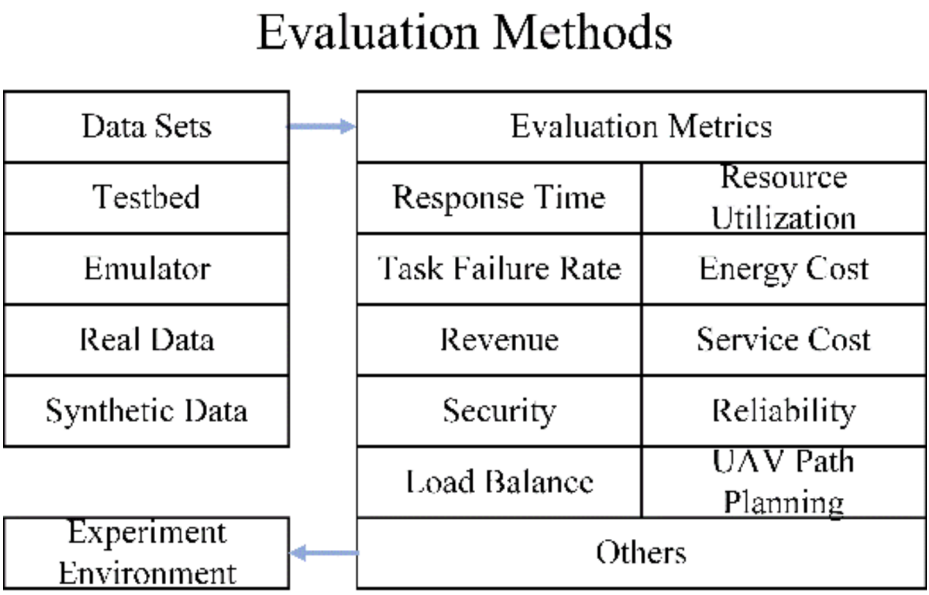


Fig. 6 Outline of evaluation methods

5.1.1 Testbed

Testbed can provide consistent experimental conditions with actual application scenarios and can generate real network traffic and device behavior data, which are highly realistic and can more accurately reflect the performance of algorithms or models in the actual environment. Testbeds can also provide consistent experimental conditions to ensure the repeatability of experiments and avoid uncertainties that may occur in the virtual environment. By conducting experiments on Testbed, researchers can more accurately assess the performance and applicability of algorithms or models, obtain experimental results with practical application value, and promote the development of technology and the landing of applications. Details of open-resource Testbed are presented in Table 10. We then detailed multiple testbeds.

EdgeNet is a public Kubernetes cluster dedicated to network and distributed systems research, supporting experiments that are deployed concurrently by independent groups. Its nodes are hosted by multiple institutions around the world. EdgeNet is up and running at over 40 nodes worldwide, including 5 in Europe, 1 in Brasil, 1 in Australia, and the oth-

Table 10 Details of open-resource testbeds

Testbed	Type	Applicable research areas	Features
EdgeNet	Real World	Distributed computing; Networking; Storage	The EdgeNet cluster currently consists of 72 vCPUs and 96 GB of memory. A typical EdgeNet node has 2 GB of memory, 2 vCPUs, and 15 GB of storage.
CloudLab	Real World	Computing infrastructure; Platform services; Applications	CloudLab gives users unmediated "raw" access to hardware. Wisconsin's hardware is designed for networking and storage work, Clemson's for analytics and high-performance workloads, and Utah's for scale-out workloads.
FIT loT-LAB	Real World	IoT; WSN	FIT loT-LAB is composed of 2728 low-power wireless nodes and 117 mobile robots. Some nodes are equipped with GPS
SMOTEC	Real World	IoT; Smart objects	SMOTEC supports plug-and-play Docker container integration with the SUMO simulator and interconnected Raspberry Pi devices.
BenchFaaS	Real World	Edge computing; Serverless computing; IoT	Users can run serverless functions over a Kubernetes cluster of VMs and Raspberry Pis (RPs).
CIPyZ	Virtual	Smart city; IoT; Information communication technology	The client-end has a web interface built-in PHP and the back-end database is developed using MySQL. It supports diversified virtualization file and disk formats and more can be added. It can be installed on any Linux distribution supporting standard shell programming for OS level operations.
Fogbed	Virtual	Fog computing architecture; Protocols; Applications	Fogbed enables the deployment of fog nodes as software containers under different network configurations. Fogbed extends the Mininet framework to allow the use of Docker containers as virtual nodes.
EmuFog	Virtual	Behavior and performance of fog computing nodes	EmuFog allows developers to design the Fog infrastructure topology in an extensible manner through a two-step process

ers in the United States. The EdgeNet cluster currently consists of 72 vCPUs and 96 GB of memory. A typical EdgeNet node has 2 GB of memory, 2 vCPUs, and 15 GB of storage.

CloudLab is a testbed for research and education in cloud computing, it gives users unmediated "raw" access to hardware. The primary CloudLab hardware is hosted at three sites: the University of Utah, Clemson University, and the University of Wisconsin–Madison. Wisconsin's hardware is designed for networking and storage work, Clemson's for analytics and high-performance workloads, and Utah's for scale-out workloads.

FIT loT-LAB is an open testbed composed of 2728 low-power wireless nodes and 117 mobile robots available for experimenting with largescale wireless loT technologies. The loT-LAB Testbed is deployed in 6 sites across France. Each site features different node and hardware capabilities, but all sites are interconnected and available through the same web portal, common REST interfaces and consistent eLI tools. Some nodes (200 in Rocquencourt, and 32 Grenoble) are equipped with GPS.

Nezami et al. (2023) introduces SMOTEC, a novel open-source testbed for adaptive smart mobility experimentation with edge computing. SMOTEC provides a modular end-to-end instrumentation for prototyping and optimizing the placement of intelligence services on edge devices, such as augmented reality and real-time traffic monitoring, for the first time. SMOTEC supports plug-and-play Docker container integration with the SUMO simulator and interconnected Raspberry Pi devices. SMOTEC is, by design, a distributed system that consists of at least one orchestrator and a set of edge nodes.

Carpio et al. (2023) introduced BenchFaaS, an open-source edge computing network testbed which automates the deployment and benchmarking of serverless functions. Users can run serverless functions over a Kubernetes cluster of VMs and Raspberry Pis (RPs). It complements the related community-driven experimental studies focused on the performance of different serverless platforms over third-party cloud/edge services. It also complements other related efforts focused on open-source serverless platforms but without consideration of the underlying networked infrastructure.

Nayyer et al. (2020) developed a novel open-source virtualization platform CIPyZ。 The main features of ClPyZ include custom orchestration, resource tracking, resource monitoring and alert, GUI for clients, multi-cloud management, centralized control and visibility, scalability and recourse sharing, and VM migration. The server end of ClPyZ is developed in Python 2.7 language using the Eclipse PyDev extension and PsUtil library. The client end has a web interface built-in PHP, and the back-end database is developed using MySQL. It introduces a centralized brokerage system to manage multiple cloudlets belonging to different clouds. ClPyZ addresses the heterogeneity problems of cloudlets by offering platform-independent, flexible, and generalized features that can be customized or redefined as needed. It supports diversified virtualization file and disk formats, and more can be added. ClPyZ has been extensively tested and is ready for production. It can be installed on any Linux distribution and supports standard shell programming for OS-level operations.

Fogbed is a framework and toolset integration for rapid prototyping of fog components in virtualized environments. Using a desktop approach, Fogbed enables the deployment of fog nodes as software containers under different network configurations. Its design meets the requirements of low cost, flexible setup and compatibility with real-world technologies. Fogbed extends the Mininet framework to allow the use of Docker containers as virtual nodes.

EmuFog is an extensible emulation framework tailored for Fog Computing scenarios—that enables the from-scratch design of Fog Computing infrastructures and the emulation of real applications and workloads. EmuFog is an extensible emulation framework built on top of MaxiNet that allows developers to design the Fog infrastructure topology in an extensible manner through a two-step process (Şenel et al. 2021; Coutinho et al. 2018; Adjih 2015; Coutinho et al. 2018; Mayer 2017).

Pepito (2021) highlights an open-source 5G security testbed that was developed for the purpose of edge computing security research. It provides comparative simulation results from the Testbed, highlighting the ability to control latency, latency distribution, packet loss, packet corruption and quality of service for an edge or central cloud server interface. It can also serve as a baseline for validating 5G network edge computing security. The Testbed is virtualized, and all components are hosted on a single server running VMWare's ESXi hypervisor.

Muñoz et al. (2022) present the first such mostly hardware-agnostic wireless testbed automation framework, Ainur. It is designed to deploy experimental runs from a workload perspective by configuring the physical Testbed, initializing all involved software components, deploying and executing the experimental workload, collecting logs and data, and finally, gracefully degrading the system. Ainur is built on top of cloud-native technologies such as Docker and is provided as FOSS to the community through the KTH-EXPECA/Ainur repository on GitHub.

Diao (2019) present a scalable testbed for task offloading and deployment of heterogeneous edge computing adopting cross-platform container technique to encapsulate the software environment and runtime dependencies of heterogeneous computing units. The CPUs of this system include a variety of ARM architecture processors and Intel processors. Dedicated deep neural network inference chips, including NVIDIA Jetson TX2 and Intel Neural Computing Stick, are also integrated into the Testbed.

Mavromatis and Simeonidou (2020) builds an IoT testbed with an edge computing orientation, and constructs a testbed outdoors in the city center of Bristol (United Kingdom). The Testbed is able to host heterogeneous devices and accommodate multiple applications designed for the edge and cloud network. Bristol IoT testbed is built based on open-source software such as Metal as a Service (MAAS) ubuntu, Kubernetes as the service orchestrator and different implementations of Context Information Management (CIM). Bristol IoT testbed is equipped with wireless interfaces for sensor devices enabling WIFI, Cellular, 6LowPAN and Bluetooth Low Energy (BLE).

Yamanaka et al. (2021) present an edge computing testbed system that automatically determines which servers to place Docker containers on according to the testbed users' preferred RTTs between the servers and the wireless base stations. The proposed system reduces the configuration data size to one divided by the number of servers. The key idea is that the virtual region-based interface reduces configuration data size by allowing testbed users to specify the preferred RTT without seeing the RTTs in a network.

5.1.2 Emulator

Emulators offer a controllable environment for mimicking the diverse resources, devices, and application scenarios within an edge computing network. Researchers utilize these emulators to assess the impact of various resource management and scheduling strategies

on performance metrics like latency, throughput, and energy consumption. They also facilitate the comparison of different algorithms and strategies, enabling the selection of the most optimal solutions. Figure 7 outlines the relationship between emulators.

CloudSim is a widely used open-source simulator for modeling resource management and scheduling strategies in cloud computing environments. Its high configurability aids researchers in simulating and evaluating different resource management and scheduling strategies to optimize performance and efficiency in cloud computing settings.

FogSim, based on CloudSim, is tailored for simulating and evaluating resource management and scheduling strategies within fog computing environments. It offers modeling and simulation capabilities for edge nodes, fog nodes, and cloud nodes, as well as simulating network topologies and data centers.

iFogSim, built upon CloudSim and FogSim, is designed to simulate resource management and scheduling strategies in edge computing environments. It provides modeling and simulation for edge nodes, fog nodes, cloud nodes, and MDs.

EdgeCloudSim is an extension framework based on CloudSim, designed for simulating resource management and scheduling strategies in edge cloud environments. It facilitates the modeling and simulation of edge nodes, cloud nodes, and user terminals.

MobEdgeSim, built on Network Simulator 3 (NS-3), is a simulation tool for edge computing that focuses on resource management and scheduling strategies in mobile edge computing scenarios. It offers modeling of MDs and edge nodes, simulating the communication and collaboration between them.

GreenEdge is a simulator for green edge computing networks aimed at researching and evaluating strategies to enhance energy efficiency and reduce carbon footprints within edge computing environments. It simulates the energy consumption and supply of edge nodes, along with various factors related to energy management.

EdgeSim is a general-purpose edge computing network simulator based on OMNeT++, designed to simulate and evaluate various resource management, scheduling, and network topology schemes. It offers modeling capabilities for edge nodes, cloud nodes, and user terminals, as well as simulations of network topologies and communication protocols.

Network Simulator 2 (NS-2) is a widely used network simulation tool that operates as a discrete event simulator, supporting simulations of both wired and wireless networks. It provides a rich set of network models and protocol simulation capabilities for assessing the performance and behavior of computer networks.

NS-3 is the successor to NS-2 and another widely utilized network simulation tool. It supports simulating various network technologies and protocols, including wired, wire-

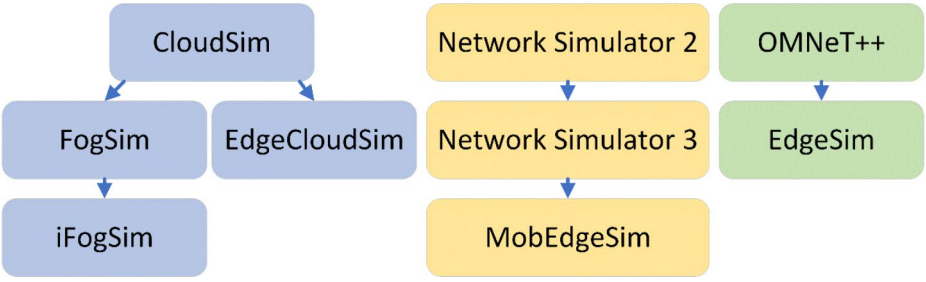


Fig. 7 Relationship between emulators

less, and sensor networks. Compared to NS-2, NS-3 offers a more modern architecture and enhanced simulation capabilities.

Mekala (2022)used the FogSim to simulate Laser Interferometer Gravitational-Wave Observatory (LIGO), Montage and Cybershake real-time workload data, and workloads of various sizes such as 100, 150, and 250 errands with the DAG phenomenon. Hosseinzadeh (2021). used the iFogSim in conjunction with the CloudSim to handle events between MEC components. Chai et al. (2023) used a Python simulator to randomly generate different sets of DAG to model different task dependencies. The generated DAG sets validate the proposed scheme. Da and Costa (2023) conducts simulation experiments in the Urban Traffic Simulator (SUMO). It reproduced vehicle movements from 6 a.m. to 8 a.m. on weekdays using a 114-square-kilometer sub-map of the city of Cologne, Germany, where the number of vehicles varies considerably. Ma et al. (2021) used VanetMobiSim to generate realistic motion of vehicles, and NS-2 was used to simulate the behavior of edge computing. Wang et al. (2023) conducted several simulation experiments using NS-3 to verify the accuracy of link quality prediction.

5.1.3 Real data

Real datasets consist of actual data samples from the real world, reflecting true aspects of user behavior, application demand, and network traffic, thus providing an accurate simulation of edge computing scenarios. Researchers can identify potential issues and challenges by analyzing and mining real data and proposing appropriate solutions. Research validated by real datasets carries more persuasive power. However, when utilizing real datasets, it is crucial to adhere to privacy and security principles, ensuring that dataset usage complies with relevant legal and ethical standards. Researchers should implement suitable measures for data anonymization, desensitization, and protection to safeguard individual privacy and sensitive information. Details of commonly used public real datasets are presented in Table 11.

In addition to using public datasets, some researchers used real datasets collected by themselves. Guo (2022). uses a real base station dataset from Shanghai, which includes about 3000 base stations, and the paper randomly selects the locations of 300 of these base stations as the addresses of the edge servers and deploys 30 microservices on each server. In addition, Awad Abdellatif (2021) uses EMOTIV EPOC + to collect data from patients receiving routinely scheduled treatments at Hamad Medical Corporation's Patient Rehabilitation Center. Specifically, the data for this dataset was collected from 30 patients who received intravenous antibiotic medications, and each patient was monitored for 30 min before, during, and after administering the medication. By transmitting these data, the authors demonstrated that the proposed BCO scheme could send emergency data with the lowest latency and computational cost and send non-emergency data with the highest level of security. Gao et al. (2023) used a real dataset collected by the MIT Media Lab that contains location information of 94 testers using the service over 9 months. Although this dataset provides two features representing locations, cell tower ID (cellID) and areaID, the paper considers a privacy-oriented computational offloading environment. Therefore, only areaID locations were chosen for the paper, which includes 174 services, 703 location points, and 94 users.

Table 11 Details of commonly used Public Real datasets

Dataset	Size	Columns	Details
Geolife	17,621 tracks	6	Data collection period: April 2007 to August 2012; Number of users: 182; Total track distance: 1,251,654 km
Google Cluster	Millions to hundreds of millions of rows	Machine Events: 7 Machine Attributes: 5 Job Events: 8 CollectionEvents: 17 InstanceEvents: 13 InstanceUsage: 18	ClusterData2019 provides data from eight Borg cells over the month of May 2019; ClusterData2011 provides data from a single 12.5k-machine Borg cell from May 2011; TraceVersion1 is a short trace that describes a 7-hour period from one cell from 2009
MNIST	Train: 60,000 Test: 10,000	A picture of handwritten numbers from 0 to 9 and the corresponding labels	Widely used in machine learning and deep learning; Image size: 28×28 pixels; Color: single channel;
FEMNIST	Total: 341,873 Train: 194,373 Test: 77,500 Verify: 70,000	A handwritten character image and corresponding label	Based on the EMNIST dataset, mainly used for federated learning; image size: 28×28 pixels; color: single channel;
Melbourne Mobile Dataset (some papers call it EUA)	More than 50,000 tracks	User information: 4 Cellular base stations: 4 Mobile devices: 4 Communication records: 7	Contains real cellular network topology in urban environments; mobile device trajectory data reflects daily mobility patterns
Planetlab CoMon	Millions of records	25	The data collection frequency is high; it has a wide geographical coverage; and the time span is long;
WSN-DS	Over 2 billion records	sensor_data:16 node_status:8 link_quality: 6 event_log: 9	Data collected by 54 sensor nodes over a 3-year period
N-BaIoT	Over 7,000,000 records	normal_traffic: 80 attack_traffic: 23 device_info:9	Covers 23 common IoT attacks; various device types; rich traffic characteristics

5.1.4 Synthetic data

Researchers can generate a large volume of samples through synthetic datasets when there is a lack of sufficient sample size, covering various boundary conditions, anomalies, and rare events to enhance the model's robustness and generalization capability. Synthetic datasets allow researchers complete control over the attributes and distribution of data to meet specific research needs and experimental designs, aiding in the experimentation and evaluation of particular problems. For sensitive data, synthetic datasets can prevent privacy breaches associated with real data. Moreover, because the data generation process can be accurately recorded and reproduced, synthetic datasets ensure the repeatability of experiments. However, synthetic datasets may not fully capture the complexity and diversity of the real world. In some cases, they may not grasp all the details and variations of real scenarios, and the performance of models trained on synthetic data may slightly differ from their performance in real-world applications.

Deng (2021) created several edge service provisioning systems, and the specific system configuration settings are listed in Table 3 in the paper. Liu et al. (2022) generated sample data using the model $y = argmax\left(softmax\left(\boldsymbol{\omega}\, x + b\right)\right), x \in \mathbb{R}^{60}, \boldsymbol{\omega} \in \mathbb{R}^{10\times 60}, b \in \mathbb{R}^{60}$. For service k, the generated sample data is (x_k, y_k) with heterogeneity among the sample data.

5.2 Evaluation metrics

Evaluation Metrics are important basis for measuring model performance. Reasonable evaluation indicators can help us objectively evaluate the pros and cons of models and provide a basis for model optimization and selection. In the following content, we will introduce some commonly used evaluation indicators one by one. We will also give examples of how researchers express these indicators in combination with problems.

5.2.1 Response time

Response time refers to the total time elapsed from sending a request to receiving a response, typically comprising four main components: request transmission time, queue waiting time, task execution time, and response transmission time. It is a crucial aspect of user experience perception. Response time, also known as execution time, denotes the duration required to completely execute a workload, proving especially pertinent in evaluating performance tests, productivity applications, and graphics-oriented workloads. Request transmission time is when a request is sent until it reaches an edge computing node. Task execution time refers to the duration needed to process a request at an edge computing node, including computations, data processing, and algorithm execution. Response transmission time is when an edge computing node generates a response and transmits it back to the requester. In summary, a lower response time indicates that tasks can be processed and responded to swiftly within the edge computing network, enhancing user experience and service quality (Mahenge et al. 2022; Mekala 2022; Gao et al. 2022; Ma et al. 2023).

Dai (2023) expresses the communication rate from the user equipment to the MEC server as

$$r_{n,m}^{t,\mathrm{mec}} = B_{\mathrm{mec}} \log_2 \left(1 + \frac{p_{n,m}^{t,\mathrm{mec}} | h_{n,m}^{t,\mathrm{mec}} |^2}{\delta_{\mathrm{mec}}^2}\right), \tag{56}$$

$p_{n,m}^{t,\mathrm{mec}}$ is the channel power, $h_{n,m}^{t,\mathrm{mec}}$ is the transmit power, and the wireless bandwidth and noise power are B_{mec} and δ_{mec}^2, respectively, and remain fixed. Correspondingly, the request transmission time is expressed as $T_{n,m,\mathrm{mec}}^{t,\mathrm{trans}} = b_n^t / r_{n,m}^{t,\mathrm{mec}}$, where b_n^t represents the data bits of task n at time slot t. If the task needs to be relayed, a network propagation delay $t_{n,m}^{t,\mathrm{pro}}$ needs to be added. In the case of network congestion and queuing, the queue waiting time is expressed as $T_{n,m,\mathrm{mec}}^{t,\mathrm{queue}} = t_{\mathrm{exp}} / 1 - t_{\mathrm{exp}} c_0$, where t_{exp} is the expected delay for sending and receiving data without network congestion, and c0 is the normalized computation tasks processed by MEC cooperation. The task execution time can be expressed as $T_{n,m,\mathrm{mec}}^{t,\mathrm{compute}} = c_n^t / f_{m,n}^{t,\mathrm{mec}}$, c_n^t is the computational requirement of task n, $f_{m,n}^{t,\mathrm{mec}}$ is the allocated computing resources. Then, the total offloading delay of task n is expressed as $T_{\mathrm{mec}}^{t,n} = T_{n,m,\mathrm{mec}}^{t,\mathrm{trans}} + T_{n,m,\mathrm{mec}}^{t,\mathrm{queue}} + T_{n,m,\mathrm{mec}}^{t,\mathrm{compute}}$.

Zhang et al. (2021) modeled the optimization problem of task offloading and resource allocation in MEC as a mixed-integer programming problem. In this paper, the task j processed by AP for user i is represented as $T_i^A = \sum_{j=1}^{K} (T_{ij}^u + T_{ij}^d + T_{ij}^a) o_{ij}$, o_{ij} is a discrete 0–1 decision variable. $T_{ij}^u = S_{ij}^U / (\eta_i^u b_i^u)$ and $T_{ij}^d = S_{ij}^D / (\eta_i^d b_i^d)$ represent the wireless transmission and reception time between the user and the AP, S is the processing cycle required to process the data, b is the link bandwidth, η is link spectral efficiency.

Tang (2022) first expresses the total computing power (in CPU cycles per second) of each computing node$n$ in a time slot τ as

$$\mu_{n,f}(\tau) = \frac{\Delta\tau - t_{n,f}}{\Delta\tau} q_{n,f}(\tau)\omega_n + (\delta_{n,f}(\tau) - q_{n,f}(\tau))\omega_n. \tag{57}$$

Where, $q_{n,f}(\tau)$ is the number of CPU cores that require a cold start, $\delta_{n,f}(\tau)$ is the computing resource allocation strategy, ω_n is the computing capability of each CPU core for edge computing node n. $t_{n,f}$ is the cold start delay. Therefore, the queue waiting time can be expressed as

$$D_{m,f}^{\mathrm{blog},n}(\tau) = \frac{\vartheta_{m,f}^n(\tau)\kappa_f \mathcal{B}_{m,f}(\tau)}{\widetilde{\mu}_{n,f}(\tau)}, \tag{58}$$

$\vartheta_{m,f}^n(\tau)$ is the number of tasks planned to be executed, $\mathcal{B}_{m,f}(\tau)$ is the data queue length. The task execution time can also be expressed as $D_{m,f}^{\mathrm{esec},n}(\tau) = \vartheta_{m,f}^n(\tau)\kappa_f / \omega_n$. When the task needs to be further offloaded to the Cloud Layer, the transmission delay and execution delay can be expressed as $D_{m,f}^{\mathrm{tcc}}(\tau) = \vartheta_{m,f}^n(\tau) z_f / c_0$ and $D_{m,f}^{\mathrm{ecc}}(\tau) = \vartheta_{m,f}^n(\tau)\kappa_f / \omega_0$, where c_0 is the data rate between cloud center and loT clients, ω_0 is the computing capability of the cloud center. Note that the computing resources in the cloud center are sufficient, thus we assume that the waiting time at the buffer of the cloud can be disregarded.

5.2.2 Resource utilization

Resource utilization refers to the degree to which edge nodes or edge computing systems effectively use available resources, namely the proportion of time resources are consumed to execute workloads. Reasonable resource utilization can maximize the benefits of the

entire edge computing network system. Resources may include CPU, RAM, storage, bandwidth, and energy. Optimizing resource utilization can enhance the efficiency and performance of edge computing systems, with the goal of maximizing the revenue and profit of resource providers while also delivering better computing services and user experiences. However, it is essential to note that a higher CPU utilization rate is not always better. High CPU utilization on a single server may indicate negative factors such as excessive load or poor resource management efficiency (Mahenge et al. 2022; Cong et al. 2022; Naouri et al. 2021; Miao et al. 2022; Aslanpour 2020).

In Xiao (2022), each edge servere_j has a remaining capacity of $F_i^j = \{F_1^j, F_2^j, \ldots, F_i^j\}$, which denotes the service workload of u_i placed on server e_j. s_i is the service required by u_i. The aggregate workload of each resource type incurred by all allocated users must not exceed the remaining capacity of their assigned server. The total workloads generated by all users allocated to an edge server must not exceed its remaining capacity. The resource utilization is expressed as

$$R_j = \frac{w_j}{F^j} s.t. F^j \neq 0, w_j \leq F^j, R_j < \Omega_j. \tag{59}$$

Where w_j denotes the workloads undertaken by e_j, and it equals the sum of services' workloads on e_j. Ω_j is the maximum utilization rate permitted of e_j, and R_j should not exceed Ω_j.

Mekala (2022) expresses the CPU utilization of edge node$h$ as $\xi(h) = \sum_{i=1}^{|\mathrm{ES}_i|} \mathrm{A}_i / \sum_{i=1}^{|\mathrm{ES}_i|} \mathrm{B}_i$, B_i is the node activity time, A_i is the node processing time, and $|\mathrm{ES}_i|$ is the number of node activities.

Ning (2023) introduced the Price of Anarchy (PoA), which is widely used in game theory to measure the performance utilization of the system. PoA describes the gap between the performance of the system and the optimal solution in a non-cooperative environment. In simple terms, PoA=system performance under the worst Nash equilibrium / global optimal system performance. The article also comprehensively considers the wireless transmission conditions (i.e., transmission delay and energy consumption)θ. Under the random game model constructed in this article, PoA is rewritten as

$$PoA = \frac{\max\limits_{(\mathbf{s},\mathbf{a}) \in \mathrm{Y}} \sum\limits_{i \in \mathcal{N}} \theta_i Z_i(\mathbf{s},\mathbf{a})}{\sum\limits_{i \in \mathcal{N}} \theta_i Z_i(\mathbf{s}^*,\mathbf{a}^*)}. \tag{60}$$

Where θ reflects the wireless transmission conditions between the user equipment and the UAV, and Z_i is the performance of the user equipment.

Tang (2022) uses container cold start technology to avoid the situation where some CPU cores are not always able to provide services in a time slot. This paper calculates the number of CPU cores that require a cold start to launch task f in the time slot τ as follows

$$q_{n,f}(\tau) = \begin{cases} \delta_{n,f}(\tau) - \delta_{n,f}(\tau - 1), \delta_{n,f}(\tau) \geq \delta_{n,f}(\tau - 1) \\ 0, otherwise \end{cases} \tag{61}$$

Where $\delta_{n,f}(\tau)$ is the computing resource allocation strategy of the current time slot τ.

Hao (2021) introduced the energy transfer attenuation ratio (ETAR) to measure how much energy is lost when transferring between nodes and to optimize energy efficiency. In a containerized environment, the smallest ETAR is expressed as

$$METR_{n,m} = min\left\{METR_{n,m}^{l} | l \in [1, L_{n,m}]\right\}. \tag{62}$$

Where $METR_{n,m}^{l} = 1 - \prod_{j \in LS_{n,m}} (1 - DETR_{j,j+1})$ is energy transferring attenuation of l th route between node N_n and node N_m. It is worth noting that $DETR_{j,j+1}$ is the energy transfer attenuation between nodes and is a predefined parameter.

5.2.3 Task failure rate

Task failure scenarios include tasks being canceled or failing to respond to user requests within a given timeframe. The causes of task failures are diverse, such as node crashes where the task runs, resource overruns, network failures, and excessive queue times. A high task failure rate often indicates deficiencies in the edge computing network's resource scheduling strategy or load balancing, which can severely impact the user experience and reduce the anticipated revenue for resource providers. Sometimes, researchers also use the task completion rate to measure the success and failure of tasks (Ma et al. 2021; Naouri et al. 2021; Gao et al. 2023; Luo et al. 2022; Hui 2022).

Zhu et al. (2022) expressed the task completion rate as $\bar{\eta} = \sum_{i=1}^{k} \mu_i \rho_i / \sum_{i=1}^{k} \rho_i$, where $\mu_i \in \{0,1\}$ represents whether the task is successfully finished or not and ρ is the task size.

In Tang (2022), when the edge computing node queue is full, the computing tasks will be discarded. The number of tasks discarded by the edge computing node can be given by

$$\Psi_{n,f}(\tau) = max\left\{\mathcal{B}_{n,f}(\tau) + \Lambda_{n,f}(\tau) - \Omega_{n,f}(\tau) - \mathcal{B}_{n,f}^{max}, 0\right\}. \tag{63}$$

Here, $\Psi_{n,f}(\tau) > 0$ indicates that an event of edge computing queue overflow occurs. $\mathcal{B}_{n,f}(\tau)$ is the data queue length, the overall arrived task requests are given by $\Lambda_{n,f}(\tau)$. $\Omega_{n,f}(\tau)$ is the number of executions of task f processed by serverless edge computing node n at time slot τ, $\mathcal{B}_{n,f}^{max}$ is the maximum length of the data queue.

5.2.4 Energy cost

Energy cost associated with the energy used to run edge computing are among the primary expenses in operating such a network. These costs include expenses for electricity supply, device operation, and maintenance. Singh (2016). highlighted that the energy efficiency metric is widely utilized to optimize energy costs. This metric represents the ratio of the number of successfully executed workloads in an edge computing network to the total energy consumed to execute these workloads. However, not all devices within a node can track energy consumption. Some researchers describe energy cost as the total energy consumption expenses of a node during task execution, while others simply refer to the energy consumption expenses of computing devices executing a specific task. For scholars focused on improving system efficiency, it is more appropriate to discuss how much revenue is gen-

erated per unit of energy cost (Zhao 2022; Deng et al. 2021; Tang 2022; Gao et al. 2023; Zhu and Zhou 2021; Naouri et al. 2021; Xiao et al. 2022).

Kong (2022) assumes that BS and RSU have the same unit computing energy consumption and defines the total energy cost in time slot$t$ as $D_{\mathrm{cp}}(t)=\sum_{i=1}^{V}\sum_{j=1}^{K}\left(\eta_{ij}(t)e_0C_j+\eta_{i0}(t)e_0C_0\right)$. Where e_0 is unit computation energy consumption, C_j is the energy cost of RSU, C_0 is the energy cost of BS, $\eta_{ij}(t)$ and $\eta_{i0}(t)$ are the sizes of RSU and BS computing tasks, respectively. In addition, the article also expresses the caching energy cost of BS and RSU as $E_{i0}(t)=\xi_{i0}(t)a_0$ and $E_{ij}(t)=\xi_{ij}(t)a_0$, a_0 is the unit caching energy consumption, $\xi_{i0}(t)$ and $\xi_{ij}(t)$ are caching resources allocated by BS and RSU for vehicle i in timeslot t 。

Zhu and Zhou (2021) expresses the energy consumption of executing the task of useru_i locally as $e_i^c=\xi_i(\phi_i^u)^2\beta_i$, ξ_i denotes the effective switched capacitance of the CPU, ϕ_i^u (CPU cycles/second) denotes the computing capacity, β_i (CPU cycles) is the computational resource demand. The energy required for a user to send a task to an AP is expressed as $e_i^t=p_i\alpha_i/r_i^u$, p_i is the transmission power of user u_i, α_i(bits) is the input data size. r_i^u is the transmission rate of user u_i calculated according to Shannon's theorem when the available channel bandwidth is $w_i=B/\sum_{1\le j\le M\wedge\vartheta_j=\vartheta_i}$ and the channel gain and background noise are $g_{i,j}$ and σ^2 respectively. Thus, The total energy consumption of a user u_i is given as

$$e_i=e_i^c\lambda_i\left(1-\sum_{1\le j\le L}z_{i,j}\right)+e_i^t\lambda_i\sum_{1\le j\le L}z_{i,j}. \tag{64}$$

Denote average task arrival rate as λ_i, $z_{i,j}$ as the probability of offloading a task to cloudlet.

Naouri et al. (2021) represented the device energy consumption for each of the three layers of the architecture separately. In the cloud layer, the average node computing energy of each cluster is represented as:

$$e_{\mathrm{avg}}(S_i(j))=\frac{\sum_{j=1}^{|S_i(j)|}e(j)_{\mathrm{r}}}{|S_i(j)|}, \tag{65}$$

$e(j)_{\mathrm{r}}$ is the residual energy of the j_{th} member node. The energy cost of the edge server at the edge layer is

$$e_{(i)}=\frac{\mathrm{In}_i}{U}\cdot p_s+\frac{\omega_i}{f}\cdot p_j+\frac{\mathrm{out}_i}{D}\cdot p_r. \tag{66}$$

p_s and p_r are the transmitting energy for uploading and downloading data, and p_j represents the computation energy of device j for task i. In_i is the size of input data, out_i is the output data size, and ω_i is the workload of task i. f is the device CPU frequency, U and D are the uplink and downlink data transmission rates, respectively. The energy consumption of a user device performing a task at the device layer is represented as $E(i,j)=\omega(i,j)*P_j/f_j$, $\omega(i,j)$ represent task i workload on device j.

In Miao (2023), the flight energy consumption of the UAV from the takeoff point to the edge of the mission area is denoted as $E = (GD/V) * F$, with V being the UAV flight speed and F being the flight power. The flight path distance of the UAV is expressed as

$$GD = 2arc\sin\left(\frac{ED-r}{2(R+H)}\right)(R+H). \tag{67}$$

The formula is explained in detail in the UAV Path Planning section below. When entering the mission area, the flight energy consumption is expressed as $E_u^{\mathrm{Lo}} = \sum_{i=1}^{I}[E_{u,i}^{\mathrm{Ho}} + E_{u,i+1,i}^{\mathrm{lo}}]$. $E_{u,t}^{\mathrm{Ho}} = F_u^{\mathrm{ho}} T_{u,i}^{Ta}$ is the time that UAV hovers at task i, F_u^{ho} is the hovering time, and $T_{u,i}^{Ta}$ is the hovering power. $E_{u,i+1,i}^{\mathrm{lo}} = (GD'/V) * F$ is the flight energy consumption for flying from task i to $i+1$. At this point, the distance traveled by the UAV is denoted as

$$GD' = 2arc\sin\left(\frac{ED}{2(R+H)}\right)(R+H). \tag{68}$$

Both (67) and (68) use geodesic distance to calculate the flight distance of the UAV. However, (68) subtracts the radius of the mission area r when calculating the flight distance. When the UAV performs a return flight, the flight energy consumption is then expressed as

$$E_u^{Ba} = \frac{X^2+Y^2+H^2}{V}F. \tag{69}$$

This means treating the return path as the hypotenuse of a right triangle formed by the return point and the ground station.

5.2.5 Revenue

Revenue refers to the monetary compensation resource providers earn by offering services to users. It is a critical indicator for measuring the commercial value and economic viability of edge computing networks. Using GT models, many researchers have explored the relationship between service pricing and user task offloading decisions. Revenue is typically assessed from several aspects: service sales income, data exchange, and traffic fees, advertising and partnership income, and value-added service revenue. A reasonable pricing strategy can generate more revenue for providers while regulating user demand so that it does not exceed the service capacity. Therefore, in the study of edge computing networks, it is necessary to consider a comprehensive array of factors, including business models, market strategies, and technical implementations (Deng et al. 2021; Teng et al. 2022; Gill 2017; Madni et al. 2017; Coutinho et al. 2015).

Zhang (2022) expressed the price of an unit computing resource of vehicle$i$ as

$$z_i = \ln\left(1 + \frac{\sum_{j=1}^{J_i}\left(1-\beta_{i,j,k}\right)\eta/T_{i,j}^{max}\left(f_i - C_{i,j}\right)}{\sum_{j=1}^{J_i}\left(1-\beta_{i,j,k}\right)}\right). \tag{70}$$

Where, η is a constant coefficient, f_i is CPU cycles per second, J_i is types of tasks, $C_{i,j}$ is the amount of computing resources required to execute this task, $T_{i,j}^{max}$ is the maximum delay that task can tolerant, $\beta_{i,j,k} \in \{0,1\}$ represents whether the task offloads its task to vehicular.

Tang (2022) expressed the revenue gained from edge computing nodes helping cloud computing centers to perform tasks as $\rho_n^r r_n(\tau)$, ρ_n^r is weight factor, $r_n(\tau)$ is defined as the delay reduction compared to the cloud center processing. Which is presented as

$$r_n(\tau) = \sum_{f \in \mathcal{F}_n} \sum_{m \in \mathcal{M}} \left[\left(D_{m,f}^{\text{tcc}}(\tau) + D_{m,f}^{\text{ecc}}(\tau) \right) - \left(D_{m,f}^{\text{tsec},n}(\tau) + D_{m,f}^{\text{blog},n}(\tau) + D_{m,f}^{\text{wait},n}(\tau) + D_{m,f}^{\text{esec},n}(\tau) \right) \right]. \quad (71)$$

Where, $D_{m,f}^{\text{tcc}}(\tau)$ is the latency of the user device to transfer the task to the cloud computing center, $D_{m,f}^{\text{ecc}}(\tau)$ is the time for the cloud computing center to execute the task, $D_{m,f}^{\text{tsec},n}(\tau)$ is the latency of the user device to transfer the task to the edge computing node, $D_{m,f}^{\text{blog},n}(\tau)$ is the queue latency, $D_{m,f}^{\text{wait},n}(\tau)$ is average waiting time among all newly arrived computation tasks until computation task f is processed, $D_{m,f}^{\text{esec},n}(\tau)$ is the task execution time.

Zhang and Zeng (2023) expressed the cost paid by vehicles for task offloading as $U_m = u_\nu + u_c$, u_ν and u_c are the costs for task offloading to the edge server and the cloud, respectively. According to the Weber–Fechner law, the costs of offloading task to the edge server and cloud can be expressed as $u_\nu = \omega_{1,i} P_i S_{1,j}$, $u_c = \omega_{2,i} P_i S_{2,j}$. Where $\omega_{1,i}$ and $\omega_{2,i}$ represent the task data proportions offloaded from vehicle to the edge server and the cloud, respectively. P represents the task data volume, S represents the unit service price.

5.2.6 Service cost

Service cost refers to the total cost incurred by an edge computing network to provide a service to users, including infrastructure, energy, operational, and security investment costs. It is an essential metric for assessing the economic viability and resource efficiency of edge computing networks. It is noteworthy that beyond discussing how much revenue is generated per unit cost, it is essential to consider the long-term benefits that these costs may yield. Evaluating the increased service cost in the context of specific business models and operational strategies is crucial to determine how much competitive advantage these costs have garnered for service providers (Lan 2022; Wang 2021; Xiao et al. 2022; Hui 2022; Li 2023; Zhao 2023; Fan 2024).

Lan (2022) defines a new cost function that takes into account both system latency and resource currency costs $K_{e,A}^i = \lambda_t * D_{e,A}^i + \lambda_e * C_{e,A}^i$. Where, λ_t represents the weight of delay, λ_e represents the weight of resource cost. $\lambda_t + \lambda_e = 1$, and $\lambda_t, \lambda_e \in [0,1]$. D is the transmission delay, C is the total cost of performing the task. After computation, the edge node A with minimum $K_{e,A}^i$ will run task i.

Pang et al. (2022) defines the cost function as $C(t) = \omega E(t) + (1-\omega) D(t)$, where $\omega \in [0,1]$ is used to weigh energy consumption and computational latency. The formula for energy consumption is

$$E(t) = E_l(t) + E_s(t) = \kappa f_l^2 v_l(t) \gamma + \frac{p v_s(t)}{B \log_2\left(1 + \frac{g(t) p}{N_0 B}\right)}. \quad (72)$$

Where $E_l(t)$ is the task execution energy consumption and $E_s(t)$ is the energy consumption for the user to offload the task to the edge server. In $E_l(t)$, κ is the energy consumed per CPU cycle, f_l is the CPU frequency, $v_l(t)$ is the task volume, and γ is the computational intensity (in CPU cycles per bit). In $E_s(t)$, p is the transmit power of the edge server, $v_s(t)$ is the task volume, B is the bandwidth (in HZ), and N_0 is the noise power. $g(t) = d(t)^{-\theta}$ is the channel power, where $d(t)$ is the distance from the mobile device to the edge server, and θ is the path loss parameter.

Considering that user mobility is unavoidable in the MEC paradigm, Gao et al. (2022) denotes The total switching cost of all users between a time slot as

$$D^s(\overrightarrow{x}(t), \overrightarrow{x}(t-1)) = \sum_{k\in\mathcal{N}} \sum_{i\in\mathcal{M}} \sigma\left[x_{ik}(t) - x_{ik}(t-1)\right]^+. \tag{73}$$

Where, $[x_{ik}(t) - x_{ik}(t-1)]^+ = max\{x_{ik}(t) - x_{ik}(t-1), 0\}$, $x_{ik}(t)$ represents whether the service is placed on edge cloud or not.

5.2.7 Security

Security refers to the capability to protect user privacy and system safety, a key metric in the era of big data for building edge computing networks and a prerequisite for users to utilize services. The security of edge computing networks is a comprehensive issue that requires consideration of network architectural design, security policy formulation, technological implementation, and continuous monitoring. By adopting holistic protection mechanisms and strategies, it is crucial to ensure the confidentiality, integrity, and availability of systems and data (Pang et al. 2022; Aslanpour 2020).

Gao et al. (2023) point out that an attacker with a priori knowledge attacks the user's location based on the user offloading event$O_u(t,s)$ and obtains the user location $\widehat{g}$. The result of the attack is defined as follows: $h(\widehat{g} \mid s) = \Pr\{d(t) =< t, \widehat{g} > \mid O_u(t,s)\}$, where $d(t)$ is an inference event. The function d_p that represents the correctness between the location inferred by the opponent and the user's actual location is defined as follows:

$$d_p(g,\widehat{g}) = \begin{cases} 1, if \widehat{g} = g \\ 0, otherwise \end{cases} \tag{74}$$

$\pi(s \mid g) = Pr\{O_u(t,s) \mid \mathrm{r(t)} = (\mathrm{t,g})\}$ is the probability of offloading event $O_u(t,s)$ occurring, given that service request event $\mathrm{r(t)}$ has already occurred. Thus, the user privacy loss ρ can be expressed as

$$\rho_i(\psi, \pi, h, d_p) = \sum_{\widehat{g}} \psi(g)\pi(s \mid g) h(\widehat{g} \mid s) d_p(\widehat{g}, g). \tag{75}$$

Where $\psi(g)$ is the prior knowledge that represents the probability that user will offload the task to server s at location g.

Pang et al. (2022) assumes that the privacy protection level of the offloading ratio is measured by pattern chaos (PC), which can be defined as the difference between the user's original offloading ratio$\alpha(t)$ and the perturbed offloading ratio $\bar{\alpha}(t)$. That is:

$PC\,(t) = |\alpha\,(t) - \bar{\alpha}\ (t)|$. Note that a larger pattern chaos leads to stronger privacy guarantee but more overhead for the user. This paper defines privacy leakage as the probability that the perturbed offloading ratio equals the original offloading ratio plus or minus $\frac{1}{T}\chi$. Privacy Leakage is presented as $\text{Privacy Leakage} = \frac{1}{T}\sum_{t=1}^{T} Pr\,(\alpha\,(t) = \bar{\alpha}\ (t) \pm \frac{1}{T}\chi\,)$, where T is the number of cycles, Pr is the probability function, χ is the privacy preservation requirement.

5.2.8 Reliability

Reliability in the context of edge computing networks refers to the ability to consistently, accurately, and predictably provide services in the face of various faults, errors, and attacks. A network is considered reliable if it can offer consistent services during unforeseen events; otherwise, it is deemed unreliable. Reliability is generally calculated using the following formula: Reliability = MTBF / (MTBF + MTTR), where the Mean Time Between Failures (MTBF) is the ratio of total operational time to the number of failures, calculated as MTBF = Total operational time / Number of failures, and the Mean Time to Repair (MTTR) is the ratio of total downtime to the number of failures: MTTR = Total downtime / Number of failures. This metric is particularly effective in assessing websites, storage and backup services, and mobile computing services. Fault Tolerance is generally calculated using the following formula: Fault Tolerance = Number of Faults Detected / Total Number of Faults. Enhancing the reliability of edge computing networks can improve the system's stability and availability, reducing the impact of failures on business operations and making it a crucial consideration in the design and management of edge computing networks. By designing and implementing effective fault-tolerance mechanisms, it is possible to increase the fault-tolerance of the system and enhance its robustness and recoverability in the face of failures (Gill 2017; Ferrer et al. 2019).

In order to evaluate the effectiveness of task containerizing, Zhang et al. (2021) proposes to use the number of container launches (denoted as Ω_n) to measure the operational stability of the application, resource utilization and container management efficiency. The lower the number of container startups, the more stable the operation of the application is indicated. The numbers of times to start containers are evaluated by the normalized numbers of times to start containers, denoted as the container start times (CST). CST is calculated as the number of times to start containers divided by the number of tasks (equals the maximum number of times to start containers).

Liu (2022) introduce the impact of container or VM failures on the reliability of MEC. The probability of $N_j\,(t_{iqj}) = k$ during the time interval $(0, t_{iqj}]$ can be expressed as

$$Pr\,\{N_j(\,t_{iqj}) = k\} = \frac{(\lambda_j t_{iqj})^k}{k!} e^{-\lambda_j t_{iqj}}, k \geq 0. \tag{76}$$

The mean of $N_j\,(t_{iqj})$ can be computed in the following: $E\,[N_j(\,t_{iqj}\,)] = \lambda_j t_{iqj}$. λ_j is the failure rate of C or VM j , t_{iqj} is the average total processing time of the $q-th$ task of loT application i on the $j-th$ container or virtual machine.

5.2.9 Load balance

Load balance is a resource management technique designed to distribute network resource loads reasonably, aiming to enhance the system's performance, reliability, and scalability. By implementing load balancing technology, requests, data, or tasks can be dynamically distributed across different edge nodes. This approach helps avoid resource overload or imbalance, thereby improving the system's performance and reliability. Consequently, it optimizes the overall network performance and resource utilization, meeting the growing demand for high-quality services (Gao et al. 2022; Fan 2024).

Xiao (2022) models the load balancing problem in a complex environment as a coalitional game $\Psi = (N, \nu)$. Players $N = \{1,2\ldots\}$, and edge servers are modeled as players. Characteristic value ν that specifies the value created by different subsets of the players. This paper assumes that maximizing the payoff $\nu\ (c)$ means maximizing the load fairness.

Yang et al. (2020) presents the degree of load balancing among all drones as

$$S_O(\Omega) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\Omega_i - \bar{\Omega})^2}, \tag{77}$$

where $\bar{\Omega} = \sum_{i=1}^{N}(\Omega_i)/\mathrm{N}$. Note that $S_O(\Omega) \geq 0$. Hence, the larger $S_O(\Omega)$ is, the more nonuniform the distribution of tasks among UAVs is.

Duan (2022) defined load as the number of messages a node processes in each period of time. Cluster load is defined in terms of the average number of messages in the cluster for a node to process: $Load_A = \sum_{n\in A} load_n/Num_A$. Where Num_A is the number of nodes in cluster A, and n is a node index.

5.2.10 UAV path planning

UAV path planning refers to the process of determining the flight path for a drone to achieve the best route selection for specific tasks or objectives, enabling the drone to complete tasks efficiently. Reasonable path planning can help optimize drone flight efficiency by selecting the optimal path based on task requirements and environmental conditions, thereby improving the efficiency and reliability of task execution (Miao 2023).

Cong et al. (2022) points out that drone path planning can be viewed as the classic traveling salesman problem (TSP). This paper uses a $1 \times 2n$ dimensional vector $\mathbf{P} = (p_1, p_2, \ldots, p_{2n})$ to describe the edge server moving path. While the flight distance between task i and task j is then expressed in terms of the Euclidean distance: $dis(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

Miao (2023) points out that the actual distance traveled by a drone cannot simply be calculated as Euclidean distance, but rather as geodesic distance. Therefore, the paper expresses the distance flown by the UAV as (67), where R is the radius of the Earth, H is the flight height of UAV, ED is the Euclidean distance, r is the radius of the mission area. The reason why the Euclidean distance between the UAV and the center of the mission area is derived and then also subtracted from r, i.e., only the distance flown by the UAV to the edge of the mission area is calculated, is because this more accurately reflects the actual needs of the UAV in carrying out its mission. When the UAV makes a return trip, the flight

distance is expressed as $X^2 + Y^2 + H^2$, i.e., the return path is regarded as the hypotenuse of the right triangle formed by the return point and the ground station.

5.2.11 Others

Several studies propose new evaluation metrics for optimizing the cost of edge computing networks and give the corresponding evaluation methods.

He et al. (2022) proposed the concept of AoI to describe the time interval between the generation of a message and its reception, where the smaller the value of AoI, the better. AoI is an important metric for evaluating freshness and latency in network communication systems, and is particularly suitable for application scenarios that require real-time information. By understanding and optimizing AoI, the efficiency and performance of a system can be significantly improved. Suppose the first task in the system is generated at the initial time slot (e.g.,$t = 0$). The instantaneous AoI of WD i at slot t, denoted by $a_i(t) = t - u_i(t)$, is defined as the time elapsed since the earliest unprocessed task is generated. where $u_i(t) = \min\{t_{ij} : t_{ij} \leq t, t'_{ij} \geq t, \forall j \in \mathcal{J}_i\}$ is the generation time of the head-of-line task in WD i. If the task is not completed at slot $t-1$, then $a_i(t)$ grows linearly with respect to t, so we have $a_i(t) = a_i(t-1) + 1$. Otherwise, $a_i(t)$ will jump downward to $t - t_{i,j+1}$ or 0, depending on whether the next task has been generated.

Chen (2023) constructs scheduling tasks as DAG graphs and proposes a scheduling length ratio (SLR) for measuring the quality of scheduling results, where a lower SLR value implies better scheduling performance. SLR is a key performance metric for evaluating the efficiency of scheduling algorithms in terms of resource utilization and task completion time. By calculating SLR, the relative merits of different algorithms can be quantified to guide the optimization and improvement of system scheduling. It is worth mentioning that SLR is a parameter that considers the lower bound of an algorithm, which reflects the improvement of an algorithm better than the response time. Its formula is$SLR = makespan / \sum_{n_i \in CP_{MIN}} min_{p_j \in P}[\omega_{ij}]$. Where $makespan$ is the maximum completion time and the denominator is the minimum computational cost of the critical path tasks. Completion time here is the maximum time required to complete the task scheduling, i.e., the time span from the point in time when the task execution starts to the final point in time when all the tasks are completed, which is similar to the response time we mentioned above.

5.3 Experiment environment

Most researchers conduct their experiments on personal computers with the Windows operating system, utilizing programming languages such as Python or MATLAB. A portion of researchers also use servers based on the Linux operating system for their experiments. In all our cited papers, the highest percentage of using Python language is 74%; MATLAB and Java users are not much, 21% and 5%, respectively. The favorite version of Python used is Python 3.7 and the favorite version of MATLAB used is MATLAB 2018b. Figure 8 shows our simple statistics on the percentage of programming languages used by researchers.

The Python language has an easy-to-understand syntax and a low barrier to entry, which makes it easy for researchers to quickly prototype. The rich third-party libraries in Python provide a powerful boost to edge computing network research. NumPy provides powerful

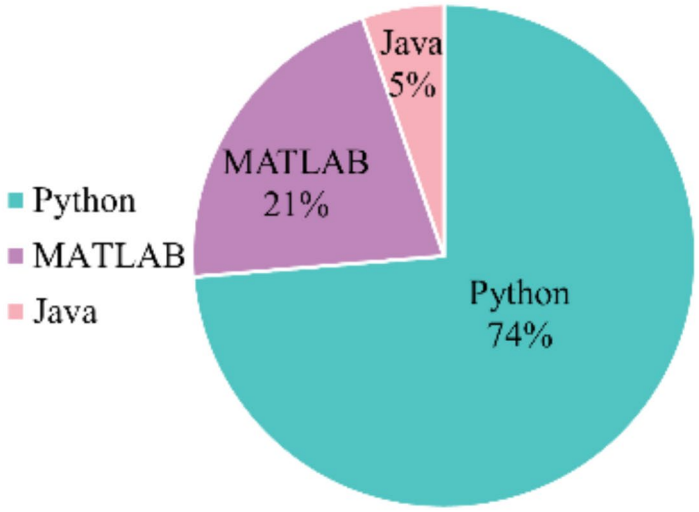


Fig. 8 Programming Language Usage

underlying numerical computation capabilities in scientific computing, and Pandas provides high-level abstraction and rich functionality in data processing and analytics. TensorFlow and PyTorch excel in machine learning and deep learning and can build out network models quickly and easily. Socket, Asyncio, Scapy and Mininet provide powerful network programming, manipulation, simulation and analysis capabilities, enabling researchers to implement complex network programming and simulation. At the same time, Dask, Ray, and Celery provide researchers with the ability to handle large-scale data and computational tasks, making the study of large-scale edge-computing networks a reality. Python 3.7 has been widely used since its release, and its compatibility and stability have been well-proven. Many third-party libraries and frameworks support Python 3.7 so comprehensively that researchers do not need to worry about compatibility issues and can focus on experimentation and research. These features make Python 3.7 ideal for the edge computing research field.

MATLAB provides abundant toolboxes, such as communication toolbox, control system toolbox, etc., researchers can use these tools to quickly build simulation environments for edge computing networks, and test and verify various algorithms and schemes. MATLAB also has a large user community and third-party libraries, researchers can use these resources to quickly access the required functions and algorithms. MATLAB 2018b in the underlying optimization and parallel computation has made great progress, for the computationally intensive tasks have good performance. MATLAB 2018b is a mature version that has been extensively tested and optimized for stability and reliability. It is the excellent overall performance of this version that makes it the preferred version for researchers.

6 Case study

In this section, we introduce several typical application scenarios involved in the research on cost optimization in edge computing. When we were analyzing references, we recorded the applications involved in each paper. Through statistics, we have summarized several more researched and more common applications, which serve as the typical applications of this survey, including UAV and CAV, etc. The outline of case study is shown in Fig. 9 and is described in detail below.

6.1 UAV

UAVs, especially low-cost quad-rotor aircraft, are experiencing explosive growth and have been widely used in a variety of fields. With the advantages of high mobility and low cost, UAVs can provide more flexible service coverage and can mitigate the high latency plague of mobile edge computing networks by acting as MEC servers.

Zhao (2021) considers offloading partial computation tasks of the industrial vehicles (IVs) to multiple available intelligent devices (IDs) of the industrial mobile edge computing, including UAVs, and the fixed-position MEC servers, to optimize the system cost including execution time, energy consumption, and the ID rental price. This paper formulates the decision-making problem of multi-device service for multi-user computation task offloading as multi-device for multi-user sequential game.

The paper evaluates the performance of the MITA algorithm in offloading computation tasks to multiple industrial devices in terms of optimizing system energy consumption and improving system throughput. The energy consumption of executing computation is expressed as

$$E_n^O = E_n^{\text{loc}} + E_n^{\text{off}} = \left(1 - \sum_{i=1}^{q} x_i\right) B_n * L_{\text{cpu}}^V + x_q D_n L_{\text{send}}^V + \sum_{i=1}^{q-1} x_i B_n L_{\text{cpu}}^{UAV} + \sum_{i=1}^{q-1} x_i D_n L_{\text{send}}^V + + \sum_{i=1}^{q-1} x_i J_n L_{\text{send}}^{UAV}. \tag{78}$$

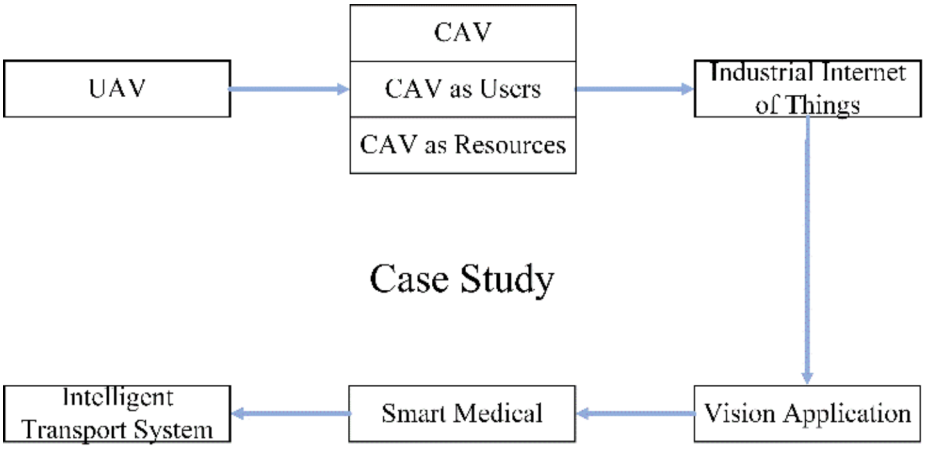


Fig. 9 Outline of case study

Where, E_n^{loc} is the energy consumption of the local part, E_n^{off} is the offloading part, which represents the sum of MEC servers' energy consumption of the computation task.

The multi-device for multiuser sequential game is defined by $G(N, K, U)$, the vehicle players is defined by $N = \{1,2,3,\ldots,n,\ldots,e\}$, the strategy set of vehicle players is defined by $K_e = \{k_1, k_2, \ldots, k_n, \ldots, k_e\}$, the utility function of the player nk_n can be given as $U_n^{k_n} = (1 - k_n) * C_n^L + k_n * C_n^O$, where C_n^L is total cost of the IV_n executing computation task, C_n^O is the cost of renting computing resources from other IDs. For a strategy profile $K_e^* = (k_1^*, k_2^*, \ldots, k_n^*, \ldots, k_e^*)$, if

$$U_n\left(k_n^*, k_{-n}^*\right) \leq U_n\left(k_n, k_{-n}^*\right), \forall\, k_n \in K_e, \tag{79}$$

then the strategy profile K_e^* is the nash equilibrium of the game G. We need to solve the following problems to get the optimal strategy k_n^*:

$$\mu_n(k_n) = arg \min_{\{T_n, E_n, P_n\}} U_n^{k_n} = (1 - k_n) * C_n^L + k_n * C_n^O. \tag{80}$$

It can be easily proved that the formula is convex. Hence, there is an optimal strategy, and NE exists in the multi-device service for multi-user sequence games. Then, the optimal strategy k_n^* of IV_n can be given as

$$k_n^* = \mu_n(k_n) = \begin{cases} arg \min_{\{T_n, E_n, P_n\}} C_n^L, \text{ if } k_n = 0 \\ arg \min_{\{T_n, E_n, P_n\}}, \text{ if } k_n = 1 \end{cases} \tag{81}$$

A minimum incremental task allocation (MITA) algorithm is proposed to divide the whole task and assign the divided units for the minimum cost increment each time. Algorithm 1 is performed to calculate the utility increment of the unit task, and the MITA algorithm is performed to solve the problem of minimum system cost.

This paper used Java to build a multi-device assisted multivehicle computation offloading platform, which has been made open source. Moreover, the paper adopts a 2000×2000 m topology and uses the traffic simulation software SUMO to simulate the trajectory of multiple vehicles. The CPU computing power of the MEC server is considered ten times more powerful than that of a UAV and 100 times more powerful than IVs. This article also considers that the energy consumed by vehicles and UAVs to transmit unit-size data through wireless access is 40 times more than the local computation energy cycle of the same data unit on the vehicle and UAV.

In this scenario, the data size of computation tasks generated by IVs is 7800 KB, and the number of CPU cycles required to execute the task is 18,570 Megacycles, the number of IVs in the system is $N = 100$, and the IDs that can provide computing services for these users include $M = 9\text{UAVs and } 1\text{ MEC server}$. The contrasting scenes are Local computing (LC), No UAV (NU), and Full offload (FO). Results show that the average cost of the proposed MITA is 39.7%, 10.6%, and 13.4% lower than that of LC, NU, and FO, respectively, while the throughput of LC, NU, and FO is 71.0%, 43.2%, and 45.6% lower than MITA. With the increase in task data size, the system average cost of our proposed MITA

decreased from 13.8 to 37.1% compared with other scenarios, while the throughput of other scenarios decreased from 27.9 to 67.7% compared with MITA.

6.2 CAV

Technological advancements in vehicular intelligence have enabled connected smart vehicles to be pervasive edge computing platforms for a plethora of powerful applications. However, since vehicles also have limited local computing resources, it is sometimes necessary to offload some latency-sensitive and computationally intensive tasks to the edge computing network.

6.2.1 CAVs as users

Zhao (2023) proposed an intelligent partial offloading scheme, namely, Digital Twin-Assisted Intelligent Partial Offloading (IGNITE). First, to find the optimal offloading space in advance, we combine the improved clustering algorithm with the Digital Twin technique, in which unreasonable decisions can be avoided by reducing the size of the decision space. Second, to reduce the overall cost of the system, the DRL algorithm is employed to train the offloading strategy, allowing for automatic optimization of computational delay and vehicle service price.

The task of TV_n is denoted as $R_n = \{D_n, C_n, T_n\}$ .When the vehicles on the road perform tasks locally, the local computation delay is represented as $t_n^{loc} = C_n / f_n$, where f_n is the computational power of TV_n (the number of CPU cycles per second of TV_n). When the task of TV_n is offloaded out, the total delay during partial offloading is

$$t_n^{off} = max\left\{(t_n^{up} + t_n^{comp}), t_n^{loc}\right\}. \tag{82}$$

Upload delay, and the time delay are denoted by $t_n^{up} = (\rho_{\,n} \times D_n)/r_n$ and $t_n^{loc} = (1 - \rho_{\,n})\, C_n / f_n$, compute delay is expressed as

$$t_n^{comp} = \begin{cases} t_{n,m}^{comp1} = \frac{\rho_{\,n} C_n}{f'_m}, \text{ If offloading to } SV_m \\ t_n^{comp2} = \frac{\rho_{\,n} C_n}{f'_{rsu}}, \text{ If offloading to RSUs} \end{cases} \tag{83}$$

Note that r_n denotes the data transmission rate according to Shannon's theorem, $\rho_{\,n}$ is the task offload ratio, f' represents the computational resources provided, δ is the ratio of the size of the output data to the size of the input data. when vehicle decides to offload the task, the service price to be paid to the resource is expressed as: $P_n = \rho_{\,n} p_n$. Where the service price is p_n.

This paper represents the state of the system as $S = \{S_n, S_m\}$, where S_n denotes the task information of T_n, including size, computing resources required, size of computing power. The action space is defined as $A = \{\rho_{\,n}, p_n, x_m^n\}$, where x_m^n represents the offload selection. The reward function can be expressed as $R(S, A) = -\sum_{n=1}^{N} \left(t_n^{off} + \rho_{\,n} p_n\right)$. The total cost of all vehicles in the aggregation group can be denoted by Ω, and Ω can be defined as $\Omega = \sum_{n=1}^{N} \left(\alpha\, t_n^{off} + \beta\, \eta\, P_n\right)$. Where $\alpha, \beta \in [0,1]$ denotes the relative

weights accounted for by delay and service price, which can be adjusted based on the actual situation, and η is a regulator. Our goal is to minimize Ω, which can be defined as

$$min\ \Omega = \sum_{n=1}^{N} \left(\alpha\, t_n^{off} + \beta\, \eta\, P_n\right) \tag{84a}$$

$$s.t.\ N \cup M = K, \tag{84b}$$

$$t_n^{off} \leq T_n, \forall\, n \in N, \tag{84c}$$

$$\rho_n \in [0,1], \forall\, n \in N, \tag{84d}$$

$$0 \leq (f'_{rsu} | \rho_n \neq 0) \leq f_{rsu}, \forall\, n \in N, \tag{84e}$$

$$0 \leq (f'_m | \rho_n \neq 0) \leq f_m, \forall\, n \in N, \forall\, m \in M, \tag{84f}$$

$$\sum_{n=1}^{i} (f'_{rsu} \mid \rho_n \neq 0) \leq f_{rsu}, \forall\, n \in N. \tag{84g}$$

The time complexity of finding the best solution increases exponentially with the number of TVs. This problem has been shown to be an NP problem. We have thus constructed the MDP model for task offloading.

In the clustering part of the model, we design a gravity model-based algorithm to improve the original k-means algorithm (Gk-means). Based on the MDP model, the article uses a Deep Deterministic Policy Gradient (DDPG)-based DRL algorithm to obtain an optimal solution to the task offloading problem by training a neural network. In addition, the article designed a feedback mechanism to readjust the clustering process parameters based on the offloading results to achieve better clustering results.

This paper generates the simulation platform in Python 3.7, assuming the targeted area consists of two intersections that are 200 m in length and width, and there are 25 vehicles in the area. The value of k in the Gk-means algorithm is defined as 3. Then, after one clustering, there are 15 vehicles within one of the aggregation groups to be studied. As for the channel model, we let the transmission bandwidth of the channel be $W = 10\mathrm{MHz}$, the noise power density be $N_0 = -174\mathrm{dBm/Hz}$, and the channel gain between TV and SV vary from moment to moment between $\left(10^{-8}, 10^{-7}\right)$.In addition, we set the power of the vehicle uploading data to $P = 0.1\mathrm{W}$. In terms of the task model, it is assumed that the data size of the task is $(0.8,1.2)\,\mathrm{MB}$, and the amount of CPU cycles required to execute the task is $(0.1,1.0)\,G$. Regarding the computing power, it is assumed that the CPU frequency of the VEC server is randomly distributed between $2.0G$ and $2.8Gcycles/s$, while the CPU frequency of the vehicle is $0.5Gcycles/s$.

The contrasting scenes are no clustering+DDPG (NC-DDPG), k-means clustering+DDPG (K-DDPG), Gk-means clustering+DQN (GK-DQN) and Gk-means clustering+local computing (GK-Local). As the task size increases, the vehicle computational power increases, the RSU processing power increases, and the number of vehicles increases,

the computational cost of GK-DDPG scheme is always the lowest. In terms of latency, convergence speed, and offload rate, GK-DDPG performs the best.

6.2.2 CAVs as resources

Ma et al. (2021) proposed the idea of parking edge computing, which makes use of parked vehicles to assist edge servers in offloaded task handling. On-street and off-street parked vehicles are organized into parking clusters to act as virtual edge servers, participating in offloaded tasks execution. A local task scheduling policy is proposed, which reasonably allocates parked vehicles to perform the tasks with the aim of improving task offloading performance.

The set of moving vehicles is $N = \{1,2,3,\ldots,N\}$. The set of tasks is $T = \{1,2,3,\ldots,T\}$. The set of all the edge servers in the VEC server layer is $L = \{1,2,3,\ldots,m,m+1,m+2,\ldots,L\}$. An edge server node $k \in L$, in which $k \leq m$ demonstrates it is a physical edge server while $k > m$ demonstrates it is a virtual edge server. Task j generated by vehicle i is presented by a three-tuple $T_{ij} = (s_{ij}, c_{ij}, t_{ij}^{max})$, where s_{ij} denotes the size of the input data of the task, c_{ij} represents the amount of computing resources needed to complete the task, t_{ij}^{max} is the maximum tolerated delay of this task. When computational tasks need to be offloaded to virtual edge servers (i.e., parked vehicles), the task completion time is given by

$$t_{s,ij} = t_{\text{tran},ijs} + \frac{c_{ij}}{r_{s,ij}} + t_{\text{out},ij}, \tag{85}$$

where $t_{\text{tran},ijs}$ is the input task data transmission delay from the source vehicle to the virtual edge server, $r_{s,ij}$ is the number of computing resources assigned by this virtual edge server to perform this task, $t_{\text{out},ij}$ is the time to send back the final result. Depending on whether the source vehicle and the virtual edge server are within the coverage of the RSU, $t_{\text{tran},ijs}$ is accordingly represented in four different ways. If the source vehicle and the virtual edge server are all covered by RSUs, $t_{\text{tran},ijs}$ is given by

$$t_{\text{tran},ijs} = \frac{s_{ij}}{c_{(i,\text{RSU}_i)}} + h_{ss}\left(\frac{d_{\text{RSU}}}{V} + \frac{s_{ij}}{T}\right) + \frac{s_{ij}}{B \times \bar{c}_{(\text{RSU}_s,p)}}. \tag{86}$$

Where B denotes the number of parked vehicles to process task, $c_{(i,\text{RSU}_i)}$ denotes the throughput from vehicle to the RSU that covers this vehicle, $\bar{c}_{(\text{RSU}_s,p)}$ is the average throughput from RSU, which covers the virtual edge server s to all parked vehicle members, h_{ss} is the number of hops that the input data needs to be transmitted among RSUs to reach the server provider, d_{RSU} denotes the average distance between two adjacent RSUs. $c_{(\text{RSU}_s,p)}$ is computed from Shannon's theorem. If the source vehicle is in the radio coverage of RSU, but the virtual edge server is out of the coverage of the RSU, we have

$$t_{\text{tran},ijs} = \frac{s_{ij}}{c_{(i,\text{RSU}_i)}} + h_{ss}\left(\frac{d_{\text{RSU}}}{V} + \frac{s_{ij}}{T}\right) + \frac{s_{ij}}{c_{(\text{RSU},q')}} + t_{q',s}. \tag{87}$$

Where $c_{(\text{RSU},q')}$ is the throughput from the RSU nearest to the virtual edge server to the vehicle q', in which vehicle q' denotes the vehicle located nearest to the virtual edge server

among vehicles covered by this RSU, while $t_{q',s}$ denotes the data delivery delay from vehicle q' to the parked vehicles that will perform the task. If the source vehicle is out of the radio coverage of any RSU whereas the virtual edge server is in the coverage of the RSU, we have

$$t_{\text{tran},ijs} = t_{i,q} + \frac{s_{ij}}{c_{(q,\text{RSU}_i)}} + h_{ss} \cdot \left(\frac{d_{\text{RSU}}}{V} + \frac{s_{ij}}{T}\right) + \frac{s_{ij}}{B \times \bar{c}_{(\text{RSU}_s,p)}}. \tag{88}$$

If both the source vehicle and the virtual edge server are all out of the radio coverage of RSUs, we have

$$t_{\text{tran},ijs} = t_{i,q} + \frac{s_{ij}}{c_{(q,\text{RSU}_i)}} + h_{ss} \cdot \left(\frac{d_{\text{RSU}}}{V} + \frac{s_{ij}}{T}\right) + \frac{s_{ij}}{c_{(\text{RSU}_s,q')}} + t_{q',s}. \tag{89}$$

This task scheduling issue is modeling as a mixed integer nonlinear programming problem, the resource allocation problem is modeling as a knapsack problem. Suppose the server choice profile is $Y = \{y_{k,ij} \mid k \in L, i \in N, j \in \mathrm{T}\}$, where $y_{k,ij} = 1$ denotes task T_{ij} is performed by the kth node in L and $y_{k,ij} = 0$ otherwise. The task scheduling can be expressed as the following optimization problem:

$$\min\ f = \sum_{k=1}^{L} \sum_{i=1}^{N} \sum_{j=1}^{T} y_{k,ij} t_{k,ij} \tag{90a}$$

$$s.t.\ y_{k,ij} \in \{0,1\} \forall\, k \in L, i \in N, j \in T, \tag{90b}$$

$$\sum_{k \in L} y_{k,ij} = 1 \forall\, i \in N, j \in T, \tag{90c}$$

$$\sum_{k \in L} y_{k,ij} t_{k,ij} \leq t_{ij}^{max} \forall\, i \in N, j \in T, \tag{90d}$$

$$\sum_{i \in N} \sum_{j \in T} r_{k,ij} \leq R_k \forall\, k \in L, \tag{90e}$$

$$r_{k,ij} \leq r_{\max} \forall\, k \in L, i \in N, j \in T. \tag{90f}$$

The proposed task scheduling problem is NP-hard. The resource allocation problem can be converted to a knapsack problem as follows:

$$\min\ f' = \sum_{i=1}^{N} \sum_{j=1}^{T} \sum_{m=1}^{r_{max}} x_{ij}^{m} \frac{c_{ij}}{r_{ij}^{m}} \tag{91a}$$

$$s.t.\ x_{ij}^{m} \in \{0,1\} \forall\, i \in N, j \in T, m \in r_{\max}, \tag{91b}$$

$$\sum_{i=1}^{N} \sum_{j=1}^{T} \sum_{m=1}^{r_{\max}} r_{k,ij}^{m} x_{ij}^{m} \leq R_k \forall\, k \in L, \tag{91c}$$

$$\sum_{m=1}^{r_{\max}} x_{ij}^{m} = 1 \forall\, i \in\ N, j \in\ T. \tag{91d}$$

Clearly, this problem is a multidimensional multichoice knapsack problem (MMKP), in which x_{ij}^{m} denotes the resource item choice profile of each task resource set.

The parking data was obtained from a six-week survey conducted by the authors of this paper in a 1600×1600 m urban area in Chengdu, China. This paper applys VanetMobiSim-1.1 to generate realistic movement of vehicles. NS-2.33 tool is used for simulation. The communication range of the vehicle is set to $300m$, and the MAC protocol is 2 MHz 802.11. The default number of moving vehicles is 200. The local computation capacity of each vehicle Lv is 0.4 GHz. Moreover, four RSUs are located uniformly in the simulation area, and the transmission range of the RSU is supposed to be $300m$. For wireless transmission between the RSUs and the vehicles, the bandwidth is divided into a total of 20 equal subbands, with each subband that is equal to 10 MHz. The data transmission speed on the wireless backhauls is $3 \times\ 10^{8}\mathrm{m/s}$. Furthermore, the interference is set to zero. α is 0.1, and γ is 10^{-26}. The full energy of the parked vehicle is 3 W and Emax is set to 30% of the vehicle's full energy.

The metrics for evaluation are average completion time, ratio of successful task, and offloading ratio. The contrasting schemes are: Local Computing (LC), MEC Only Scheme (MECO), and Randomly Offloading Scheme (RO). As the number of tasks, the number of moving vehicles, and the required computing resources change, the framework proposed in this article always has an advantage of about 10–20%.

6.3 Industrial Internet of things

Dai (2023) proposes a co-offloading framework, integrating migration cost and offloading willingness, in device-to-device (D2D) assisted MEC networks. Then, a learning-based task co-offloading algorithm is investigated, with the goal of minimal system cost.

The system cost of task n conducted by edge node m processing at time slot t is expressed as

$$\Xi_{m,n}^{t} = \alpha\, T_{n}^{t} + (1-\alpha)\, G_{n}^{t}, m \in\ M. \tag{92}$$

Where, T_{n}^{t} is the total delay, G_{n}^{t} is the migration cost. The weighting factor $\alpha\ \in\ [0,1]$ is introduced to balance task delay and service migration cost. The co-offloading problem is formulated as follows:

$$\min_{\mathbf{x}_{n}^{t}}\ \frac{1}{T}\sum_{t=1}^{T}\sum_{n\in\mathcal{N}}\Xi_{m,n}^{t}, m \in\ M \tag{93a}$$

$$s.t.\ x_{m}^{t,n} \in\ \{0,1\}, \tag{93b}$$

$$\sum_{m=1}^{M} x_{m}^{t,n} = 1, \tag{93c}$$

$$f_{m,n}^{t} \leq\ F^{m}. \tag{93d}$$

This paper proposes a learning-based task co-offloading algorithm based on MAB theory, with the goal of minimal task delay and service migration cost. The algorithm consists of two stages: service devices choosing and co-offloading decision making.

The contrasting methods are: task co-offloading with global information (GI), task offloading in MEC net-works (MEC), task offloading in D2D networks (D2D). Compared with MEC and D2D, our proposed co-offloading algorithm shows superiority in system cost.

6.4 Vision application

Lan (2022) presents a system framework, EDGEVISION, for computer vision applications partitioning and orchestration on heterogeneous edge computing platforms considering both CPUs and GPUs. This paper also proposes two scheduling algorithms in the framework, minimum latency task scheduling (MLTS) and minimum cost task scheduling (MCTS), aiming to minimize the processing latency and the overall system cost.

Based on the QoS requirement, we can formulate the optimization problem as a multi-objective constrained optimization problem:

$$min\ C_A = \sum_{e=1}^{K} C^e_{\text{usage}} \tag{94a}$$

$$min\ D_A = \sum_{i=1}^{N} D^i_{\text{compu}} + \sum_{i=1}^{N} D^i_{\text{commu}} \tag{94b}$$

$$s.t.\ G_{task}, G_{com}\ are\ satisfied. \tag{94c}$$

where C_A is the total cost for processing the application A calculated as the sum of the usage cost for each edge node C^e_{usage}, consisting of CPU/GPU, RAM, and network resources usage. D_A is the total delay for processing the application A consisting of two parts, computing latency D^i_{compu} and communication latency D^i_{commu} for all the partitioned tasks.

This paper implements the framework on a real-world testbed, which consists of ten heterogeneous edge devices. A workstation that is equipped with 3.5-GHz Dual-Core Intel Core i7 CPU and 16-GB RAM running Ubuntu 18.04 as the local server. The local server acts as the master node and runs the scheduling algorithms. This heterogeneous edge computing platform contains four Jetson AGX Xavier and one Jetson Nano, from Nvidia (commonly used in autonomous driving, video processing, and portable computation applications), three Raspberry Pi3 Model B boards, and two personal computers. All edge devices and the local server are connected via a local router.

The contrasting task schedulers are: LRP scheduler, BRA scheduler, and SSP scheduler. The program used for test runs is 3-D SLAM.

6.5 Smart medical

Awad Abdellatif (2021) proposes medical-edge-blockchain (MEdge-Chain), a holistic framework that exploits the integration of edge computing and blockchain-based technologies to process large amounts of medical data. Furthermore, a blockchain-based optimiza-

tion model is developed that aims to optimize the latency and computational cost of medical data exchange between different health entities, hence providing effective and secure healthcare services.

The model focuses on three main metrics: latency (L), security (η), and cost (C). However, these metrics have different values and units, which must be first normalized with respect to their maximum values (denoted by l_m, η_m, and c_m, respectively) to make them comparable. Then, an aggregate utility U is defined

$$U = \alpha \cdot \frac{L}{l_m} + \beta \cdot \frac{\eta_m}{\eta} + \gamma \cdot \frac{C}{c_m}. \tag{95}$$

where α, β, and γ are weighting parameters representing the relative importance of the considered metrics, such that $\alpha + \beta + \gamma = 1$. Accordingly, the optimization problem can be expressed as:

$$\min_{m,n} \quad (U) \tag{96a}$$

$$s.t.\ c_i \geq \rho_i \cdot x_i \forall\, i \in \{1, \ldots, m\}, \tag{96b}$$

$$v \leq m \leq M, \tag{96c}$$

$$t \leq n \leq \chi, \tag{96d}$$

Note that m is the number of selected validators, n is the number of transactions per block.

MEdge-Chain system architecture is comprised of diverse e-health entities and is divided into two main networks: a local network and a blockchain network. Considering the block verification optimization, an efficient BCO algorithm is proposed. The BCO algorithm leverages the idea of problem decomposition to find the optimal solution of the optimization problem in practical scenarios.

Data were collected at the patient recovery center of Hamad Medical Corporation. The simulation parameters are set as follows: $q = 4, O = 5Mb, \theta = 1, r_d = 1.2Mb/s, r_u = 1.3Mb/s, K = 100, b = 0.5kb$. Our proposed BCO algorithm outperforms Exhaustive search in terms of combined cost. MLTS has the best performance when only CPU is used, MCTS has some advantage, but the cost of MCTS is far less than other schemes. When both CPU and GPU are used, MLTS still leads in performance, but in MCTS is not as effective as LPR in reducing latency. in terms of cost, MCTS is far better than other schemes. Actually, an algorithm combining MLTS and MCTS (MLTS/MCTS) to provide better scheduling performance for the system is designed. It can be found from the experiments that MLTS/MCTS performs less well than LRP in terms of delay at frame rate below 40 and outperforms the three compared algorithms in terms of delay at frame rate above 40. In all cases, this hybrid algorithm outperforms the other comparison algorithms in terms of cost.

6.6 Intelligent transport system

Gao et al. (2023) proposes DRL method PPO2 to solve the planning problem of task offloading with good generalization and convergence speed, where we focus on the location privacy requirement. The method can handle large-scale task offloading and obtain offloading policies with reduced privacy loss, energy consumption and time delays.

The time delay and energy consumption in the processing of the local computing are expressed as $t_n^L = c_i/f_n$ and $e_n^L = p_n t_n^L$. c_i is the total number of CPU cycles required to complete the computing task, f_n is CPU frequency, and p_n is the CPU power. The energy consumption and time delay of the offloading computing are expressed as $t_{up}^o + t_{ex}^O = d_i/tr_n + d_i/p_s$ and $e_n^O = p_n\left(t_{up}^O + t_{dl}^O\right)$. Here, we assume that the energy consumption of mobile device n while waiting for the server to compute tasks can be ignored. t_{up}^o is upload time and download time of the data, t_{ex}^O is the processing time for edge server. d_i is the size of the input data, tr_n is the data transmission rate, p_s is the transmission power of edge server. An attacker with a priori knowledge attacks the user's location based on the user offloading event $O_u(t,s)$ and obtains the user location $\widehat{g}$. The result of the attack is defined as follows

$$h\left(\widehat{g} \mid s\right) = Pr\left\{d\left(t\right) =< t, \widehat{g} > \mid O_u\left(t,s\right)\right\}. \tag{97}$$

The offloading policy is expressed as

$$\pi\ (s|g) = Pr\left\{O_u(t,s)\,|\mathrm{r}(\mathrm{t}) = (\mathrm{t},\mathrm{g})\right\}, \tag{98}$$

here $\mathrm{r}\left(\mathrm{t}\right)$ represents a service request event. The user privacy loss ρ is expressed as

$$\rho_{\,i}\left(\psi\,,\pi\,,h,d_p\right) = \sum_{\widehat{g}}\ \psi\ (g)\,\pi\ (s \mid g)\,h\left(\widehat{g} \mid s\right)d_p\left(\widehat{g},g\right). \tag{99}$$

Specifically, $\psi\ (g)$ is the prior knowledge that represents the probability that user will offload the task to server, $\pi\ (s \mid g)$ refers to the probability that user will offload the service to edge server s, the probability of attack result can be obtained according to $h\left(\widehat{g} \mid s\right)$, $d_p\left(\widehat{g},g\right)$ determines if the attacking result is correct.

The offloading MDP model is formulated as:

$$min\ \ \Omega\ = \sum_{n=1}^{N}\left(\alpha\, t_n + \beta\, e_n + \gamma\, \rho_{\,n} + \delta\, \varphi_{\,n}\right) \tag{100a}$$

$$s.t.\ t_n \leq\ T, e_n \leq\ E, \rho_{\,n} \in\ [0{,}1]\,, \varphi_{\,n} = \delta\ \phi\,, \forall\, n \in\ N, \tag{100b}$$

$$\sum_{n=1}^{N} t_n \leq\ t_{sum}, \forall\, n \in\ N, \tag{100c}$$

$$\sum_{n=1}^{N} e_n \leq\ e_{sum}, \forall\, n \in\ N. \tag{100d}$$

Note that $\varphi_n = \delta\phi$ is the penalty calculation formula, penalty δ is only valid when the task is dropped $(\varphi = 1)$.

The method is tested on the Reality Mining dataset, which collected by the MIT Media Laboratory. The dataset consists of location information and provides two features to represent a location: cell tower ID (cellID) and area ID (areaID). However, in this work, only the location indexed by areaID is considered. In the convergence evaluation experiment, the contrasting methods are Q-learning-based offloading, SARSA-based offloading, and PPO-based offloading. In the performance evaluation experiment, the contrasting methods are the local computing method and distance-based offloading.

The experiment shows that: (1) the Q-learning-based and SARSA-based methods perform better in small-scale applications. However, as the number of users increases, only the PPO-based method and PPO2 method converge to the optimal solution, which demonstrates that the models become robust after adding the actor–critic mechanism. Moreover, PPO2 always converges faster than PPO. (2) The local computing method has no privacy consumption, and PPO2 has a much smaller privacy loss than the nearby priority method. (3) The local computing method has maximum energy consumption, and the nearby priority method has minimal energy consumption. The energy consumption of PPO2 was in between and significantly smaller than that of the local computing method. (4) PPO2 performs similarly in terms of time consumption as it does in terms of energy consumption.

7 Open issues

Despite the significant achievements in research on cost reduction within edge computing, numerous key issues still need to be explored. This section discusses several open research challenges. The outline of open issues is shown in Fig. 10 and is described in detail below.

7.1 Data security

For users, utilizing edge computing services requires uploading data to edge nodes, which introduces two major concerns: (1) How can the connected devices be ensured they are trustworthy? (2) How can eavesdropping be prevented during communication? Although some research attempts have been made to address these issues by introducing technolo-

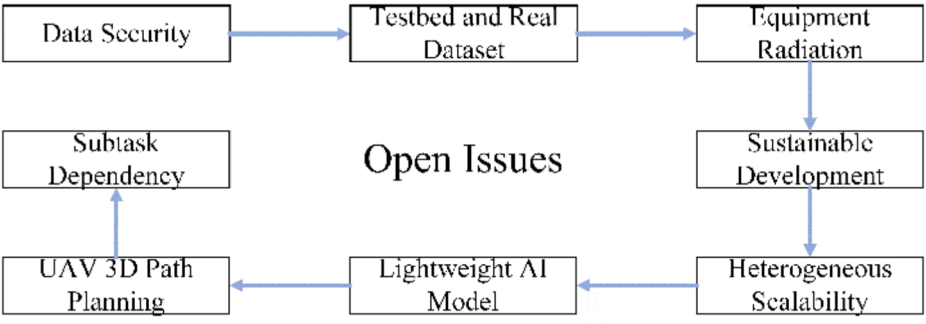


Fig. 10 Outline of open issues

gies like blockchain, they have not yet met the commercialization requirements. Therefore, establishing efficient device identification mechanisms and effective eavesdropping prevention technologies, especially those applicable across different access networks (such as 5G, WIFI, LTE, etc.), is essential. For operators, opening edge computing services to the public necessitates addressing the following issues: (1) How can attackers be prevented from stealing or tampering with sensitive information? (2) How can we defend against malicious attackers controlling the network through business injection attacks? (3) How do we stop access attacks (such as DDOS attacks) from excessively consuming communication resources? Data security is a prerequisite for users to utilize edge computing networks. However, in the absence of legal supervision, it is crucial to design effective anti-attack mechanisms before introducing edge computing networks to the market (Xue 2023; Cruz 2023; Feng 2022; Pang et al. 2022; Nguyen 2021; Gao et al. 2023).

The introduction of blockchain technology and federated learning techniques has been shown to be feasible in some studies, but the resulting increase in resource consumption must also be considered. Traditional firewalls are not able to adapt to highly dynamic edge computing environments, and virtual firewall technologies have been adopted to meet the flexibility requirements in networks that provide services in the form of VMs. It has also been pointed out that an attacker can reconstruct the original data for federated learning by extrapolation, thus proposing the use of homomorphic encryption. Homomorphic encryption allows computations to be performed directly on the encrypted data, and the result inferred by the attacker remains ciphertext, effectively guaranteeing data privacy.

7.2 Tested and real dataset

Real datasets better reflect real-world scenarios and user behaviors, and contain a variety of uncertainties, noise, and complexity. These features help resource scheduling models in edge computing networks achieve better generalization and improve network performance and reliability in real environments. However, due to the challenges of collecting real data, most researchers use simulators to generate data. While the simulator's ability to control the data generation conditions and parameters has some advantages in some cases, the results of training and testing with simulator data may differ from reality. Challenges may be encountered when applying algorithms or models to real-world applications. In addition, most researchers have not used Testbed, which provides realistic network environments and device behaviors and generates realistic training and testing data with high fidelity and reliability. Therefore, extra efforts are needed to develop Testbeds to improve the performance of algorithms or models in real applications. We also call for more researchers to collect real data and open source these real datasets. This will propel the field of edge computing network research to a brighter future (Raeisi-Varzaneh et al. 2023).

When building a Testbed, it is important to select the appropriate equipment configuration based on actual needs, such as: edge devices, network devices and servers. The network topology and the form of connection between edge devices and servers also deserve careful consideration. Using containers or virtual machines to isolate different applications and services can improve system security, consistency, manageability, agility, and resiliency, helping to build a more stable, reliable, and efficient testbed. When collecting real datasets, pay attention to their comprehensiveness and representativeness. In addition, it is necessary to anonymize the data to protect user privacy and avoid disclosing sensitive information.

7.3 Equipment radiation

In modern society, there is growing concern over radiation exposure, especially with the proliferation of wireless communication technologies increasing our exposure to electromagnetic radiation sources. This has heightened interest in the potential health impacts of radiation, particularly for those who spend long periods near radiation sources. Although some studies suggest that exposure to radiofrequency signals from wireless communications is safe, there is a lack of research on the effects of exposure to multiple radiation sources. Additionally, the safety of some wireless energy transfer methods remains uncertain. It is imperative to continue expanding research efforts to prevent potential harm from densely deployed edge devices to users and other equipment, thereby avoiding disputes and liabilities (Wang 2023).

When building an edge computing network, we should choose equipment that complies with international electromagnetic radiation standards and certifications (e.g., FCC, CE certification, etc.). The new generation of wireless communication technologies (e.g., 5G, etc.) are designed with radiation in mind, which is reduced by advanced antenna technology and beamforming. In addition, we can also consider low-power wide area network technologies (e.g., Sigfox, etc.) for communication.

7.4 Sustainable development

The development and application of edge computing networks have direct and indirect impacts on the ecological environment, necessitating consideration of environmental factors to ensure the sustainable development of these networks. This is an action that institutions or companies with a sense of social responsibility must undertake. Edge computing networks require a substantial supply of electrical energy, and power generation using fossil fuels, which is currently the most common method worldwide, imposes a significant burden on the Earth's environment. Hence, it is essential to utilize renewable and clean energy sources (such as solar, wind, and nuclear power) to reduce environmental impacts. Moreover, manufacturing and disposal devices within edge computing networks generate considerable carbon emissions. Controlling the carbon footprint is one effective way to reduce emissions. When purchasing equipment, it is advisable to opt for devices produced in carbon-neutral factories and to implement proper disposal measures to minimize pollution from waste. Optimizing to reduce device energy consumption and enhance resource utilization is also viable, with many researchers achieving results in this direction (Li 2022; Hao 2021; Chen 2022; Tang 2022; Liu et al. 2023).

In response to the energy crisis, the conventional approach is to enhance the energy efficiency of machinery with the objective of curbing the reliance on fossil fuels. Nowadays, some researchers try to use clean energy or introduce energy recovery devices with the aim of reducing the environmental impact. We would like to urge you to consider the concept of carbon footprint, which is the amount of carbon dioxide emissions that result directly or indirectly from a given period of time. The calculation of a carbon footprint covers the emissions of a product or service throughout its entire life cycle, from production, transportation, end use to disposal. This comprehensive assessment methodology enables us to understand and evaluate the impact of human activities on the environment more accurately. We support the use of Life Cycle Assessment (LCA) calculations by researchers as opposed to

the Energy Fossil Fuel Emissions (IPCC) calculations used by the United Nations Climate Change Committee.

7.5 Heterogeneous scalability

Heterogeneous scalability refers to the ability to ensure service availability by adding edge devices, regardless of the number of tasks offloaded by users or the demands of those tasks. Excellent heterogeneous scalability allows an edge computing network to utilize devices with various configurations, reducing equipment costs and meeting sudden increases in task demands through methods like leasing idle devices. In the edge computing environment, tasks submitted by users may involve different programming languages, hardware architectures, and processing units. Moreover, to meet the growing demands of users, the network should support the integration of various edge devices, achieving scalability. Especially under the future context of 6G, the massive amount of data and devices will place higher demands on the scalability of edge computing. However, current research efforts are not yet sufficient to meet these requirements (Alkaabi et al. 2024; Xue 2023; Iftikhar 2023).

Currently, some research is attempting to use standardized communication protocols to enable interoperability between different devices. Container technology can package an application and all its dependencies into a single unit, enabling it to run on different hardware and operating systems. Container orchestration technology also ensures high availability and load balancing of containers in large-scale distributed environments.

7.6 Lightweight AI model

While numerous studies have applied artificial intelligence (AI) models to edge computing, achieving significant progress, the complexity and high dynamism of the edge computing network scheduling process often require large AI models. However, some edge nodes have limited computational resources and struggle to run such large models. Therefore, future research should focus on developing and designing lightweight AI models that can be deployed in edge nodes with limited computational resources and energy. This would free up more resources for providing services. Additionally, using AI models for efficient scheduling to reduce costs can increase the profitability of edge computing networks (Hua et al. 2023).

Compression techniques such as knowledge distillation, quantization, and weight pruning are effective ways to achieve lightweight AI models. However, while model compression can help AI models run at the edge, it is often accompanied by a loss of model accuracy. Meanwhile, static compression techniques cannot adapt to the dynamic hardware configuration and load of edge nodes. Therefore, researchers are looking for model compression techniques that are suitable for complex conditioned edge nodes.

7.7 UAV 3D path planning

Although many studies have focused on drone path optimization, they commonly assume that drones fly at a fixed height, opting for lateral movements when encountering obstacles rather than considering altitude adjustments to bypass obstructions. In some cases, this can lead to wasteful energy consumption by the drones. Therefore, it is essential to further

develop 3D path planning algorithms for drones, allowing for more efficient navigation that includes elevation changes to circumvent obstacles efficiently (Huda 2022; Wang 2023).

The 3D path planning problem for UAVs is an NP-hard problem with huge computational effort and high response speed requirements.3D path planning is studied by first modeling the 3D environment, setting appropriate constraints, and then exploring path planning algorithms that are fast and consume little resources.

7.8 Subtask dependency

Due to the limited computing resources of ESs, larger tasks are often divided into interconnected subtasks that may be offloaded to different ESs and cloud servers. Dependencies may exist among subtasks, which makes minimizing task completion time challenging. Specifically, subtask resource allocation should consider not only the number of allocated resources, but also the starting point of the time gap between the allocated resources so that tasks can be executed coherently. In order to guarantee the concurrent execution of a number of tasks, it is necessary to allocate resources to the subtasks in accordance with the delay constraints of the larger tasks (Raeisi-Varzaneh et al. 2023; Cai et al. 2024; Liu et al. 2023).

The first step in solving the subtask dependency problem is task splitting and prioritization. Using DAG to model the dependencies between subtasks and clarifying the order of precedence is verified to be feasible. In addition, we have to consider both the amount of remaining available resources and the amount of resources that will be released soon.

8 Conclusion

This survey provides a systematic and comprehensive review of research on cost reduction in edge computing. Initially, we briefly introduced the development of edge computing, tracing its evolution from cloud computing. Subsequently, we detailed the three-layer architecture of the edge computing paradigm: thing layer, edge layer, cloud layer, and the four collaborative working paradigms: thing-edge collaboration, edge-edge collaboration, edge-cloud collaboration, and thing-edge-cloud collaboration. We then provide a comprehensive overview of the eight main objectives of cost optimization: energy, server deployment, storage, communication, task execution time, load balancing, pricing strategy, and privacy protection, and their corresponding optimization methods. We then provide an in-depth demonstration of the mainstream mathematical models used to construct optimization problems for edge computing networks: MDP, GT, INLP, Lyapunov Optimization, Convex Optimization, Max-Min Optimization and QCQP. In addition, we summarize the training and testing used for the Testbeds, Emulators, Datasets, and commonly used evaluation metrics. Then, we delve into typical cases in seven typical application scenarios of edge computing. Finally, we discuss eight open questions and provide research recommendations. We expect further contributions from researchers in these directions in the future.

Acknowledgements This work is supported by the National Natural Science Foundation of China's top-level program under grant No.52275480 and the Guizhou Provincial Department of Science and Technology Project under grant No. QKHZYD [2023]002.

Author contributions S.L and T.H made a significant contribution to the conception or design of the work; L.C and X.Z wrote the main manuscript text; Y.C and G.L and F.W and Y.L made significant contributions to the acquisition, analysis or interpretation of data about the work; Y.Z and Q.X made significant contributions to picture and table production.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Abreha HG, Hayajneh M, Serhani MA (2022) Federated learning in edge computing: a systematic survey. Sensors 22(2):450. https://doi.org/10.3390/s22020450

Adjih C et al (2015) FIT IoT-LAB: A large scale open experimental IoT testbed. In: IEEE 2nd World Forum on Internet of Things (WF-IoT). IEEE, Milan, Italy. pp 459–464. https://doi.org/10.1109/WF-IoT.2015.7389098

Alkaabi SR, Gregory MA, Li S (2024) Multi-access edge computing handover strategies, management, and challenges: a review. IEEE Access 12:4660–4673. https://doi.org/10.1109/ACCESS.2024.3349587

Aslanpour MS, Gill SS, Toosi AN (2020) Performance evaluation metrics for cloud, fog and edge computing: a review, taxonomy, benchmarks and standards for future research. Internet Things 12:100273. https://doi.org/10.1016/j.iot.2020.100273

Awad Abdellatif A et al (2021) MEdge-chain: leveraging edge computing and blockchain for efficient medical data exchange. IEEE Internet Things J 8(21):15762–15775. https://doi.org/10.1109/JIOT.2021.3052910

Balas E, Mazzola JB (1984) Nonlinear 0–1 programming: I. Linearization techniques, mathematical programming 30(1):1–21. https://doi.org/10.1007/BF02591796

Burer S, Letchford AN (2012) Non-convex mixed-integer nonlinear programming: a survey. Surv Oper Res Manag Sci 17(2):97–106. https://doi.org/10.1016/j.sorms.2012.08.001

Cai Q, Zhou Y, Liu L, Qi Y, Shi J (2024) Prioritized assignment with task dependency in collaborative mobile edge computing. IEEE Trans Mob Comput 1–17. https://doi.org/10.1109/TMC.2024.3427380

Carpio F, Michalke M, Jukan A (2023) BenchFaaS: benchmarking serverless functions in an edge computing network testbed. IEEE Netw 37(5):81–88. https://doi.org/10.1109/MNET.125.2200294

Chai F, Zhang Q, Yao H, Xin X, Gao R, Guizani M (2023) Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT. IEEE Trans Veh Technol 72(6):7783–7795. https://doi.org/10.1109/TVT.2023.3238771

Chen Y, Zhang N, Zhang Y, Chen X (2019) Dynamic computation offloading in edge computing for Internet of Things. IEEE Internet Things J 6(3):4242–4251. https://doi.org/10.1109/JIOT.2018.2875715

Chen Y, Zhao F, Chen X, Wu Y (2022) Efficient multi-vehicle task offloading for mobile edge computing in 6G networks. IEEE Trans Veh Technol 71(5):4584–4595. https://doi.org/10.1109/TVT.2021.3133586

Chen Y, Liu S, Chen Y, Ling X (2023) A scheduling algorithm for heterogeneous computing systems by edge cover queue. Knowl Based Syst 265:110369. https://doi.org/10.1016/j.knosys.2023.110369

Cheng Y, Li J, Liang C, Chai R, Chen Q, Yu FR (2024) Online convex optimization for resource allocation scheme in edge computing-enabled networks. In: IEEE Wireless Communications and Networking Conference (WCNC), Dubai, United Arab Emirates: IEEE, pp. 1–6. https://doi.org/10.1109/WCNC57260.2024.10571274

Chu W, Yu P, Yu Z, Lui JCS, Lin Y (2023) Online optimal service selection, resource allocation and task offloading for multi-access edge computing: a utility-based approach. IEEE Trans Mob Comput 22(7):4150–4167. https://doi.org/10.1109/TMC.2022.3152493

Cong R, Zhao Z, Min G, Feng C, Jiang Y (2022) EdgeGO: a mobile resource-sharing framework for 6G edge computing in massive IoT systems. IEEE Internet Things J 9(16):14521–14529. https://doi.org/10.1109/JIOT.2021.3065357

Coutinho EF, De Carvalho Sousa FR, Rego PAL, Gomes DG, De Souza JN (2015) Elasticity in cloud computing: a survey. Ann Telecommun 70(7–8):289–309. https://doi.org/10.1007/s12243-014-0450-7

Coutinho A, Greve F, Prazeres C, Cardoso J (2018) Fogbed: A rapid-prototyping emulation environment for fog computing. In: IEEE International Conference on Communications (ICC). IEEE, Kansas City, MO. pp 1–7. https://doi.org/10.1109/ICC.2018.8423003

Cruz P, Achir N, Viana AC (May 2023) On the Edge of the deployment: a survey on multi-access edge computing. ACM Comput Surv 55(5):1–34. https://doi.org/10.1145/3529758

Cui G, He Q, Chen F, Jin H, Xiang Y, Yang Y (2021) Location privacy protection via delocalization in 5G mobile edge computing environment. IEEE Trans Serv Comput 1–1. https://doi.org/10.1109/TSC.2021.3112659

Da JBD, Costa et al (2023) Mobility and deadline-aware task scheduling mechanism for vehicular edge computing. IEEE Trans Intell Transport Syst 24(10):11345–11359. https://doi.org/10.1109/TITS.2023.3276823

Dai X et al (2023) Task co-offloading for D2D-Assisted mobile edge computing in industrial Internet of Things. IEEE Trans Ind Inf 19(1):480–490. https://doi.org/10.1109/TII.2022.3158974

Deng S et al (2021) Optimal application deployment in resource constrained distributed edges. IEEE Trans Mob Comput 20(5):1907–1923. https://doi.org/10.1109/TMC.2020.2970698

Deng X, Sun Z, Li D, Luo J, Wan S (2021) User-centric computation offloading for edge computing. IEEE Internet Things J. 8(16):12559–12568. https://doi.org/10.1109/JIOT.2021.3057694

Diao B et al (2019) A Scalable Testbed for Task Offloading and Deployment of Heterogeneous Edge Computing. In: IEEE International Conferences on Ubiquitous Computing & Communications (IUCC) and Data Science and Computational Intelligence (DSCI) and Smart Computing, Networking and Services (SmartCNS), Shenyang, China: IEEE, Oct. 2019, pp 586–591. https://doi.org/10.1109/IUCC/DSCI/SmartCNS.2019.00123

Duan Z et al (2022) A novel load balancing scheme for mobile edge computing. J Syst Softw 186:111195. https://doi.org/10.1016/j.jss.2021.111195

Duplyakin D et al (2019) The design and operation of CloudLab

Escamilla-Ambrosio PJ, Rodríguez-Mota A, Aguirre-Anaya E, Acosta-Bermejo R, Salinas-Rosales M (2018) Distributing computing in the Internet of Things: cloud, fog and edge computing overview, in NEO 2016. In: Maldonado Y, Trujillo L, Schütze O, Riccardi A, Vasile M (Eds) Studies in computational intelligence, vol. 731. Springer: Cham. pp 87–115. https://doi.org/10.1007/978-3-319-64063-1_4

Fan W et al (2024) Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation. IEEE Trans Mob Comput 23(1):238–256. https://doi.org/10.1109/TMC.2022.3219261

Fazeldehkordi E, Grønli T-M (2022) A survey of security architectures for edge computing-based IoT. IoT 3(3):332–365. https://doi.org/10.3390/iot3030019

Feng H, Guo S, Yang L, Yang Y (2021) Collaborative Data caching and computation offloading for multi-service mobile edge computing. IEEE Trans Veh Technol 70(9):9408–9422. https://doi.org/10.1109/TVT.2021.3099303

Feng C, Han P, Zhang X, Yang B, Liu Y, Guo L (2022) Computation offloading in mobile edge computing networks: a survey. J Netw Comput Appl 202:103366. https://doi.org/10.1016/j.jnca.2022.103366

Ferrer AJ, Marques JM, Jorba J (2019) Ad-Hoc edge cloud: a framework for dynamic creation of edge computing infrastructures. In: 28th International Conference on Computer Communication and Networks (ICCCN). IEEE, Valencia, Spain. pp 1–7. https://doi.org/10.1109/ICCCN.2019.8847142

Gao B, Zhou Z, Liu F, Xu F, Li B (2022) An online framework for joint network selection and service placement in mobile edge computing. IEEE Trans Mob Comput 21(11):3836–3851. https://doi.org/10.1109/TMC.2021.3064847

Gao M, Shen R, Shi L, Qi W, Li J, Li Y (2023) Task partitioning and offloading in DNN-task enabled mobile edge computing networks. IEEE Trans Mob Comput 22(4): 2435–2445. https://doi.org/10.1109/TMC.2021.3114193

Gao H, Huang W, Liu T, Yin Y, Li Y (2023) PPO2: Location privacy-oriented task offloading to edge computing using reinforcement learning for intelligent autonomous transport systems. IEEE Trans Intell Transport Syst 24(7):7599–7612. https://doi.org/10.1109/TITS.2022.3169421

Gao Z, Yang L, Dai Y (2023) Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing. IEEE Trans Mobile Comput 22(6):3425–3443. https://doi.org/10.1109/TMC.2022.3141080

Gill KS (2017) Erratum to: Hermeneutic of performing knowledge. AI Soc. https://doi.org/10.1007/s00146-017-0725-4

Guo Y, Zhao R, Lai S, Fan L, Lei X, Karagiannidis GK (2022) Distributed machine learning for multiuser mobile edge computing systems. IEEE J Sel Top Signal Process 16(3):460–473. https://doi.org/10.1109/JSTSP.2022.3140660

Guo F, Tang B, Tang M (2022) Joint optimization of delay and cost for microservice composition in mobile edge computing. World Wide Web 25(5):2019–2047. https://doi.org/10.1007/s11280-022-01017-2

Hao Y, Cao J, Wang Q, Du J (2021) Energy-aware scheduling in edge computing with a clustering method. Future Gener Comput Syst 117:259–272. https://doi.org/10.1016/j.future.2020.11.029

He X, Wang S, Wang X, Xu S, Ren J (2022) Age-based scheduling for monitoring and control applications in mobile edge computing systems. In: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications. IEEE, London, United Kingdom. pp 1009–1018. https://doi.org/10.1109/INFOCOM48880.2022.9796654

Hosseinzadeh M et al (2021) Improved butterfly optimization algorithm for data placement and scheduling in edge computing environments. J Grid Comput 19(2):14. https://doi.org/10.1007/s10723-021-09556-0

Hu S, Huang S, Huang J, Su J (Mar. 2021) Blockchain and edge computing technology enabling organic agricultural supply chain: a framework solution to trust crisis. Comput Ind Eng 153:107079. https://doi.org/10.1016/j.cie.2020.107079

Hua H, Li Y, Wang T, Dong N, Li W, Cao J (2023) Edge computing with Artificial Intelligence: a machine learning perspective. ACM Comput Surv 55(9):1–35. https://doi.org/10.1145/3555802

Huang X, Yu R, Ye D, Shu L, Xie S (2021) Efficient workload allocation and user-centric utility maximization for task scheduling in collaborative vehicular edge computing. IEEE Trans Veh Technol 70(4):3773–3787. https://doi.org/10.1109/TVT.2021.3064426

Huang J, Zhou A, Wang S (2022) Price-aware service deployment in hierarchical mobile-edge computing. IEEE Internet Things J 9(13):11533–11541. https://doi.org/10.1109/JIOT.2021.3130409

Huda SMA, Moh S (2022) Survey on computation offloading in UAV-Enabled mobile edge computing. J Netw Comput Appl 201:103341. https://doi.org/10.1016/j.jnca.2022.103341

Hui Y et al (2022) Secure and personalized edge computing services in 6g heterogeneous vehicular networks. IEEE Internet Things J 9(8):5920–5931. https://doi.org/10.1109/JIOT.2021.3065970

Iftikhar S et al (2023) AI-based fog and edge computing: a systematic review, taxonomy and future directions. Internet Things 21:100674. https://doi.org/10.1016/j.iot.2022.100674

Irshad A, Chaudhry SA, Ghani A, Mallah GA, Bilal M, Alzahrani BA (2022) A low-cost privacy preserving user access in mobile edge computing framework. Comput Electr Eng 98:107692. https://doi.org/10.1016/j.compeleceng.2022.107692

Jayanetti A, Halgamuge S, Buyya R (2022) Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments. Future Gener Comput Syst 137:14–30. https://doi.org/10.1016/j.future.2022.06.012

Jia X, Luo M, Choo K-KR, Li L, He D (2022) A redesigned identity-based anonymous authentication scheme for mobile-edge computing. IEEE Internet Things J 9(12):10108–10120. https://doi.org/10.1109/JIOT.2021.3119517

Jia Y, Zhang C, Huang Y, Zhang W (2022) Lyapunov optimization based mobile edge computing for internet of vehicles systems. IEEE Trans Commun 70(11):7418–7433. https://doi.org/10.1109/TCOMM.2022.3206885

Kaur G, Batth RS (2021) Edge computing: classification, applications, and challenges. In 2nd International Conference on Intelligent Engineering and Management (ICIEM), London, United Kingdom. IEEE, 2021, pp 254–259. https://doi.org/10.1109/ICIEM51511.2021.9445331

Kiani A, Ansari N (2018) Edge computing aware NOMA for 5G networks. IEEE Internet Things J 5(2):1299–1306. https://doi.org/10.1109/JIOT.2018.2796542

Kong X et al (2022) Deep reinforcement learning-based energy-efficient edge computing for internet of vehicles. IEEE Trans Ind Inf 18(9):6308–6316. https://doi.org/10.1109/TII.2022.3155162

Kong X, Wu Y, Wang H, Xia F (2022) Edge computing for internet of everything: a survey. IEEE Internet Things J 9(23):23472–23485. https://doi.org/10.1109/JIOT.2022.3200431

Kong L et al (2023) Edge-computing-driven Internet of Things: a survey. ACM Comput Surv 55(8):1–41. https://doi.org/10.1145/3555308

Lan D et al (2022) Task partitioning and orchestration on heterogeneous edge platforms: the case of vision applications. IEEE Internet Things J 9(10):7418–7432. https://doi.org/10.1109/JIOT.2022.3153970
Li M et al (2020) Energy-efficient UAV-assisted mobile edge computing: resource allocation and trajectory optimization. IEEE Trans Veh Technol 69(3):3424–3438. https://doi.org/10.1109/TVT.2020.2968343
Li Y, Cheng Q, Liu X, Li X (2021) A secure anonymous identity-based scheme in new authentication architecture for mobile edge computing. IEEE Syst J 15(1):935–946. https://doi.org/10.1109/JSYST.2020.2979006
Li C, Zhang Y, Gao X, Luo Y (Aug. 2022) Energy-latency tradeoffs for edge caching and dynamic service migration based on DQN in mobile edge computing. J Parallel Distrib Comput 166:15–31. https://doi.org/10.1016/j.jpdc.2022.03.001
Li M, Lei H, Guo H, Sulaiman R, Deebani W, Shutaywi M (2023) Efficient data offloading using Markovian decision on state reward action in edge computing. J Grid Comput 21(2):25. https://doi.org/10.1007/s10723-023-09659-w
Liang J, Li K, Liu C, Li K (Feb. 2021) Joint offloading and scheduling decisions for DAG applications in mobile edge computing. Neurocomputing 424:160–171. https://doi.org/10.1016/j.neucom.2019.11.081
Liao H et al (2020) Learning-based context-aware resource allocation for edge-computing-empowered industrial IoT. IEEE Internet Things J 7(5):4260–4277. https://doi.org/10.1109/JIOT.2019.2963371
Liao L, Lai Y, Yang F, Zeng W (2023) Online computation offloading with double reinforcement learning algorithm in mobile edge computing. J Parallel Distrib Comput 171:28–39. https://doi.org/10.1016/j.jpdc.2022.09.006
Lin H, Zeadally S, Chen Z, Labiod H, Wang L (2020) A survey on computation offloading modeling for edge computing. J Netw Comput Appl 169:102781. https://doi.org/10.1016/j.jnca.2020.102781
Lin J, Huang L, Zhang H, Yang X, Zhao P (2022) A novel lyapunov based dynamic resource allocation for uavs-assisted edge computing. Comput Netw 205:108710. https://doi.org/10.1016/j.comnet.2021.108710
Liu J et al (2022) Reliability-enhanced task offloading in mobile edge computing environments. IEEE Internet Things J 9(13):10382–10396. https://doi.org/10.1109/JIOT.2021.3115807
Liu S, Yu J, Deng X, Wan S (2022) FedCPF: An efficient-communication federated learning approach for vehicular edge computing in 6G communication networks. IEEE Trans Intell Transport Syst 23(2):1616–1629. https://doi.org/10.1109/TITS.2021.3099368
Liu Y, Liu C, Liu J, Hu Y, Li K, Li K (Jun. 2022) Mobility-aware and code-oriented partitioning computation offloading in multi-access edge computing. J Grid Comput 20(2). https://doi.org/10.1007/s10723-022-09599-x
Liu D, Zhang Y, Jia D, Zhang Q, Zhao X, Rong H (2022) Toward secure distributed data storage with error locating in blockchain enabled edge computing. Comput Stand Interfaces 79:103560. https://doi.org/10.1016/j.csi.2021.103560
Liu J et al (2023) Adaptive asynchronous federated learning in resource-constrained edge computing. IEEE Trans Mob Comput 22(2):674–690. https://doi.org/10.1109/TMC.2021.3096846
Liu L, Feng J, Mu X, Pei Q, Lan D, Xiao M (2023) Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing. IEEE Trans Intell Transport Syst 24(12):15513–15526. https://doi.org/10.1109/TITS.2023.3249745
Liu F, Huang J, Wang X (2023) Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies. IEEE Trans Cloud Comput 11(3):3027–3039. https://doi.org/10.1109/TCC.2023.3251561
Luo Q, Hu S, Li C, Li G, Shi W (2021) Resource scheduling in edge computing: a survey. IEEE Commun Surv Tutorials 23(4):2131–2165. https://doi.org/10.1109/COMST.2021.3106401
Luo Q, Li C, Luan TH, Shi W (2022) Minimizing the delay and cost of computation offloading for vehicular edge computing. IEEE Trans Serv Comput 15(5):2897–2909. https://doi.org/10.1109/TSC.2021.3064579
Lyu X et al (2017) Optimal schedule of mobile edge computing for Internet of Things using partial information. IEEE J Select Areas Commun 35(11):2606–2615. https://doi.org/10.1109/JSAC.2017.2760186
Ma C, Zhu J, Liu M, Zhao H, Liu N, Zou X (2021) Parking edge computing: parked-vehicle-assisted task offloading for urban VANETs. IEEE Internet Things J 8(11):9344–9358. https://doi.org/10.1109/JIOT.2021.3056396
Ma X, Zhou A, Zhang S, Li Q, Liu AX, Wang S (2023) Dynamic task scheduling in cloud-assisted mobile edge computing. IEEE Trans Mob Comput 22(4):2116–2130. https://doi.org/10.1109/TMC.2021.3115262
Madni SHH, Latiff MSA, Coulibaly Y, Abdulhamid SM (2017) Recent advancements in resource allocation techniques for cloud computing environment: a systematic review. Clust Comput 20(3):2489–2533. https://doi.org/10.1007/s10586-016-0684-4

Mahenge MPJ, Li C, Sanga CA (2022) Energy-efficient task offloading strategy in mobile edge computing for resource-intensive mobile applications. Digit Commun Netw 8(6):1048–1058. https://doi.org/10.1016/j.dcan.2022.04.001

Mao Y, You C, Zhang J, Huang K, Letaief KB (2017) A survey on mobile edge computing: the communication perspective. IEEE Commun Surv Tutor 19(4):2322–2358. https://doi.org/10.1109/COMST.2017.2745201

Maray M, Shuja J (2022) Computation offloading in mobile cloud computing and mobile edge computing: survey, taxonomy, and open issues. Mob Inf Syst 2022:1–17. https://doi.org/10.1155/2022/1121822

Mavromatis A, Simeonidou D (2020) Experiences from building a multi-access edge computing Internet of Things testbed. In: 2020 European Conference on Networks and Communications (EuCNC). IEEE, Dubrovnik, Croatia. pp 259–264. https://doi.org/10.1109/EuCNC48522.2020.9200924

Mayer R, Graser L, Gupta H, Saurez E, Ramachandran U (2017) EmuFog: extensible and scalable emulation of large-scale fog computing infrastructures. In: 2017 IEEE Fog World Congress (FWC). IEEE, Santa Clara, CA, pp 1–6. https://doi.org/10.1109/FWC.2017.8368525.

Mekala MS et al (2022) A DRL-based service offloading approach using DAG for edge computational orchestration. IEEE Trans Comput Soc Syst 1–9. https://doi.org/10.1109/TCSS.2022.3161627

Miao Z, Yong P, Jiancheng Z, Quanjun Y (2022) Efficient flow-based scheduling for geo-distributed simulation tasks in collaborative edge and cloud environments. IEEE Trans Parallel Distrib Syst 33(12):3442–3459. https://doi.org/10.1109/TPDS.2022.3155713

Miao Y, Hwang K, Wu D, Hao Y, Chen M (2023) Drone swarm path planning for mobile edge computing in industrial Internet of Things. IEEE Trans Ind Inf 19(5):6836–6848. https://doi.org/10.1109/TII.2022.3196392

Mitsis G, Tsiropoulou EE, Papavassiliou S (2022) Price and risk awareness for data offloading decision-making in edge computing systems. IEEE Syst J 16(4):6546–6557. https://doi.org/10.1109/JSYST.2022.3188997

Moura J, Hutchison D (2019) Game theory for multi-access edge computing: survey, use cases, and future trends. IEEE Commun Surv Tutor 21(1):260–288. https://doi.org/10.1109/COMST.2018.2863030

Muñoz MO, Mostafavi SS, Moothedath VN, Gross J (2022) Ainur: A framework for repeatable end-to-end wireless edge computing testbed research

Naouri A, Wu H, Nouri NA, Dhelim S, Ning H (2021) A Novel framework for mobile-edge computing by optimizing task offloading. IEEE Internet Things J 8(16):13065–13076. https://doi.org/10.1109/JIOT.2021.3064225

Nayyer MZ, Raza I, Hussain SA (2019) A survey of cloudlet-based mobile augmentation approaches for resource optimization, ACM Comput Surv 51(5):1–28. https://doi.org/10.1145/3241738

Nayyer MZ, Raza I, Hussain SA (2020) CFRO: cloudlet federation for resource optimization. IEEE Access 8:106234–106246. https://doi.org/10.1109/ACCESS.2020.2999938

Nayyer MZ et al (2022) LBRO: load balancing for resource optimization in edge computing. IEEE Access 10:97439–97449. https://doi.org/10.1109/ACCESS.2022.3205741

Nezami Z, Pournaras E, Borzouie A, Xu J (2023) SMOTEC: an edge computing testbed for adaptive smart mobility experimentation. In: IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). IEEE: Toronto, ON, Canada. pp 1–7. https://doi.org/10.1109/ACSOS-C58168.2023.00021

Nguyen DC et al (2021) Federated learning meets blockchain in edge computing: opportunities and challenges. IEEE Internet Things J 8(16):12806–12825. https://doi.org/10.1109/JIOT.2021.3072611

Ning Z et al (2021) Distributed and dynamic service placement in pervasive edge computing networks. IEEE Trans Parallel Distrib Syst 32(6):1277–1292. https://doi.org/10.1109/TPDS.2020.3046000

Ning Z et al (2023) Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing. IEEE Trans Mob Comput 22(5):2628–2644. https://doi.org/10.1109/TMC.2021.3129785

Ning Z et al (2024) Mobile edge computing and machine learning in the internet of unmanned aerial vehicles: a survey. ACM Comput Surv 56(1):1–31. https://doi.org/10.1145/3604933

Pang X, Wang Z, Li J, Zhou R, Ren J, Li Z (2022) Towards online privacy-preserving computation offloading in mobile edge computing. In: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications. IEEE, London, United Kingdom. pp 1179–1188. https://doi.org/10.1109/INFOCOM48880.2022.9796748

Pepito R, Dutta A (2021) Open source 5G security testbed for Edge Computing. In: 2021 IEEE 4th 5G World Forum (5GWF). IEEE, Montreal, QC, Canada, pp 388–393. https://doi.org/10.1109/5GWF52925.2021.00075.

Pu L, Chen X, Xu J, Fu X (2016) D2D Fogging: an energy-efficient and incentive-aware task offloading framework via network-assisted D2D collaboration. IEEE J Select Areas Commun 34(12):3887–3901. https://doi.org/10.1109/JSAC.2016.2624118

Puterman ML (1990) Markov decision processes

Qu X, Hu Q, Wang S (2020) Privacy-preserving model training architecture for intelligent edge computing. Comput Commun 162:94–101. https://doi.org/10.1016/j.comcom.2020.07.045

Raeisi-Varzaneh M, Dakkak O, Habbal A, Kim B-S (2023) Resource scheduling in edge computing: architecture, taxonomy, open issues and future research directions. IEEE Access 11:25329–25350. https://doi.org/10.1109/ACCESS.2023.3256522

Ranaweera P, Jurcut AD, Liyanage M (2021) Survey on multi-access edge computing security and privacy. IEEE Commun Surv Tutorials 23(2):1078–1124. https://doi.org/10.1109/COMST.2021.3062546

Ren J, Yu G, He Y, Li GY (May 2019) Collaborative cloud and edge computing for latency minimization. IEEE Trans Veh Technol 68(5):5031–5044. https://doi.org/10.1109/TVT.2019.2904244

Sadatdiynov K, Cui L, Zhang L, Huang JZ, Salloum S, Mahmud MS (2023) A review of optimization methods for computation offloading in edge computing networks. Digit Commun Netw 9(2):450–461. https://doi.org/10.1016/j.dcan.2022.03.003

Şenel BC, Mouchet M, Cappos J, Fourmaux O, Friedman T, McGeer R (2021) EdgeNet: a multi-tenant and multi-provider edge cloud. In: Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking, Online United Kingdom: ACM. pp 49–54. https://doi.org/10.1145/3434770.3459737

Shi W, Cao J, Zhang Q, Li Y, Xu L (2016) Edge computing: vision and challenges. IEEE Internet Things J 3(5):637–646. https://doi.org/10.1109/JIOT.2016.2579198

Shi W, Pallis G, Xu Z (2019) Edge computing [Scanning the Issue]. Proc IEEE 107(8):1474–1481. https://doi.org/10.1109/JPROC.2019.2928287

Singh S, Chana I, Singh M, Buyya R (2016) Self-optimization of energy-efficient cloud resources. Clust Comput 19(4):1787–1800. https://doi.org/10.1007/s10586-016-0623-4

Sohrabi MK, Azgomi H (2020) A survey on the combined use of optimization methods and game theory. Arch Computat Methods Eng 27(1):59–80. https://doi.org/10.1007/s11831-018-9300-5

Sun W, Liu J, Yue Y, Zhang H (2018) Double auction-based resource allocation for mobile edge computing in industrial internet of things. IEEE Trans Ind Inf 14(10): 4692–4701. https://doi.org/10.1109/TII.2018.2855746

Sun F, Zhang Z, Zeadally S, Han G, Tong S (2022) Edge computing-enabled internet of vehicles: towards federated learning empowered scheduling. IEEE Trans Veh Technol 71(9):10088–10103. https://doi.org/10.1109/TVT.2022.3182782

Tang Q et al (2022) Distributed task scheduling in serverless edge computing networks for the internet of things: a learning approach. IEEE Internet Things J 9(20):19634–19648. https://doi.org/10.1109/JIOT.2022.3167417

Tang B, Guo F, Cao B, Tang M, Li K (2023) Cost-aware deployment of microservices for IoT applications in mobile edge computing environment. IEEE Trans Netw Serv Manag 20(3):3119–3134. https://doi.org/10.1109/TNSM.2022.3232503

Teng H, Li Z, Cao K, Long S, Guo S, Liu A (2022) Game theoretical Task offloading for profit maximization in mobile edge computing. IEEE Trans Mob Comput 1–1. https://doi.org/10.1109/TMC.2022.3175218

Till SFTSM, Beck MW, Feld S, Schimper T (2014) Mobile edge computing: a taxonomy. In: Advances in Future Internet (AFIN 2014), The Sixth International Conference

Tütüncüoğlu F, Dán G (2024) Optimal service caching and pricing in edge computing: a Bayesian Gaussian process bandit approach. IEEE Trans Mob Comput 23(1):705–718. https://doi.org/10.1109/TMC.2022.3221465

Wang J, Wang L (2021) Resource allocation optimization strategy for massive internet of health things devices considering privacy protection in cloud edge computing environment. J Grid Comput 19(2):17. https://doi.org/10.1007/s10723-021-09558-y

Wang W, Huang H, Xue L, Li Q, Malekian R, Zhang Y (2021) Blockchain-assisted handover authentication for intelligent telehealth in multi-server edge computing environment. J Syst Architect 115:102024. https://doi.org/10.1016/j.sysarc.2021.102024

Wang P, Di B, Song L, Jennings NR (2022) Multi-layer computation offloading in distributed heterogeneous mobile edge computing networks. IEEE Trans Cogn Commun Netw 8(2):1301–1315. https://doi.org/10.1109/TCCN.2022.3161955

Wang W et al (2022) Privacy protection federated learning system based on blockchain and edge computing in mobile crowdsourcing. Comput Netw 215:109206. https://doi.org/10.1016/j.comnet.2022.109206

Wang Z, Zhang W, Jin X, Huang Y, Lu C (2022) An optimal edge server placement approach for cost reduction and load balancing in intelligent manufacturing. J Supercomput 78(3):4032–4056. https://doi.org/10.1007/s11227-021-04017-7

Wang X et al (2023) Wireless powered mobile edge computing networks: a survey. ACM Comput Surv 55(13):1–37. https://doi.org/10.1145/3579992
Wang T, Lu B, Wang W, Wei W, Yuan X, Li J (2023) Reinforcement learning-based optimization for mobile edge computing scheduling game. IEEE Trans Emerg Top Comput Intell 7(1):55–64. https://doi.org/10.1109/TETCI.2022.3145694
Xiao D, Li M, Zheng H (2020) Smart privacy protection for big video data storage based on hierarchical edge computing. Sensors 20(5):1517. https://doi.org/10.3390/s20051517
Xiao X et al (2022) Novel workload-aware approach to mobile user reallocation in crowded mobile edge computing environment. IEEE Trans Intell Transport Syst 23(7): 8846–8856. https://doi.org/10.1109/TITS.2021.3086827
Xiao H, Zhao J, Pei Q, Feng J, Liu L, Shi W (2022) Vehicle selection and resource optimization for federated learning in vehicular edge computing. IEEE Trans Intell Transport Syst 23(8):11073–11087. https://doi.org/10.1109/TITS.2021.3099597
Xu X et al (2019) An edge computing-enabled computation offloading method with privacy preservation for internet of connected vehicles. Future Gener Comput Syst 96:89–100. https://doi.org/10.1016/j.future.2019.01.012
Xu J, Xu Z, Shi B (2022) Deep reinforcement learning based resource allocation strategy in cloud-edge computing system. Front Bioeng Biotechnol 10:908056. https://doi.org/10.3389/fbioe.2022.908056
Xue H, Chen D, Zhang N, Dai H-N, Yu K (2023) Integration of blockchain and edge computing in internet of things: A survey. Future Gener Comput Syst 144:307–326. https://doi.org/10.1016/j.future.2022.10.029
Yamanaka H, Teranishi Y, Kawai E, Nagano H, Harai H (2021) Design of an edge computing testbed to simplify experimental setup. In: 24th International Symposium on Wireless Personal Multimedia Communications (WPMC). IEEE: Okayama, Japan. pp 1–6. https://doi.org/10.1109/WPMC52694.2021.9700447
Yan J, Bi S, Duan L, Zhang Y-JA (2021) Pricing-driven service caching and task offloading in mobile edge computing. IEEE Trans Wirel Commun 20(7):4495–4512. https://doi.org/10.1109/TWC.2021.3059692
Yang L, Yao H, Wang J, Jiang C, Benslimane A, Liu Y (2020) Multi-UAV-enabled load-balance mobile-edge computing for IoT networks. IEEE Internet Things J 7(8):6898–6908. https://doi.org/10.1109/JIOT.2020.2971645
Yu W et al (2018) A Survey on the Edge Computing for the internet of things. IEEE Access 6:6900–6919. https://doi.org/10.1109/ACCESS.2017.2778504
Zhang Z, Zeng F (2023) Efficient task allocation for computation offloading in vehicular edge computing. IEEE Internet Things J 10(6):5595–5606. https://doi.org/10.1109/JIOT.2022.3222408
Zhang G, Zhang W, Cao Y, Li D, Wang L (2018) Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices. IEEE Trans Ind Inf 14(10):4642–4655. https://doi.org/10.1109/TII.2018.2843365
Zhang P, Durresi M, Durresi A (2021) Internet network location privacy protection with multi-access edge computing. Computing 103(3):473–490. https://doi.org/10.1007/s00607-020-00860-3
Zhang G, Zhang S, Zhang W, Shen Z, Wang L (2021) Joint Service caching, computation offloading and resource allocation in mobile edge computing systems. IEEE Trans Wirel Commun 20(8):5288–5300. https://doi.org/10.1109/TWC.2021.3066650
Zhang J, Zhou X, Ge T, Wang X, Hwang T (2021) Joint task scheduling and containerizing for efficient edge computing. IEEE Trans Parallel Distrib Syst 32(8):2086–2100. https://doi.org/10.1109/TPDS.2021.3059447
Zhang K, Cao J, Zhang Y (2022) Adaptive digital twin and multiagent deep reinforcement learning for vehicular edge computing and networks. IEEE Trans Ind Inf 18(2):1405–1413. https://doi.org/10.1109/TII.2021.3088407
Zhang P et al (2023) Deep reinforcement learning based computation offloading in UAV-assisted edge computing. Drones 7(3):213. https://doi.org/10.3390/drones7030213
Zhao L et al (2021) Vehicular computation offloading for industrial mobile edge computing. IEEE Trans Ind Inf 17(11):7871–7881. https://doi.org/10.1109/TII.2021.3059640
Zhao F, Chen Y, Zhang Y, Liu Z, Chen X (2021) Dynamic offloading and resource scheduling for mobile-edge computing with energy harvesting devices. IEEE Trans Netw Serv Manag 18(2):2154–2165. https://doi.org/10.1109/TNSM.2021.3069993
Zhao N, Ye Z, Pei Y, Liang Y-C, Niyato D (2022) Multi-agent deep reinforcement learning for task offloading in uav-assisted mobile edge computing. IEEE Trans Wirel Commun 21(9):6949–6960. https://doi.org/10.1109/TWC.2022.3153316
Zhao X, Huang G, Jiang J, Gao L, Li M (2022) Task offloading of cooperative intrusion detection system based on deep Q network in mobile edge computing. Expert Syst Appl 206:117860. https://doi.org/10.1016/j.eswa.2022.117860

Zhao L et al (2023) A digital twin-assisted intelligent partial offloading approach for vehicular edge computing. IEEE J Select Areas Commun 41(11):3386–3400. https://doi.org/10.1109/JSAC.2023.3310062

Zhong X, Wang X, Yang T, Yang Y, Qin Y, Ma X (2022) POTAM: A parallel optimal task allocation mechanism for large-scale delay sensitive mobile edge computing. IEEE Trans Commun 70(4):2499–2517. https://doi.org/10.1109/TCOMM.2022.3151064

Zhou H, Wang Z, Cheng N, Zeng D, Fan P (2022) Stackelberg-game-based computation offloading method in cloud–edge computing networks. IEEE Internet Things J 9(17):16510–16520. https://doi.org/10.1109/JIOT.2022.3153089

Zhou H, Wang Z, Min G, Zhang H (2023) UAV-aided computation offloading in mobile-edge computing networks: a Stackelberg game approach. IEEE Internet Things J 10(8):6622–6633. https://doi.org/10.1109/JIOT.2022.3197155

Zhou H, Wu T, Chen X, He S, Guo D, Wu J (2023) Reverse auction-based computation offloading and resource allocation in mobile cloud-edge computing. IEEE Trans Mob Comput 22(10):6144–6159. https://doi.org/10.1109/TMC.2022.3189050

Zhou H, Zhang Z, Li D, Su Z (2023) Joint optimization of computing offloading and service caching in edge computing-based smart grid. IEEE Trans Cloud Comput 11(2):1122–1132. https://doi.org/10.1109/TCC.2022.3163750

Zhu X, Zhou M (2021) Multiobjective optimized cloudlet deployment and task offloading for mobile-edge computing. IEEE Internet Things J 8(20):15582–15595. https://doi.org/10.1109/JIOT.2021.3073113

Zhu Y, Mao B, Kato N (2022) A dynamic task scheduling strategy for multi-access edge computing in IRS-aided vehicular networks. IEEE Trans Emerg Top Comput 10(4):1761–1771. https://doi.org/10.1109/TETC.2022.3153494

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.